

Real-Time Systems Series

Hermann Kopetz

Real-Time Systems

Design Principles for Distributed
Embedded Applications

Second Edition

 Springer

Real-Time Systems

Second Edition

Real-Time Systems Series

Series Editor

John A. Stankovic

University of Virginia, Virginia, USA

For further volumes:

<http://www.springer.com/series/6941>

Hermann Kopetz

Real-Time Systems

Design Principles for Distributed
Embedded Applications

Second Edition

 Springer

Hermann Kopetz
Vienna University of Technology
Department of Computer Engineering
Real Time Systems Group
Treitlstrasse 3, 3rd floor
1040 Wien, Austria
hk@vmars.tuwien.ac.at

ISSN 1867-321X e-ISSN 1867-3228
ISBN 978-1-4419-8236-0 e-ISBN 978-1-4419-8237-7
DOI 10.1007/978-1-4419-8237-7
Springer New York Dordrecht Heidelberg London

Library of Congress Control Number: 2011925551

© Springer Science+Business Media, LLC 2011

All rights reserved. This work may not be translated or copied in whole or in part without the written permission of the publisher (Springer Science+Business Media, LLC, 233 Spring Street, New York, NY 10013, USA), except for brief excerpts in connection with reviews or scholarly analysis. Use in connection with any form of information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed is forbidden.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Preface

The primary objective of this book is to serve as a textbook for students who take a senior undergraduate or a first-year graduate course on real-time embedded systems, also called *cyber-physical systems*. The structure of the book – the material is organized into 14 chapters – maps to the 14 weeks of a semester. The book is also intended for practitioners in industry who want to learn about the state of the art in real-time embedded system design and need a reference book that explains the fundamental concepts of the field. More than 1,000 students used the first edition of this book, published about 14 years ago, as a text for the real-time systems course at the Vienna University of Technology. The feedback from these students and many new developments in this dynamic field of embedded real-time systems have been incorporated in this fundamentally revised second edition of the book. The focus of the book is on the design of distributed real-time systems at the architecture level. While a significant part of the established computer science literature abstracts from the progression of real-time, real-time system designers cannot get away with such an abstraction. In this book, the progression of physical time is considered a first-order citizen that shapes many of the relevant concepts. The book explains the fundamental concepts related to the progression of time on a number of practical insightful examples from industry. The conceptual model of a distributed real-time distributed system has been extended and precise definitions of important time-related concepts, such as *sparse time*, *state*, *temporal accuracy of real-time data*, and *determinism* are given.

Since the evolving cognitive complexity of large computer systems is a topic of utmost concern, a new chapter on *simplicity* has been included in this second edition. This chapter builds on some of the recent insights from the field of cognition – concerning concept formation, understanding, human simplification strategies and model building – and formulates seven principles that lead to the design of *simple* systems. These principles are followed in the remaining 12 chapters of the book. The other two new chapters, one on *energy and power awareness*, and one on the *Internet of things* cover topics of increasing importance in the enormous market of mobile devices. The chapters on *communication*, *dependability*, *system design*, and *validation* have been substantially revised with

a focus on *component-based* and *model-based design*. The chapter on dependability includes new sections on *security* and *safety*. The final chapter describes the *time-triggered architecture* that integrates all presented concepts into a coherent framework for the development of dependable embedded real-time systems. Since the first edition of the book has been published, a visible paradigm shift from the event-triggered to the time-triggered design methodology for dependable distributed real-time systems has taken place in a number of applications.

It is assumed that the reader of this book has a background in basic computer science or computer engineering or has some practical experience in the design or implementation of embedded systems.

The glossary at the end of the book is an integral part of the book, providing definitions for many of the technical terms that are used throughout the book. If the reader is not sure about the meaning of a term, she/he is advised to refer to the glossary.

Acknowledgements

It is impossible to name all students, colleagues from industry and fellow scientists who have contributed to this second edition of the book by asking intelligent questions or making constructive comments over the last decade – thanks to all of you. In the final stages of finishing the manuscript of this second edition, in October 2010, I have given a course at Vanderbilt University, organized by Janos Sztipánovits, and got valuable comments from an unbiased audience. I am especially grateful to Christian Tessarek who did the artwork, and the following persons who have read part or all of the evolving manuscript and made many valuable suggestions for improvement: Sven Bunte, Christian El-Salloum, Bernhard Frömel, Oliver Höftberger, Herbert Grünbacher, Benedikt Huber, Albrecht Kadlec, Roland Kammerer, Susanne Kandl, Vaclav Mikolasek, Stefan Poledna, Peter Puschner, Brian Randell, Andreas Steininger, Ekarin Suethanuwong, Armin Wasicek, Michael Zolda, and the following students from Vanderbilt: Kyounggho An, Joshua D. Carl, Spencer Crosswy, Fred Eisele, Fan Qui, and Adam C. Trewyn.

Vienna, Austria
January 2011

Hermann Kopetz

Contents

1	The Real-Time Environment	1
1.1	When Is a Computer System Real-Time?	2
1.2	Functional Requirements	3
1.2.1	Data Collection.....	3
1.2.2	Direct Digital Control.....	5
1.2.3	Man–Machine Interaction.....	6
1.3	Temporal Requirements	6
1.3.1	Where Do Temporal Requirements Come from?	6
1.3.2	Minimal Latency Jitter.....	9
1.3.3	Minimal Error-Detection Latency	10
1.4	Dependability Requirements.....	10
1.4.1	Reliability	10
1.4.2	Safety.....	11
1.4.3	Maintainability	12
1.4.4	Availability.....	12
1.4.5	Security.....	13
1.5	Classification of Real-Time Systems	13
1.5.1	Hard Real-Time System Versus Soft Real-Time System	13
1.5.2	Fail-Safe Versus Fail-Operational	15
1.5.3	Guaranteed-Response Versus Best-Effort	16
1.5.4	Resource-Adequate Versus Resource-Inadequate	16
1.5.5	Event-Triggered Versus Time-Triggered.....	16
1.6	The Real-Time Systems Market	17
1.6.1	Embedded Real-Time Systems	18
1.6.2	Plant Automation Systems.....	20
1.6.3	Multimedia Systems	21

1.7	Examples of Real-Time Systems	22
1.7.1	Controlling the Flow in a Pipe.....	22
1.7.2	Engine Control	23
1.7.3	Rolling Mill	24
	Points to Remember.....	25
	Bibliographic Notes	26
	Review Questions and Problems.....	27
2	Simplicity.....	29
2.1	Cognition.....	30
2.1.1	Problem Solving	30
2.1.2	Definition of a Concept	32
2.1.3	Cognitive Complexity.....	33
2.1.4	Simplification Strategies	34
2.2	The Conceptual Landscape	35
2.2.1	Concept Formation.....	36
2.2.2	Scientific Concepts.....	37
2.2.3	The Concept of a Message.....	38
2.2.4	Semantic Content of a Variable	39
2.3	The Essence of Model Building	40
2.3.1	Purpose and Viewpoint	41
2.3.2	The Grand Challenge	42
2.4	Emergence	43
2.4.1	Irreducibility	43
2.4.2	Prior and Derived Properties.....	44
2.4.3	Complex Systems	44
2.5	How Can We Achieve Simplicity?.....	45
	Points to Remember.....	47
	Bibliographic Notes	49
	Review Questions and Problems.....	49
3	Global Time.....	51
3.1	Time and Order	52
3.1.1	Different Orders.....	52
3.1.2	Clocks	53
3.1.3	Precision and Accuracy	55
3.1.4	Time Standards.....	56
3.2	Time Measurement.....	57
3.2.1	Global Time.....	58
3.2.2	Interval Measurement.....	59
3.2.3	π/Δ -Precedence	60
3.2.4	Fundamental Limits of Time Measurement	61
3.3	Dense Time Versus Sparse Time	62
3.3.1	Dense Time-Base	63
3.3.2	Sparse Time-Base	64

3.3.3	Space-Time Lattice	64
3.3.4	Cyclic Representation of Time	65
3.4	Internal Clock Synchronization.....	66
3.4.1	The Synchronization Condition.....	66
3.4.2	Central Master Synchronization	68
3.4.3	Fault-Tolerant Synchronization Algorithms.....	69
3.4.4	State Correction Versus Rate Correction	72
3.5	External Clock Synchronization	73
3.5.1	External Time Sources.....	73
3.5.2	Time Gateway.....	74
3.5.3	Time Formats	75
	Points to Remember.....	76
	Bibliographic Notes	77
	Review Questions and Problems.....	77
4	Real-Time Model	79
4.1	Model Outline.....	80
4.1.1	Components and Messages	80
4.1.2	Cluster of Components.....	81
4.1.3	Temporal Control Versus Logical Control	82
4.1.4	Event-Triggered Control Versus Time-Triggered Control	84
4.2	Component State.....	84
4.2.1	Definition of State.....	84
4.2.2	The Pocket Calculator Example	85
4.2.3	Ground State	86
4.2.4	Database Components	88
4.3	The Message Concept.....	88
4.3.1	Message Structure.....	88
4.3.2	Event Information Versus State Information.....	89
4.3.3	Event-Triggered Message	90
4.3.4	Time-Triggered Message	91
4.4	Component Interfaces.....	92
4.4.1	Interface Characterization.....	93
4.4.2	Linking Interface.....	94
4.4.3	Technology Independent Control Interface.....	94
4.4.4	Technology Dependent Debug Interface	95
4.4.5	Local Interfaces	95
4.5	Gateway Component.....	95
4.5.1	Property Mismatches.....	96
4.5.2	LIF Versus Local Interface of a Gateway Component	97
4.5.3	Standardized Message Interface	98
4.6	Linking Interface Specification	99
4.6.1	Transport Specification	99
4.6.2	Operational Specification	100
4.6.3	Meta-Level Specification	101

4.7	Component Integration.....	102
4.7.1	Principles of Composability	102
4.7.2	Integration Viewpoints.....	104
4.7.3	System of Systems	104
	Points to Remember	107
	Bibliographic Notes	108
	Review Questions and Problems	109
5	Temporal Relations	111
5.1	Real-Time Entities	112
5.1.1	Sphere of Control	112
5.1.2	Discrete and Continuous Real-Time Entities.....	112
5.2	Observations	113
5.2.1	Untimed Observation	113
5.2.2	Indirect Observation	114
5.2.3	State Observation	114
5.2.4	Event Observation	114
5.3	Real-Time Images and Real-Time Objects.....	115
5.3.1	Real-Time Images.....	115
5.3.2	Real-Time Objects	116
5.4	Temporal Accuracy	116
5.4.1	Definition.....	116
5.4.2	Classification of Real-Time Images	119
5.4.3	State Estimation.....	120
5.4.4	Composability Considerations	121
5.5	Permanence and Idempotency	122
5.5.1	Permanence	122
5.5.2	Duration of the Action Delay.....	123
5.5.3	Accuracy Interval Versus Action Delay.....	124
5.5.4	Idempotency.....	124
5.6	Determinism.....	125
5.6.1	Definition of Determinism	125
5.6.2	Consistent Initial States	127
5.6.3	Non-deterministic Design Constructs.....	128
5.6.4	Recovery of Determinism.....	130
	Points to Remember	130
	Bibliographic Notes	132
	Review Questions and Problems	132
6	Dependability	135
6.1	Basic Concepts.....	136
6.1.1	Faults.....	136
6.1.2	Errors.....	138
6.1.3	Failures	139

6.2	Information Security	141
6.2.1	Secure Information Flow	142
6.2.2	Security Threats	143
6.2.3	Cryptographic Methods	145
6.2.4	Network Authentication.....	148
6.2.5	Protection of Real-Time Control Data.....	149
6.3	Anomaly Detection	150
6.3.1	What Is an Anomaly?.....	150
6.3.2	Failure Detection.....	151
6.3.3	Error Detection.....	152
6.4	Fault Tolerance.....	153
6.4.1	Fault Hypotheses.....	154
6.4.2	Fault-Tolerant Unit.....	155
6.4.3	The Membership Service	158
6.5	Robustness	160
6.5.1	The Concept of Robustness.....	160
6.5.2	Structure of a Robust System.....	160
6.6	Component Reintegration	161
6.6.1	Finding a Reintegration Point.....	162
6.6.2	Minimizing the Ground-State.....	162
6.6.3	Component Restart.....	163
	Points to Remember	164
	Bibliographic Notes	165
	Review Questions and Problems	166
7	Real-Time Communication	167
7.1	Requirements.....	168
7.1.1	Timeliness.....	168
7.1.2	Dependability	168
7.1.3	Flexibility	171
7.1.4	Physical Structure	171
7.2	Design Issues.....	172
7.2.1	A Waistline Communication Model.....	172
7.2.2	Physical Performance Limitation	174
7.2.3	Flow Control	174
7.2.4	Thrashing.....	176
7.3	Event-Triggered Communication.....	178
7.3.1	Ethernet	179
7.3.2	Controller Area Network	179
7.3.3	User Datagram Protocol	180
7.4	Rate-Constrained Communication.....	180
7.4.1	Token Protocol	181
7.4.2	Mini-slotting Protocol ARINC 629.....	181

7.4.3	Avionics Full Duplex Switched Ethernet.....	182
7.4.4	Audio Video Bus.....	182
7.5	Time-Triggered Communication	183
7.5.1	Time-Triggered Protocol.....	184
7.5.2	Time-Triggered Ethernet.....	186
7.5.3	FlexRay	187
	Points to Remember	187
	Bibliographic Notes	188
	Review Questions and Problems	188
8	Power and Energy Awareness.....	191
8.1	Power and Energy.....	192
8.1.1	Basic Concepts	192
8.1.2	Energy Estimation	193
8.1.3	Thermal Effects and Reliability	197
8.2	Hardware Power Reduction Techniques.....	200
8.2.1	Device Scaling	200
8.2.2	Low-Power Hardware Design	201
8.2.3	Voltage and Frequency Scaling.....	202
8.2.4	Sub-threshold Logic.....	202
8.3	System Architecture.....	203
8.3.1	Technology-Agnostic Design.....	203
8.3.2	Pollack's Rule.....	204
8.3.3	Power Gating.....	205
8.3.4	Real Time Versus Execution Time	206
8.4	Software Techniques.....	207
8.4.1	System Software	207
8.4.2	Application Software	208
8.4.3	Software Tools	209
8.5	Energy Sources.....	209
8.5.1	Batteries	209
8.5.2	Energy Harvesting	210
	Points to Remember	211
	Bibliographic Notes	212
	Review Questions and Problems	212
9	Real-Time Operating Systems.....	215
9.1	Inter-Component Communication	216
9.1.1	Technology Independent Interface	216
9.1.2	Linking Interface.....	216
9.1.3	Technology Dependent Debug Interface	217
9.1.4	Generic Middleware	217
9.2	Task Management.....	218
9.2.1	Simple Tasks	218

9.2.2	Trigger Tasks.....	220
9.2.3	Complex Tasks.....	220
9.3	The Dual Role of Time	221
9.3.1	Time as Data	222
9.3.2	Time as Control	223
9.4	Inter-task Interactions.....	223
9.4.1	Coordinated Static Schedules.....	224
9.4.2	The Non-blocking Write Protocol	224
9.4.3	Semaphore Operations	225
9.5	Process Input/Output.....	226
9.5.1	Analog Input/Output	226
9.5.2	Digital Input/Output.....	227
9.5.3	Interrupts	228
9.5.4	Fault-Tolerant Actuators.....	229
9.5.5	Intelligent Instrumentation.....	231
9.5.6	Physical Installation.....	232
9.6	Agreement Protocols.....	232
9.6.1	Raw Data, Measured Data, and Agreed Data	233
9.6.2	Syntactic Agreement.....	233
9.6.3	Semantic Agreement.....	233
9.7	Error Detection.....	234
9.7.1	Monitoring Task Execution Times	234
9.7.2	Monitoring Interrupts	235
9.7.3	Double Execution of Tasks	235
9.7.4	Watchdogs	235
	Points to Remember	235
	Bibliographic Notes	236
	Review Questions and Problems	237
10	Real-Time Scheduling.....	239
10.1	The Scheduling Problem	240
10.1.1	Classification of Scheduling Algorithms.....	240
10.1.2	Schedulability Test	241
10.1.3	The Adversary Argument.....	242
10.2	Worst-Case Execution Time.....	243
10.2.1	WCET of Simple Tasks	243
10.2.2	WCET of Complex Tasks	246
10.2.3	Anytime Algorithms.....	246
10.2.4	State of Practice	247
10.3	Static Scheduling	248
10.3.1	Static Scheduling Viewed as a Search	248
10.3.2	Increasing the Flexibility in Static Schedules.....	250
10.4	Dynamic Scheduling.....	251
10.4.1	Scheduling Independent Tasks	251
10.4.2	Scheduling Dependent Tasks.....	253

10.5	Alternative Scheduling Strategies	255
10.5.1	Scheduling in Distributed Systems	255
10.5.2	Feedback Scheduling	256
	Points to Remember	256
	Bibliographic Notes	257
	Review Questions and Problems	257
11	System Design	259
11.1	System Design	260
11.1.1	The Design Process	260
11.1.2	The Role of Constraints	261
11.1.3	System Design Versus Software Design	262
11.2	Design Phases	263
11.2.1	Purpose Analysis	264
11.2.2	Requirements Capture	264
11.2.3	Architecture Design	265
11.2.4	Design of Components	265
11.3	Design Styles	266
11.3.1	Model-Based Design	266
11.3.2	Component-Based Design	267
11.3.3	Architecture Design Languages	268
11.3.4	Test of a Decomposition	269
11.4	Design of Safety-Critical Systems	271
11.4.1	What Is Safety?	272
11.4.2	Safety Analysis	274
11.4.3	Safety Case	276
11.4.4	Safety Standards	279
11.5	Design Diversity	281
11.5.1	Diverse Software Versions	281
11.5.2	An Example of a Fail-Safe System	282
11.5.3	Multilevel System	283
11.6	Design for Maintainability	284
11.6.1	Cost of Maintenance	284
11.6.2	Maintenance Strategy	285
11.6.3	Software Maintenance	287
	Points to Remember	287
	Bibliographic Notes	289
	Review Questions and Problems	289
12	Validation	291
12.1	Validation Versus Verification	292
12.2	Testing Challenges	293
12.2.1	Design for Testability	293
12.2.2	Test Data Selection	294

12.2.3	Test Oracle.....	296
12.2.4	System Evolution.....	296
12.3	Testing of Component-Based Systems.....	297
12.3.1	Component Provider.....	297
12.3.2	Component User.....	298
12.3.3	Communicating Components.....	298
12.4	Formal Methods.....	299
12.4.1	Formal Methods in the Real World.....	299
12.4.2	Classification of Formal Methods.....	300
12.4.3	Benefits of Formal Methods.....	301
12.4.4	Model Checking.....	302
12.5	Fault Injection.....	302
12.5.1	Software-Implemented Fault Injection.....	303
12.5.2	Physical Fault Injection.....	303
12.5.3	Sensor and Actuator Failures.....	305
	Points to Remember.....	305
	Bibliographic Notes.....	306
	Review Questions and Problems.....	306
13	Internet of Things.....	307
13.1	The Vision of an Internet-of-Things.....	308
13.2	Drivers for an IoT.....	309
13.2.1	Uniformity of Access.....	309
13.2.2	Logistics.....	309
13.2.3	Energy Savings.....	310
13.2.4	Physical Security and Safety.....	310
13.2.5	Industrial.....	311
13.2.6	Medical.....	311
13.2.7	Life Style.....	311
13.3	Technical Issues of the IoT.....	311
13.3.1	Internet Integration.....	311
13.3.2	Naming and Identification.....	312
13.3.3	Near Field Communication.....	313
13.3.4	IoT Device Capabilities versus Cloud Computing.....	314
13.3.5	Autonomic Components.....	314
13.4	RFID Technology.....	315
13.4.1	Overview.....	316
13.4.2	The Electronic Product Code.....	316
13.4.3	RFID Tags.....	317
13.4.4	RFID Readers.....	318
13.4.5	RFID Security.....	318
13.5	Wireless Sensor Networks.....	320
	Points to Remember.....	321
	Bibliographic Notes.....	322
	Review Questions and Problems.....	322

14	The Time-Triggered Architecture	325
14.1	History of the TTA	326
14.1.1	The MARS Project	326
14.1.2	The Industrial TTA Prototype	327
14.1.3	The GENESYS Project	327
14.2	Architectural Style	328
14.2.1	Complexity Management	328
14.2.2	Component Orientation	329
14.2.3	Coherent Communication	330
14.2.4	Dependability	331
14.2.5	Time Aware Architecture	332
14.3	Services of the TTA	332
14.3.1	Component-Based Services	332
14.3.2	Core System Services	333
14.3.3	Optional System Services	334
14.4	The Time-Triggered MPSoC	336
	Points to Remember	337
	Bibliographic Notes	338
	Review Questions and Problems	338
	Abbreviations	341
	Glossary	343
	References	359
	Index	369

Chapter 1

The Real-Time Environment

Overview The purpose of this introductory chapter is to describe the environment of real-time computer systems from a number of different perspectives. A solid understanding of the technical and economic factors that characterize a real-time application helps to interpret the demands that the system designer must cope with. The chapter starts with the definition of a real-time system and with a discussion of its functional and meta-functional requirements. Particular emphasis is placed on the temporal requirements that are derived from the well-understood properties of control applications. The objective of a control algorithm is to drive a process such that a performance criterion is satisfied. Random disturbances occurring in the environment degrade system performance and must be taken into account by the control algorithm. Any additional uncertainty that is introduced into the control loop by the control system itself, e.g., a non-predictable jitter of the control loop, results in a degradation of the quality of control.

In the Sects. 1.2 to 1.5 real-time applications are classified from a number of viewpoints. Special emphasis is placed on the fundamental differences between *hard* and *soft* real-time systems. Because soft real-time systems do not have severe failure modes, a less rigorous approach to their design is often followed. Sometimes resource-inadequate solutions that will not handle the rarely occurring peak-load scenarios are accepted on economic arguments. In a hard real-time application, such an approach is unacceptable because the safety of a design in all specified situations, even if they occur only very rarely, must be demonstrated *vis-a-vis* a certification agency. In Sect. 1.6, a brief analysis of the real-time system market is carried out with emphasis on the field of embedded real-time systems. An embedded real-time system is a part of a self-contained product, e.g., a television set or an automobile. Embedded real-time systems, also called *cyber-physical* (CPS) systems, form the most important market segment for real-time technology and the computer industry in general.

1.1 When Is a Computer System Real-Time?

A *real-time computer system* is a computer system where the correctness of the system behavior depends not only on the logical results of the computations, but also on the physical time when these results are produced. By *system behavior* we mean the sequence of *outputs in time* of a system.

We model the flow of time by a directed time line that extends from the past into the future. A cut of the time line is called an *instant*. Any *ideal* occurrence that happens at an *instant* is called an *event*. Information that describes an event (see also Sect. 5.2.4 on event observation) is called *event information*. The present point in time, *now*, is a very special event that separates the *past* from the *future* (the presented model of time is based on Newtonian physics and disregards relativistic effects). An *interval* on the time line, called a *duration*, is defined by two events, the *start event* and the *terminating event* of the interval. A digital clock partitions the time line into a sequence of equally spaced durations, called the *granules* of the clock, which are delimited by special periodic events, the *ticks* of the clock.

A real-time computer system is always part of a larger system – this larger system is called a *real-time system* or a *cyber-physical system*. A real-time system changes as a function of physical time, e.g., a chemical reaction continues to change its state even after its controlling computer system has stopped. It is reasonable to decompose a real-time system into a set of self-contained subsystems called *clusters*. Examples of clusters are (Fig. 1.1): the physical plant or machine that is to be controlled (the *controlled cluster*), the real-time computer system (the *computational cluster*;) and, the *human operator* (the *operator cluster*). We refer to the *controlled cluster* and the *operator cluster* collectively as the *environment* of the *computational cluster* (the real-time computer system).

If the real-time computer system is *distributed* (and most of them are), it consists of a set of (computer) *nodes* interconnected by a real-time communication network.

The interface between the human operator and the real-time computer system is called the *man-machine interface*, and the interface between the controlled object and the real-time computer system is called the *instrumentation interface*. The man-machine interface consists of input devices (e.g., keyboard) and output devices (e.g., display) that interface to the human operator. The instrumentation

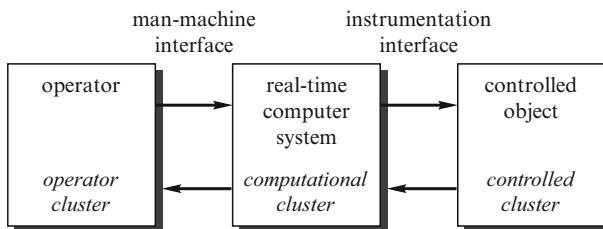


Fig. 1.1 Real-time system

interface consists of the sensors and actuators that transform the physical signals (e.g., voltages, currents) in the controlled cluster into a digital form and *vice versa*.

A real-time computer system must react to stimuli from its environment (the controlled cluster or the operator cluster) within time intervals dictated by its environment. The instant when a result must be produced is called a *deadline*. If a result has utility even after the deadline has passed, the deadline is classified as *soft*, otherwise it is *firm*. If *severe consequences* could result if a firm deadline is missed, the deadline is called *hard*.

Example: Consider a traffic signal at a road before a railway crossing. If the traffic signal does not change to *red* before the train arrives, an accident could result.

A real-time computer system that must meet at least one hard deadline is called a *hard real-time computer system* or a *safety-critical real-time computer system*. If no hard deadline exists, then the system is called a *soft real-time computer system*.

The design of a hard real-time system is fundamentally different from the design of a soft real-time system. While a hard real-time computer system must sustain a guaranteed temporal behavior under all specified load and fault conditions, it is permissible for a soft real-time computer system to miss a deadline occasionally. The differences between soft and hard real-time systems will be discussed in detail in the following sections. The focus of this book is on the design of hard real-time systems.

1.2 Functional Requirements

The functional requirements of real-time systems are concerned with the functions that a real-time computer system must perform. They are grouped into data collection requirements, direct digital control requirements, and man-machine interaction requirements.

1.2.1 Data Collection

A controlled object, e.g., a car or an industrial plant, changes its state as a function of time (whenever we use the word *time* without a qualifier, we mean *physical time* as described in Sect. 3.1). If we freeze the time, we can describe the current state of the controlled object by recording the values of its state variables at that moment. Possible state variables of a controlled object *car* are the position of the car, the speed of the car, the position of switches on the dashboard, and the position of a piston in a cylinder. We are normally not interested in *all* state variables, but only in the *subset* of state variables that is *significant* for our purpose. A significant state variable is called a *real-time (RT) entity*.

Every RT entity is in the *sphere of control (SOC)* of a subsystem, i.e., it belongs to a subsystem that has the authority to change the value of this RT entity (see also Sect. 5.1.1). Outside its sphere of control, the value of an RT entity can be observed, but its *semantic content* (see Sect. 2.2.4) cannot be modified. For example, the current position of a piston in a cylinder of the engine is in the sphere of control of the engine. Outside the car engine the current position of the piston can only be observed, but we are not allowed to modify the *semantic content* of this observation (the representation of the *semantic content* can be changed!).

The first functional requirement of a real-time computer system is the observation of the RT entities in a controlled cluster and the collection of these observations. An observation of an RT entity is represented by a *real-time (RT) image* in the computer system. Since the state of a *controlled object* in the *controlled cluster* is a function of real time, a given RT image is only *temporally accurate* for a limited time interval. The length of this time interval depends on the dynamics of the controlled object. If the state of the controlled object changes very quickly, the corresponding RT image has a very short *accuracy interval*.

Example: Consider the example of Fig. 1.2, where a car enters an intersection controlled by a traffic light. How long is the observation *the traffic light is green* temporally accurate? If the information *the traffic light is green* is used outside its accuracy interval, i.e., a car enters the intersection after the traffic light has switched to *red*, an accident may occur. In this example, an upper bound for the accuracy interval is given by the duration of the yellow phase of the traffic light.

The set of all temporally accurate real-time images of the controlled cluster is called the *real-time database*. The real-time database must be updated whenever an RT entity changes its value. These updates can be performed periodically, triggered by the progression of the real-time clock by a fixed period (*time-triggered (TT) observation*), or immediately after a change of state, which constitutes an event, occurs in the RT entity (*event-triggered (ET) observation*). A more detailed analysis of time-triggered and event-triggered observations will be presented in Chaps. 4 and 5.

Signal Conditioning. A physical sensor, e.g., a thermocouple, produces a *raw data* element (e.g., a voltage). Often, a sequence of raw data elements is collected and an averaging algorithm is applied to reduce the measurement error. In the next step the raw data must be calibrated and transformed to standard measurement units.

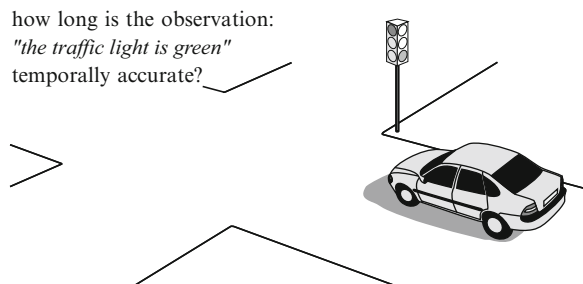


Fig. 1.2 Temporal accuracy of the traffic light information

The term *signal conditioning* is used to refer to all the processing steps that are necessary to obtain meaningful *measured data* of an RT entity from the raw sensor data. After signal conditioning, the measured data must be checked for plausibility and related to other measured data to detect a possible fault of the sensor. A data element that is judged to be a correct RT image of the corresponding RT entity is called an *agreed data element*.

Alarm Monitoring. An important function of a real-time computer system is the continuous monitoring of the RT entities to detect abnormal process behaviors.

Example: The rupture of a pipe, a *primary event*, in a chemical plant will cause many RT entities (diverse pressures, temperatures, liquid levels) to deviate from their normal operating ranges, and to cross some preset alarm limits, thereby generating a set of correlated alarms, which is called an *alarm shower*.

The real-time computer system must detect and display these alarms and must assist the operator in identifying a *primary event* that was the initial cause of these alarms. For this purpose, alarms that are observed must be logged in a special alarm log with the exact instant when the alarm occurred. The exact temporal order of the alarms is helpful in identifying the *secondary alarms*, i.e., all alarms that can be a causal consequence of the primary event. In complex industrial plants, sophisticated knowledge-based systems are used to assist the operator in the alarm analysis.

Example: In the final report on the August 14, 2003 power blackout in the United States and Canada we find on [Tas03, p. 162], the following statement: *A valuable lesson from the August 14 blackout is the importance of having time-synchronized system data recorders. The Task Force's investigators labored over thousands of data items to determine the sequence of events much like putting together small pieces of a very large puzzle. That process would have been significantly faster and easier if there had been wider use of synchronized data recording devices.*

A situation that occurs infrequently but is of utmost concern when it does occur is called a *rare-event* situation. The validation of the performance of a real-time computer system in a rare event situation is a challenging task and requires models of the physical environment (see Sect. 11.3.1).

Example: The sole purpose of a nuclear power plant monitoring and shutdown system is reliable performance in a peak-load alarm situation (a *rare event*). Hopefully, this rare event will never occur during the operational life of the plant.

1.2.2 Direct Digital Control

Many real-time computer systems must calculate the *actuating variables* for the actuators in order to control the controlled object directly (*direct digital control* – DDC), i.e., without any underlying conventional control system.

Control applications are highly regular, consisting of an (infinite) sequence of control cycles, each one starting with sampling (observing) of the RT entities,

followed by the execution of the control algorithm to calculate a new actuating variable, and subsequently by the output of the actuating variable to the actuator. The design of a proper control algorithm that achieves the desired control objective, and compensates for the random disturbances that perturb the controlled object, is the topic of the field of control engineering. In the next section on temporal requirements, some basic notions of control engineering will be introduced.

1.2.3 *Man–Machine Interaction*

A real-time computer system must inform the operator of the current state of the controlled object, and must assist the operator in controlling the machine or plant object. This is accomplished via the man–machine interface, a critical subsystem of major importance. Many severe computer-related accidents in safety-critical real-time systems have been traced to mistakes made at the man–machine interface [Lev95].

Example: *Mode confusion* at the man–machine interface of an aircraft has been identified to be the cause of major aircraft accidents [Deg95].

Most process-control applications contain, as part of the man–machine interface, an extensive data logging and data reporting subsystem that is designed according to the demands of the particular industry.

Example: In some countries, the pharmaceutical industry is required by law to record and store all relevant process parameters of every production batch in an archival storage in order that the process conditions prevailing at the time of a production run can be reexamined in case a defective product is identified on the market at a later time.

Man–machine interfacing has become such an important issue in the design of computer-based systems that a number of courses dealing with this topic have been developed. In the context of this book, we will introduce an abstract man–machine interface in Sect. 4.5.2, but we will not cover its design in detail. The interested reader is referred to standard textbooks on user interface design.

1.3 Temporal Requirements

1.3.1 *Where Do Temporal Requirements Come from?*

The most stringent temporal demands for real-time systems have their origin in the requirements of control loops, e.g., in the control of a fast process such as an automotive engine. The temporal requirements at the man–machine interface are, in

comparison, less stringent because the human perception delay, in the range of 50–100 ms, is orders of magnitude larger than the latency requirements of fast control loops.

A Simple Control Loop. Consider the simple control loop depicted in Fig. 1.3 consisting of a vessel with a liquid, a heat exchanger connected to a steam pipe, and a controlling computer system. The objective of the computer system is to control the valve (*control variable*) determining the flow of steam through the heat exchanger such that the temperature of the liquid in the vessel remains within a small range around the *set point* selected by the operator.

The focus of the following discussion is on the temporal properties of this simple control loop consisting of a *controlled object* and a *controlling computer system*.

The Controlled Object. Assume that the system of Fig. 1.3 is in equilibrium. Whenever the steam flow is increased by a step function, the temperature of the liquid in the vessel will change according to Fig. 1.4 until a new equilibrium is reached. This *response function* of the temperature in the vessel depends on the environmental conditions, e.g., the amount of liquid in the vessel, and the flow of steam through the heat exchanger, i.e., on the dynamics of the controlled object. (In the following section, we will use d to denote a *duration* and t to denote an *instant*, i.e., a point in time).

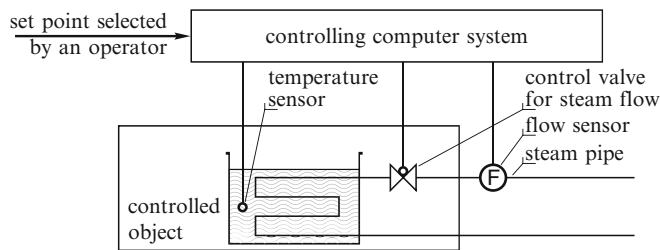


Fig. 1.3 A simple control loop

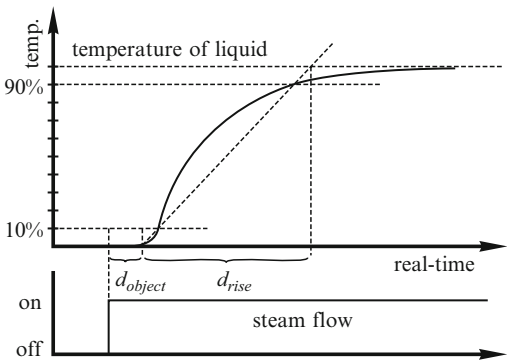


Fig. 1.4 Delay and rise time of the step response

There are two important temporal parameters characterizing this elementary step response function, the *object delay* d^{object} (sometimes called the *lag time* or *lag*) after which the *measured variable* temperature begins to rise (caused by the initial inertia of the process and the instrumentation, called the *process lag*) and the *rise time* d^{rise} of the temperature until the new equilibrium state has been reached. To determine the *object delay* d^{object} and the *rise time* d^{rise} from a given experimentally recorded shape of the step-response function, one finds the two points in time where the response function has reached 10% and 90% of the difference between the two stationary equilibrium values. These two points are connected by a straight line (Fig. 1.4). The significant points in time that characterize the *object delay* d^{object} and the *rise time* d^{rise} of the step response function are constructed by finding the intersection of this straight line with the two horizontal lines that denote the two liquid temperatures that correspond to the stable equilibrium states before and after the application of the step function.

Controlling Computer System. The controlling computer system must sample the temperature of the vessel periodically to detect any deviation between the intended value and the actual value of the controlled variable *temperature*. The constant duration between two sampling points is called the sampling period d^{sample} and the reciprocal $1/d^{sample}$ is the sampling frequency, f^{sample} . A rule of thumb says that, in a digital system which is expected to behave like a quasi-continuous system, the sampling period should be less than one-tenth of the rise time d^{rise} of the step response function of the controlled object, i.e., $d^{sample} < (d^{rise}/10)$. The computer compares the *measured temperature* to the *temperature set point* selected by the operator and calculates the *error term*. This error term forms the basis for the calculation of a new value of the control variable by a *control algorithm*. A given time interval after each sampling point, called the *computer delay* $d^{computer}$, the controlling computer will output this new value of the actuating variable to the control valve, thus closing the control loop. The delay $d^{computer}$ should be smaller than the sampling period d^{sample} .

The difference between the maximum and the minimum values of the delay of the computer is called the *jitter* of the computer delay, $\Delta d^{computer}$. This jitter is a sensitive parameter for the quality of control.

The *dead time* of the control loop is the time interval between the observation of the RT entity and the start of a reaction of the controlled object due to a computer action based on this observation. The dead time is the sum of the controlled object delay d^{object} , which is in the sphere of control of the controlled object and is thus determined by the controlled object's dynamics, and the computer delay $d^{computer}$, which is determined by the computer implementation. To reduce the dead time in a control loop and to improve the stability of the control loop, these delays should be as small as possible. The computer delay $d^{computer}$ is defined by the time interval between the *sampling points*, i.e., the observation of the controlled object, and the *use* of this information (see Fig. 1.5), i.e., the output of the corresponding actuator signal, the *actuating variable*, to the controlled object. Apart from the necessary

Fig. 1.5 Delay and delay jitter

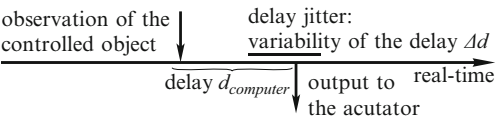


Table 1.1 Parameters of an elementary control loop

Symbol	Parameter	Sphere of control	Relationships
d^{object}	Controlled object delay	Controlled object	Physical process
d^{rise}	Rise time of step response	Controlled object	Physical process
d^{sample}	Sampling period	Computer	$d^{sample} < < d^{rise}$
$d^{computer}$	Computer delay	Computer	$d^{computer} < d^{sample}$
$\Delta d^{computer}$	Jitter of the computer delay	Computer	$\Delta d^{computer} < < d^{computer}$
$d^{deadtime}$	Dead time	Computer and controlled object	$d^{computer} + d^{object}$

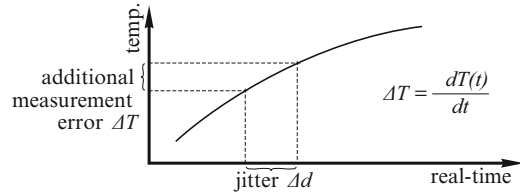
time for performing the calculations, the computer delay is determined by the time required for communication and the reaction time of the actuator.

Parameters of a Control Loop. Table 1.1 summarizes the temporal parameters that characterize the elementary control loop depicted in Fig. 1.3. In the first two columns we denote the symbol and the name of the parameter. The third column denotes the sphere of control in which the parameter is located, i.e., what subsystem determines the value of the parameter. Finally, the fourth column indicates the relationships between these temporal parameters.

1.3.2 Minimal Latency Jitter

The data items in control applications are state-based, i.e., they contain images of the RT entities. The computational actions in control applications are mostly time-triggered, e.g., the control signal for obtaining a sample is derived from the progression of time within the computer system. This control signal is thus in the sphere of control of the computer system. It is known in advance when the next control action must take place. Many control algorithms are based on the assumption that the delay jitter $\Delta d^{computer}$ is very small compared to the *delay* $d^{computer}$, i.e., the delay is close to constant. This assumption is made because control algorithms can be designed to compensate a *known* constant delay. Delay jitter brings an additional uncertainty into the control loop that has an adverse effect on the quality of control. The jitter Δd can be seen as an uncertainty about the instant when the RT-entity was observed. This jitter can be interpreted as causing an additional value error ΔT of the measured variable temperature T as shown in Fig. 1.6. Therefore, the delay jitter should always be a small fraction of the delay, i.e., if a delay of 1 ms is demanded then the delay jitter should be in the range of a few μs [SAE95].

Fig. 1.6 The effect of jitter on the measured variable T



1.3.3 Minimal Error-Detection Latency

Hard real-time applications are, by definition, safety-critical. It is therefore important that any error within the control system, e.g., the loss or corruption of a message or the failure of a node, is detected within a short time with a very high probability. The required *error-detection latency* must be in the same order of magnitude as the sampling period of the fastest critical control loop. It is then possible to perform some corrective action, or to bring the system into a safe state, before the consequences of an error can cause any severe system failure. Almost-no-jitter systems will have shorter guaranteed error-detection latencies than systems that allow for jitter.

1.4 Dependability Requirements

The notion of dependability covers the meta-functional attributes of a computer system that relate to the quality of service a system delivers to its users during an extended interval of time. (A user could be a human or another technical system.) The following measures of dependability attributes are of importance [Avi04]:

1.4.1 Reliability

The *Reliability* $R(t)$ of a system is the probability that a system will provide the specified service until time t , given that the system was operational at the beginning, i.e., $t = t_o$. The probability that a system will fail in a given interval of time is expressed by the *failure rate*, measured in *FITs* (Failure In Time). A failure rate of 1 *FIT* means that the mean time to a failure (*MTTF*) of a device is 10^9 h, i.e., one failure occurs in about 115,000 years. If a system has a constant *failure rate* of λ *failures/h*, then the reliability at time t is given by

$$R(t) = \exp(-\lambda(t - t_o)),$$

where $t - t_o$ is given in hours. The inverse of the failure rate $1/\lambda = MTTF$ is called the *Mean-Time-To-Failure MTTF* (in hours). If the failure rate of a system is required to be in the order of 10^{-9} failures/h or lower, then we speak of a system with an *ultrahigh reliability* requirement.

1.4.2 Safety

Safety is reliability regarding *critical failure modes*. A critical failure mode is said to be *malign*, in contrast with a noncritical failure, which is *benign*. In a malign failure mode, the cost of a failure can be orders of magnitude higher than the utility of the system during normal operation. Examples of malign failures are: an airplane crash due to a failure in the flight-control system, and an automobile accident due to a failure of a computer-controlled intelligent brake in the automobile. Safety-critical (hard) real-time systems must have a failure rate with regard to critical failure modes that conforms to the *ultrahigh reliability* requirement.

Example: Consider the example of a computer-controlled brake in an automobile. The failure rate of a computer-caused critical brake failure must be lower than the failure rate of a conventional braking system. Under the assumption that a car is operated about 1 h per day on the average, one safety-critical failure per million cars per year translates into a failure rate in the order of 10^{-9} failures/h.

Similarly low failure rates are required in flight-control systems, train-signaling systems, and nuclear power plant monitoring systems.

Certification. In many cases the design of a safety-critical real-time system must be approved by an independent certification agency. The certification process can be simplified if the certification agency can be convinced that:

1. The subsystems that are critical for the safe operation of the system are protected by fault-containment mechanisms that eliminate the possibility of error propagation from the rest of the system into these safety-critical subsystems.
2. From the point of view of design, all scenarios that are covered by the given load- and fault-hypothesis can be handled according to the specification without reference to probabilistic arguments. This makes a resource adequate design necessary.
3. The architecture supports a constructive modular certification process where the certification of subsystems can be done independently of each other. At the system level, only the *emergent properties* must be validated.

[Joh92] specifies the required properties for a system that is *designed for validation*:

1. A complete and accurate reliability model can be constructed. All parameters of the model that cannot be deduced analytically must be measurable in feasible time under test.

2. The reliability model does not include state transitions representing design faults; analytical arguments must be presented to show that design faults will not cause system failure.
3. Design tradeoffs are made in favor of designs that minimize the number of parameters that must be measured (see Sect. 2.2.1).

1.4.3 Maintainability

Maintainability is a measure of the time interval required to repair a system after the occurrence of a benign failure. Maintainability is measured by the probability $M(d)$ that the system is restored within a time interval d after the failure. In keeping with the reliability formalism, a constant repair rate μ (repairs per hour) and a *Mean Time to Repair (MTTR)* are introduced to define a quantitative maintainability measure.

There is a fundamental conflict between reliability and maintainability. A maintainable design requires the partitioning of a system into a set of *field replaceable units (FRUs)* connected by serviceable interfaces that can be easily disconnected and reconnected to replace a faulty *FRU* in case of a failure. A serviceable interface, e.g., a plug connection, has a significantly higher physical failure rate than a non-serviceable interface. Furthermore, a serviceable interface is more expensive to produce.

In the upcoming field of *ambient intelligence*, automatic diagnosis and *maintainability by an untrained end user* will be important system properties that are critical for the market success of a product.

1.4.4 Availability

Availability is a measure of the delivery of correct service with respect to the alternation of correct and incorrect service. It is measured by the fraction of time that the system is ready to provide the service.

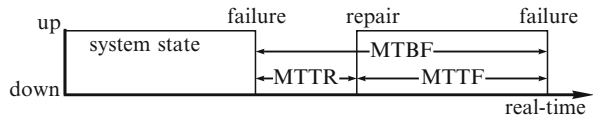
Example: Whenever a user picks up the phone, the telephone switching system should be ready to provide the telephone service with a very high probability. A telephone exchange is allowed to be out of service for only a few minutes per year.

In systems with constant failure and repair rates, the reliability (*MTTF*), maintainability (*MTTR*), and availability (*A*) measures are related by

$$A = MTTF / (MTTF + MTTR).$$

The sum $MTTF + MTTR$ is sometimes called the *Mean Time Between Failures (MTBF)*. Figure 1.7 shows the relationship between *MTTF*, *MTTR*, and *MTBF*.

Fig. 1.7 Relationship between $MTTF$, $MTBF$ and $MTTR$



A high availability can be achieved either by a long $MTTF$ or by a short $MTTR$. The designer has thus some freedom in the selection of her/his approach to the construction of a high-availability system.

1.4.5 Security

A fifth important attribute of dependability – the *security attribute* – is concerned with the authenticity and integrity of information, and the ability of a system to prevent unauthorized access to information or services (see also Sect. 6.2). There are difficulties in defining a quantitative security measure, e.g., the specification of a *standard burglar* that takes a certain time to intrude a system. Traditionally, security issues have been associated with large databases, where the concerns are confidentiality, privacy, and authenticity of information. During the last few years, security issues have also become important in real-time systems, e.g., a cryptographic theft-avoidance system that locks the ignition of a car if the user cannot present the specified access code. In the *Internet-of-Things (IoT)*, where the endpoints of the Internet are embedded systems that bridge the gap between the cyber world and physical world, security concerns are of crucial importance, since an intruder cannot only corrupt a data structure in a computer, but can cause harm in the physical environment.

1.5 Classification of Real-Time Systems

In this section we classify real-time systems from different perspectives. The first two classifications, *hard real-time* versus *soft real-time* (on-line), and *fail-safe* versus *fail-operational*, depend on the characteristics of the application, i.e., on factors *outside* the computer system. The second three classifications, *guaranteed-timeliness* versus *best effort*, *resource-adequate* versus *resource-inadequate*, and *event-triggered* versus *time-triggered*, depend on the design and implementation of the computer application, i.e., on factors *inside* the computer system.

1.5.1 Hard Real-Time System Versus Soft Real-Time System

The design of a *hard real-time system*, which always must produce the results at the correct instant, is fundamentally different from the design of a *soft-real time* or an *on-line system*, such as a transaction processing system. In this section we will

Table 1.2 Hard real-time versus soft real-time systems

Characteristic	Hard real-time	Soft real-time (on-line)
Response time	Hard-required	Soft-desired
Peak-load performance	Predictable	Degraded
Control of pace	Environment	Computer
Safety	Often critical	Non-critical
Size of data files	Small/medium	Large
Redundancy type	Active	Checkpoint–recovery
Data integrity	Short-term	Long-term
Error detection	Autonomous	User assisted

elaborate on these differences. Table 1.2 compares the characteristics of hard real-time systems versus soft real-time systems.

Response Time. The demanding response time requirements of hard real-time applications, often in the order of milliseconds or less, preclude direct human intervention during normal operation or in critical situations. A hard real-time system must be highly autonomous to maintain safe operation of the process. In contrast, the response time requirements of soft real-time and on-line systems are often in the order of seconds. Furthermore, if a deadline is missed in a soft real-time system, no catastrophe can result.

Peak-load Performance. In a hard real-time system, the peak-load scenario must be well defined. It must be guaranteed by design that the computer system meets the specified deadlines in all situations, since the utility of many hard real-time applications depends on their predictable performance during *rare event scenarios* leading to a peak load. This is in contrast to the situation in a soft-real time system, where the *average* performance is important, and a degraded operation in a rarely occurring peak load case is tolerated for economic reasons.

Control of Pace. A hard real-time computer system is often paced by the state changes occurring in the environment. It must keep up with the state of the environment (the controlled object and the human operator) under all circumstances. This is in contrast to an on-line system, which can exercise some control over the environment in case it cannot process the offered load.

Example: Consider the case of a transaction processing system, such as an airline reservation system. If the computer cannot keep up with the demands of the users, it just extends the response time and forces the users to slow down.

Safety. The safety criticality of many real-time applications has a number of consequences for the system designer. In particular, error detection and recovery must be autonomous such that the system can initiate appropriate recovery actions and arrive at a safe state within the time intervals dictated by the application without human intervention.

Size of Data Files. The *real-time database* that is composed of the temporally accurate images of the RT-entities is normally of small size. The key concern in

hard real-time systems is on the *short-term* temporal accuracy of the real-time database that is invalidated by the flow of real-time. In contrast, in on-line transaction processing systems, the maintenance of the *long-term* integrity and availability of large data files is the key issue.

Redundancy Type. After an error has been detected in an on-line system, the computation is rolled back to a previously established checkpoint to initiate a recovery action. In hard real-time systems, roll-back/recovery is of limited utility for the following reasons:

1. It is difficult to guarantee the deadline after the occurrence of an error, since the roll-back/recovery action can take an unpredictable amount of time.
2. An irrevocable action that has been effected on the environment cannot be undone.
3. The temporal accuracy of the checkpoint data may be invalidated by the time difference between the checkpoint time and the instant *now*.

The topic of temporal accuracy of real-time data is discussed at length in Sect. 5.4 while the issues of error detection and types of redundancy are dealt with in Chap. 6.

1.5.2 Fail-Safe Versus Fail-Operational

In many hard real-time systems one or more safe states, which can be reached in case of a system failure, can be identified. If such a safe state can be identified and quickly reached upon the occurrence of a failure, then we call the system *fail-safe*. Fail-safeness is a characteristic of the controlled object, not the computer system. In fail-safe applications the computer system must have a *high error-detection coverage*, i.e., the probability that an error is detected, provided it has occurred, must be close to one.

Example: In case a failure is detected in a railway signaling system, it is possible to set all signals to red and thus stop all the trains in order to bring the system to a safe state.

In many real-time computer systems a special external device, a *watchdog*, is provided to monitor the operation of the computer system. The computer system must send a periodic life-sign (e.g., a digital output of predefined form) to the watchdog. If this life-sign fails to arrive at the watchdog within the specified time interval, the watchdog assumes that the computer system has failed and forces the controlled object into a safe state. In such a system, timeliness is needed only to achieve high availability, but is not needed to maintain safety since the watchdog forces the controlled object into a safe state in case of a timing violation.

There are, however, applications where a safe state cannot be identified, e.g., a flight control system aboard an airplane. In such an application the computer system must remain operational and provide a minimal level of service even in the case of a failure to avoid a catastrophe. This is why these applications are called *fail-operational*.

1.5.3 *Guaranteed-Response Versus Best-Effort*

If we start out with a specified *fault-* and *load-hypothesis* and deliver a design that makes it possible to reason about the adequacy of the design without reference to probabilistic arguments (even in the case of a peak load and fault scenario), then we can speak of a system with a *guaranteed response*. The probability of failure of a perfect system with guaranteed response is reduced to the probability that the assumptions about the peak load and the number and types of faults do not hold in reality. This probability is called *assumption coverage* [Pow95]. Guaranteed response systems require careful planning and extensive analysis during the design phase.

If such an analytic response guarantee cannot be given, we speak of a *best-effort* design. Best-effort systems do not require a rigorous specification of the load- and fault-hypothesis. The design proceeds according to the principle *best possible effort taken* and the sufficiency of the design is established during the test and integration phases. It is difficult to establish that a best-effort design operates correctly in a rare-event scenario. At present, many non safety-critical real-time systems are designed according to the best-effort paradigm.

1.5.4 *Resource-Adequate Versus Resource-Inadequate*

Guaranteed response systems are based on the principle of resource adequacy, i.e., there are enough computing resources available to handle the specified peak load and the fault scenario. Many non safety-critical real-time system designs are based on the principle of resource inadequacy. It is assumed that the provision of sufficient resources to handle every possible situation is not economically viable, and that a dynamic resource allocation strategy based on resource sharing and probabilistic arguments about the expected load and fault scenarios is acceptable.

It is expected that, in the future, there will be a paradigm shift to resource-adequate designs in many applications. The use of computers in important volume-based applications, e.g., in cars, raises both the public awareness as well as concerns about computer-related incidents, and forces the designer to provide convincing arguments that the design functions properly under *all* stated conditions. Hard real-time systems must be designed according to the guaranteed response paradigm that requires the availability of adequate resources.

1.5.5 *Event-Triggered Versus Time-Triggered*

The distinction between *event-triggered* and *time-triggered* depends on the type of internal *triggers* and not the external behavior of a real-time system. A *trigger* is an event that causes the start of some action in the computer, e.g., the execution of a

task or the transmission of a message. Depending on the triggering mechanisms for the start of communication and processing actions in each node of a computer system, two distinctly different approaches to the design of the control mechanisms of real-time computer applications can be identified, *event-triggered control* and *time-triggered control*.

In *event-triggered (ET)* control, all communication and processing activities are initiated whenever a significant event other than the regular event of a clock tick occurs. In an ET system, the signaling of significant events to the central processing unit (CPU) of a computer is realized by the well-known interrupt mechanism. ET systems require a dynamic scheduling strategy to activate the appropriate software task that services the event.

In a *time-triggered (TT)* system, all activities are initiated by the progression of real-time. There is only one interrupt in each node of a distributed TT system, the periodic real-time clock interrupt. Every communication or processing activity is initiated at a periodically occurring predetermined tick of a clock. In a distributed TT real-time system, it is assumed that the clocks of all nodes are synchronized to form a *global time* that is available at every node. Every observation of the controlled object is time-stamped with this global time. The granularity of the global time must be chosen such that the time order of any two observations made anywhere in a distributed TT system can be established from their time-stamps with adequate faithfulness [Kop09]. The topics of global time and clock synchronization will be discussed at length in Chap. 3.

Example: The distinction between *event-triggered* and *time-triggered* can be explained by an example of an elevator control system. When you push a *call button* in the event-triggered implementation, the event is immediately relayed to the interrupt system of the computer in order to start the action of calling the elevator. In a time-triggered system, the button push is stored locally, and periodically, e.g., every second, the computer asks to get the state of all push buttons. The flow of control in a time-triggered system is managed by the progression of time, while in an event-triggered system; the flow of control is determined by the events that happen in the environment or the computer system.

1.6 The Real-Time Systems Market

In a market economy, the cost/performance relation is a decisive parameter for the market success of any product. There are only a few scenarios where cost arguments are not the major concern. The total life-cycle cost of a product can be broken down into three rough categories: non-recurring development cost, production cost, and operation and maintenance cost. Depending on the product type, the distribution of the total life-cycle cost over these three cost categories can vary significantly. We will examine this life cycle cost distribution by looking at two important examples of real-time systems, embedded real-time systems and plant-automation systems.

1.6.1 *Embedded Real-Time Systems*

The ever-decreasing price/performance ratio of microcontrollers makes it economically attractive to replace conventional mechanical or electronic control system within many products by an embedded real-time computer system. There are numerous examples of products with embedded computer systems: cellular phones, engine controllers in cars, heart pacemakers, computer printers, television sets, washing machines, even some electric razors contain a microcontroller with some thousand instructions of software code. Because the external interfaces (particularly the man-machine interface) of the product often remain unchanged relative to the previous product generation, it is often not visible from the outside that a real-time computer system is controlling the product behavior.

Characteristics. An embedded real-time computer system is always part of a well-specified larger system, which we call an *intelligent product*. An intelligent product consists of a physical (mechanical) subsystem; the controlling embedded computer, and, most often, a man-machine interface. The ultimate success of any intelligent product depends on the relevance and quality of service it can provide to its users. A focus on the genuine user needs is thus of utmost importance.

Embedded systems have a number of distinctive characteristics that influence the system development process:

1. **Mass Production:** many embedded systems are designed for a mass market and consequently for mass production in highly automated assembly plants. This implies that the production cost of a single unit must be as low as possible, i.e., efficient memory and processor utilization are of concern.
2. **Static Structure:** the computer system is embedded in an intelligent product of given functionality and rigid structure. The known *a priori* static environment can be analyzed at design time to simplify the software, to increase the robustness, and to improve the efficiency of the embedded computer system. In many embedded systems there is no need for flexible dynamic software mechanisms. These mechanisms increase the resource requirements and lead to an unnecessary complexity of the implementation.
3. **Man-Machine Interface:** if an embedded system has a man-machine interface, it must be specifically designed for the stated purpose and must be easy to operate. Ideally, the use of the intelligent product should be self-explanatory, and not require any training or reference to an operating manual.
4. **Minimization of the Mechanical Subsystem:** to reduce the manufacturing cost and to increase the reliability of the intelligent product, the complexity of the mechanical subsystem is minimized.
5. **Functionality Determined by Software in Read-Only Memory (ROM):** the integrated software that often resides in ROM determines the functionality of many intelligent products. Since it is not possible to modify the software in a ROM after its release, the quality standards for this software are high.

6. **Maintenance Strategy:** many intelligent products are designed to be non maintainable, because the partitioning of the product into replaceable units is too expensive. If, however, a product is designed to be maintained in the field, the provision of an excellent diagnostic interface and a self-evident maintenance strategy is of importance.
7. **Ability to communicate:** many intelligent products are required to interconnect with some larger system or the Internet. Whenever a connection to the Internet is supported, the topic of security is of utmost concern.
8. **Limited amount of energy:** Many mobile embedded devices are powered by a battery. The lifetime of a battery load is a critical parameter for the utility of a system.

A large fraction of the life-cycle cost of many intelligent products is in the production, i.e., in the hardware. The known *a priori* static configuration of the intelligent product can be used to reduce the resource requirements, and thus the production cost, and also to increase the robustness of the embedded computer system. Maintenance cost can become significant, particularly if an undetected design fault (software fault) requires a recall of the product, and the replacement of a complete production series.

Example: In [Neu96] we find the following laconic one-liner: *General Motors recalls almost 300 K cars for engine software flaw.*

Future Trends. During the last few years, the variety and number of embedded computer applications have grown to the point that, by now, this segment is by far the most important one in the computer market. The embedded system market is driven by the continuing improvements in the cost/performance ratio of the semiconductor industry that makes computer-based control systems cost-competitive relative to their mechanical, hydraulic, and electronic counterparts. Among the key mass markets are the domains of consumer electronics and automotive electronics. The automotive electronics market is of particular interest, because of stringent timing, dependability, and cost requirements that act as *technology catalysts*.

Automotive manufacturers view the proper exploitation of computer technology as a key competitive element in the never-ending quest for increased vehicle performance and reduced manufacturing cost. While some years ago, the computer applications on board a car focused on non-critical body electronics or comfort functions, there is now a substantial growth in the computer control of core vehicle functions, e.g., engine control, brake control, transmission control, and suspension control. We observe an integration of many of these functions with the goal of increasing the vehicle stability in critical driving maneuvers. Obviously, an error in any of these core vehicle functions has severe safety implications.

At present the topic of computer safety in cars is approached at two levels. At the basic level a mechanical system provides the proven safety level that is considered sufficient to operate the car. The computer system provides optimized performance on top of the basic mechanical system. In case the computer system fails cleanly, the mechanical system takes over. Consider, for example, an *Electronic Stability*

Program (ESP). If the computer fails, the conventional mechanical brake system is still operational. Soon, this approach to safety may reach its limits for two reasons:

1. If the computer-controlled system is further improved, the magnitude of the difference between the performance of the computer-controlled system and the performance of the basic mechanical system is further increased. A driver who is used to the high performance of the computer-controlled system might consider the fallback to the inferior performance of the mechanical system a safety risk.
2. The improved price/performance of the microelectronic devices will make the implementation of fault-tolerant computer systems cheaper than the implementation of mixed computer/mechanical systems. Thus, there will be an economic pressure to eliminate the redundant mechanical system and to replace it with a computer system using active redundancy.

The embedded system market is expected to grow significantly during the next 10 years. It is expected that many embedded systems will be connected to the Internet, forming the Internet of Things (IoT – see Chap. 13).

1.6.2 Plant Automation Systems

Characteristics. Historically, industrial plant automation was the first field for the application of real-time digital computer control. This is understandable since the benefits that can be gained by the computerization of a sizable plant are much larger than the cost of even an expensive process control computer of the late 1960s. In the early days, human operators controlled the industrial plants locally. With the refinement of industrial plant instrumentation and the availability of remote automatic controllers, plant monitoring and command facilities were concentrated into a central control room, thus reducing the number of operators required to run the plant. In the 1970s, the next logical step was the introduction of central process control computers to monitor the plant and assist the operator in her/his routine functions, e.g., data logging and operator guidance. In the beginning, the computer was considered an *add-on* facility that was not fully trusted. It was the duty of the operator to judge whether a set point calculated by a computer made sense and could be applied to the process (*open-loop control*). In the next phase, *Supervisory Control and Data Acquisition* (SCADA) systems calculated the set-points for the *programmable logic controllers* (PLC) in the plant. With the improvement of the process models and the growth of the reliability of the computer, control functions have been increasingly allocated to the computer and gradually the operator has been taken out of the control loop (*closed-loop control*). Sophisticated non-linear control techniques, which have response time requirements beyond human capabilities, have been implemented.

Usually, every plant automation system is unique. There is an extensive amount of engineering and software effort required to adapt the computer system to the

physical layout, the operating strategy, the rules and regulations, and the reporting system of a particular plant. To reduce these engineering and software efforts, many process control companies have developed a set of modular building blocks, which can be configured individually to meet the requirements of a customer. Compared to the development cost, the production cost (hardware cost) is of minor importance. Maintenance cost can be an issue if a maintenance technician must be on-site for 24 h in order to minimize the downtime of a plant.

Future Trends. The market of industrial plant automation systems is limited by the number of plants that are newly constructed or are refurbished to install a computer control system. During the last 20 years, many plants have already been automated. This investment must pay off before a new generation of computers and control equipment is installed.

Furthermore, the installation of a new generation of control equipment in a production plant causes disruption in the operation of the plant with a costly loss of production that must be justified economically. This is difficult if the plant's efficiency is already high, and the margin for further improvement by refined computer control is limited.

The size of the plant automation market is too small to support the mass production of special application-specific components. This is the reason why many VLSI components that are developed for other application domains, such as automotive electronics, are taken up by this market to reduce the system cost. Examples of such components are sensors, actuators, real-time local area networks, and processing nodes. Already several process-control companies have announced a new generation of process-control equipment that takes advantage of low-priced mass produced components that have been developed for the automotive market, such as the chips developed for the Controller Area Network (CAN – see Sect. 7.3.2).

1.6.3 Multimedia Systems

Characteristics. The multimedia market is a mass market for specially designed soft and firm real-time systems. Although the deadlines for many multimedia tasks, such as the synchronization of audio and video streams, are firm, they are not hard deadlines. An occasional failure to meet a deadline results in a degradation of the *quality of the user experience*, but will not cause a catastrophe. The processing power required to transport and render a continuous video stream is large and difficult to estimate, because it is possible to improve a good picture even further. The resource allocation strategy in multimedia applications is thus quite different from that of hard real-time applications; it is not determined by the given application requirements, but by the amount of available resources. A fraction of the given computational resources (processing power, memory, bandwidth) is allocated to a user domain. *Quality of experience* considerations at the end user determine the

detailed resource allocation strategy. For example, if a user reduces the size of a window and enlarges the size of another window on his multimedia terminal, then the system can reduce the bandwidth and the processing allocated to the first window to free the resources for the other window that has been enlarged. Other users of the system should not be affected by this local reallocation of resources.

Future Trends. The marriage of the Internet with smart phones and multimedia personal computers leads to many new volume applications. The focus of this book is not on multimedia systems, because these systems belong to the class of soft and firm real-time applications.

1.7 Examples of Real-Time Systems

In this section, three typical examples of real-time systems are introduced. These examples will be used throughout the book to explain the evolving concepts. We start with an example of a very simple system for flow control to demonstrate the need for *end-to-end protocols* in process input/output.

1.7.1 Controlling the Flow in a Pipe

It is the objective of the simple control system depicted in Fig. 1.8 to control the flow of a liquid in a pipe. A given flow set point determined by a client should be maintained despite changing environmental conditions. Examples for such changing conditions are the varying level of the liquid in the vessel or the temperature sensitive viscosity of the liquid. The computer interacts with the controlled object by setting the position of the control valve. It then observes the reaction of the controlled object by reading the flow sensor F to determine whether the desired effect, the intended change of flow, has been achieved. This is a typical example of the necessary *end-to-end protocol* [Sal84] that must be put in place between the computer and the controlled object (see also Sect. 7.1.2). In a well-engineered system, the effect of any control action of the computer must be monitored by one or more independent sensors. For this purpose, many actuators contain a number of sensors in the same physical housing. For example, the control valve in Fig. 1.8 might contain a sensor, which measures the mechanical position of the valve in the

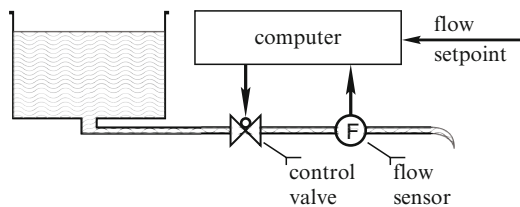


Fig. 1.8 Flow of liquid in a pipe

pipe, and two limit switches, which indicate the firmly closed and the completely open positions of the valve. A rule of thumb is that there are about three to seven sensors for every actuator.

The dynamics of the system in Fig. 1.8 is essentially determined by the speed of the control valve. Assume that the control valve takes 10 s to open or close from 0% to 100%, and that the flow sensor F has a precision of 1%. If a sampling interval of 100 ms is chosen, the maximum change of the valve position within one sampling interval is 1%, the same as the precision of the flow sensor. Because of this finite speed of the control valve, an output action taken by the computer at a given time will lead to an effect in the environment at some later time. The observation of this effect by the computer will be further delayed by the given latency of the sensor. All these latencies must either be derived analytically or measured experimentally, before the temporal control structure for a stable control system can be designed.

1.7.2 Engine Control

The task of an engine controller in an automobile engine is the calculation of the proper amount of fuel and the exact moment at which the fuel must be injected into the combustion chamber of each cylinder. The amount of fuel and the timing depend on a multitude of parameters: the intentions of the driver, articulated by the position of the accelerator pedal, the current load on the engine, the temperature of the engine, the condition of the cylinder, and many more. A modern engine controller is a complex piece of equipment. Up to 100 concurrently executing software tasks must cooperate in tight synchronization to achieve the desired goal, a smoothly running and efficient engine with a minimal output of pollutants.

The up- and downward moving piston in each cylinder of a combustion engine is connected to a rotating axle, the *crankshaft*. The intended start point of fuel injection is relative to the position of the piston in the cylinder, and must be precise within an accuracy of about 0.1° of the measured angular position of the crankshaft. The precise angular position of the crankshaft is measured by a number of digital sensors that generate a rising edge of a signal at the instant when the crankshaft passes these defined positions. Consider an engine that turns with 6,000 rpm (revolutions per minute), i.e., the crankshaft takes 10 ms for a 360° rotation. If the required precision of 0.1° is transformed into the time domain, then a temporal accuracy of $3\ \mu\text{s}$ is required. The fuel injection is realized by opening a solenoid valve or a piezoelectric actuator that controls the fuel flow from a high-pressure reservoir into the cylinder. The latency between giving an *open* command to the valve and the actual point in time when the valve opens can be in the order of hundreds of μs , and changes considerably depending on environmental conditions (e.g., temperature). To be able to compensate for this latency jitter, a sensor signal indicates the point in time when the valve has actually opened. The duration between the execution of the output command by the computer and the start of opening of the valve is measured during every engine cycle. The measured latency

is used to determine when the output command must be executed during the next cycle so that the intended effect, the start of fuel injection, happens at the proper point in time.

This example of an engine controller has been chosen because it demonstrates convincingly the need for extremely precise temporal control. For example, if the processing of the signal that measures the exact position of the crankshaft in the engine is delayed by a few μs , the quality of control of the whole system is compromised. It can even happen that the engine is mechanically damaged if a valve is opened at an incorrect moment.

1.7.3 Rolling Mill

A typical example of a distributed plant automation system is the computer control of a rolling mill. In this application a slab of steel (or some other material, such as paper) is rolled to a strip and coiled. The rolling mill of Fig. 1.9 has three drives and some instrumentation to measure the quality of the rolled product. The distributed computer-control system of this rolling mill consists of seven nodes connected by a real-time communication system. The most important sequence of actions – we call this a *real-time (RT) transaction* – in this application starts with the reading of the sensor values by the sensor computer. Then, the RT transaction passes through the model computer that calculates new set points for the three drives, and finally reaches the control computers to achieve the desired action by readjusting the rolls of the mill. The RT-transaction thus consists of three processing actions connected by two communication actions.

The total duration of the RT transaction (bold line in Fig. 1.9) is an important parameter for the quality of control. The shorter the duration of this transaction, the better the control quality and the stability of the control loop, since this transaction contributes to the *dead time* of the critical control loop. The other important term of the dead time is the time it takes for the strip to travel from the drive to the sensor. A jitter in the dead time that is not compensated for will reduce the quality of

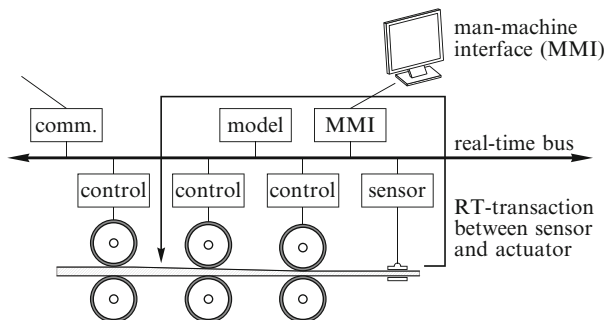


Fig. 1.9 An RT transaction

control significantly. It is evident from Fig. 1.9 that the latency jitter in an event-triggered system is the sum of the jitter of all processing and communication actions that form the critical RT transaction.

Note that the communication pattern among the nodes of this control system is *multicast*, not *point-to-point*. This is typical for most distributed real-time control systems. Furthermore, the communication between the model node and the drive nodes has an *atomicity requirement*. Either all of the drives are changed according to the output of the model, or none of them is changed. The loss of a message, which may result in the failure of a drive to readjust to a new position, may cause mechanical damage to the drive.

Points to Remember

- A real-time computer system must react to stimuli from the controlled object (or the operator) within time intervals *dictated* by its environment. If a catastrophe could result in case a firm deadline is missed, the deadline is called *hard*.
- In a hard real-time computer system, it must be guaranteed by design that the computer system will meet the specified deadlines in all situations because the utility of many hard real-time applications can depend on predictable performance during a peak load scenario.
- A hard real-time system must maintain synchrony with the state of the environment (the controlled object and the human operator) in all operational scenarios. It is thus paced by the state changes occurring in the environment.
- Because the state of the controlled object changes as a function of real-time, an observation is *temporally accurate* only for a limited time interval.
- A *trigger* is an event that causes the start of some action, e.g., the execution of a task or the transmission of a message.
- Real-time systems have only small data files, the *real-time database* that is formed by the temporally accurate images of the RT-entities. The key concern is on the *short-term* temporal accuracy of the real-time database that is invalidated by the flow of real-time.
- The real-time database must be updated whenever an RT entity changes its value. This update can be performed periodically, triggered by the progression of the real-time clock by a fixed period (*time-triggered observation*), or immediately after the occurrence of an event in the RT entity (*event-triggered observation*).
- The most stringent temporal demands for real-time systems have their origin in the requirements of the control loops.
- The temporal behavior of a simple controlled object can be characterized by *process lag* and *rise time* of the *step-response function*.
- The *dead time* of a control loop is the time interval between the observation of the RT entity and the start of a reaction of the controlled object as a consequence of a computer action based on this observation.
- Many control algorithms are based on the assumption that the delay jitter is a very small fraction of the *delay* since control algorithms are designed to

compensate a known constant delay. Delay jitter brings an additional uncertainty into the control loop that has an adverse effect on the quality of control.

- The term *signal conditioning* is used to refer to all processing steps that are needed to get a meaningful RT image of an RT entity from the raw sensor data.
- The *Reliability* $R(t)$ of a system is the probability that a system will provide the specified service until time t , given that the system was operational at $t = t_o$.
- If the failure rate of a system is required to be about 10^{-9} failures/h or lower, then we are dealing with a system with an *ultrahigh reliability* requirement.
- Safety is reliability regarding *malign (critical) failure modes*. In a malign failure mode, the cost of a failure can be orders of magnitude higher than the utility of the system during normal operation.
- *Maintainability* is a measure of the time it takes to repair a system after the last experienced benign failure, and is measured by the probability $M(d)$ that the system is restored within a time interval d after the failure.
- *Availability* is a measure for the correct service delivery regarding the alternation of correct and incorrect service, and is measured by the probability $A(t)$ that the system is ready to provide the service at time t .
- The main security concerns in real-time systems are the *authenticity*, *integrity*, and *timeliness* of the real-time information.
- The probability of failure of a perfect system with guaranteed response is reduced to the probability that the assumptions concerning the peak load and the number and types of faults are valid in reality.
- If we start out from a specified fault- and load-hypothesis and deliver a design that makes it possible to reason about the adequacy of the design without reference to probabilistic arguments (even in the case of the extreme load and fault scenarios) we speak of a system with a *guaranteed response*.
- An embedded real-time computer system is part of a well-specified larger system, an *intelligent product*. An intelligent product normally consists of a mechanical subsystem, the controlling embedded computer, and a man-machine interface.
- The static configuration, known *a priori*, of the intelligent product can be used to reduce the resource requirements and increase the robustness of the embedded computer system.
- Usually, every plant automation system is unique. Compared to development cost, the production cost (hardware cost) of a plant automation system is less important.
- The embedded system market is expected to grow significantly during the next 10 years. Compared with other information technology markets, this market will offer the best employment opportunities for the computer engineers of the future.

Bibliographic Notes

There exist a number of textbooks on real-time and embedded systems, such as *Introduction to Embedded Systems – A Cyber-Physical Systems Approach* [Lee10] by Ed Lee and Seshia, *Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems* [Mar10] by Peter Marwedel, *Real-Time Systems* by

Jane Liu [Liu00], *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications* by Giorgio Buttazzo [But04], and *Real-Time Systems and Programming Languages: Ada, Real-Time Java and C/Real-Time POSIX* by Burns and Wellings [Bur09]. The Journal *Real-Time Systems* by Springer publishes archival research articles on the topic.

Review Questions and Problems

- 1.1 What makes a computer system a *real-time* computer system?
- 1.2 What are typical functions that a real-time computer system must perform?
- 1.4 Where do the *temporal requirements* come from? What are the parameters that describe the temporal characteristics of a controlled object?
- 1.5 Give a “rule of thumb” that relates the *sampling period* in a quasi-continuous system to the *rise time* of the step-response function of the controlled object.
- 1.6 What are the effects of delay and delay jitter on the quality of control? Compare the error-detection latency in systems with and without jitter.
- 1.7 What does *signal conditioning* mean?
- 1.8 Consider an RT entity that changes its value periodically according to $v(t) = A_o \sin(2\pi t/T)$ where T , the period of the oscillation, is 100 ms. What is the maximum change of value of this RT entity within a time interval of 1 ms? (express the result in percentage of the amplitude A_o).
- 1.9 Consider an engine that rotates with 3,000 rpm. By how many degrees will the crankshaft turn within 1 ms?
- 1.10 Give some examples where the predictable rare-event performance determines the utility of a hard real-time system.
- 1.11 Consider a fail-safe application. Is it necessary that the computer system provides guaranteed timeliness to maintain the safety of the application? What is the level of error-detection coverage required in an ultrahigh dependability application?
- 1.12 What is the difference between availability and reliability? What is the relationship between maintainability and reliability?
- 1.13 When is there a simple relation between the MTTF and the failure rate?
- 1.14 Assume you are asked to certify a safety-critical control system. How would you proceed?
- 1.15 What are the main differences between a soft real-time system and a hard real-time system?
- 1.16 Why is an *end-to-end protocol* required at the interface between the computer system and the controlled object?
- 1.17 What is the fraction *development cost/production cost* in embedded systems and in plant automation systems? How does this relation influence the system design?
- 1.19 Assume that an automotive company produces 2,000,000 electronic engine controllers of a special type. The following design alternatives are discussed:

- (a) Construct the engine control unit as a single SRU with the application software in Read Only Memory (ROM). The production cost of such a unit is \$250. In case of an error, the complete unit has to be replaced.
- (b) Construct the engine control unit such that the software is contained in a ROM that is placed on a socket and can be replaced in case of a software error. The production cost of the unit without the ROM is \$248. The cost of the ROM is \$5.
- (c) Construct the engine control unit as a single SRU where the software is loaded in a Flash EPROM that can be reloaded. The production cost of such a unit is \$255.

The labor cost of repair is assumed to be \$50 for each vehicle. (It is assumed to be the same for each one of the three alternatives). Calculate the cost of a software error for each one of the three alternative designs if 300,000 cars have to be recalled because of the software error (example in Sect. 1.6.1). Which one is the lowest cost alternative if only 1,000 cars are affected by a recall?

- 1.20 Estimate the relation (development cost)/(production cost) in an embedded consumer application and in a plant automation system.
- 1.21 Compare the peak load (number of messages, number of task activations inside the computer) that can be generated in an event-triggered and a time-triggered implementation of an elevator control system!

Chapter 2

Simplicity

Overview A recent report on *Software for Dependable Systems: Sufficient Evidence?* [Jac07] by the National Academies contains as one of its central recommendations: *One key to achieving dependability at reasonable cost is a serious and sustained commitment to simplicity, including simplicity of critical functions and simplicity in system interactions. This commitment is often the mark of true expertise.* We consider *simplicity* to be the antonym of *cognitive complexity* (in the rest of this book we mean *cognitive complexity* whenever we use the word *complexity*). In every-day life, many embedded systems seem to move in the opposite direction. The ever-increasing demands on the functionality, and the non-functional constraints (such as safety, security, or energy consumption) that must be satisfied by embedded systems lead to a growth in system complexity.

In this chapter we investigate the notion of cognitive complexity and develop guidelines for building *understandable* computer systems. We ask the question: *What does it mean when we say we understand a scenario?* We argue that it is not the embedded system, but the *models* of the embedded system that must be simple and understandable relative to the *background knowledge* of the observer. The models must be based on clear concepts that capture the relevant properties of the scenario under investigation. The *semantic content* of a program variable is one of these concepts that we investigate in some detail. The major challenge of design is the building of an *artifact* that can be modeled at different levels of abstraction by models of adequate simplicity.

This chapter is structured as follows. Section 2.1 focuses on the topic of cognition and problem solving and an elaboration of the two different human problem-solving subsystems, the *intuitive-experiential* subsystem and the *analytic-rational* subsystem. *Concept formation* and the *conceptual landscape*, that is the private knowledge base that a human develops over his lifetime, are discussed in Sect. 2.2. Section 2.3 looks at the essence of model building and investigates what makes a task difficult. Section 2.4 deals with the important topic of emergence in large systems.

2.1 Cognition

Cognition deals with the study of thought processes and the interpretation and binding of sensory inputs to the existing knowledge base of an individual [Rei10]. It is an interdisciplinary effort that stands between the humanities, i.e., philosophy, language studies, and social science on one side and the natural sciences, such as neural science, logic, and computer science on the other side. The study of model building, problem solving, and knowledge representation forms an important part of the cognitive sciences.

2.1.1 Problem Solving

Humans have two quite different mental subsystems for solving problems: the *intuitive-experiential subsystem* and the *analytic-rational subsystem* [Eps08]. Neuro-imaging studies have shown that these two subsystems are executed in two different regions of the human brain [Ami01]. Table 2.1 compares some of the distinguishing characteristics of these two subsystems.

Example: A typical task for the *intuitive-experiential subsystem* is *face recognition*, a demanding task that a baby at the age of 6 months can accomplish. A typical task for the *analytic-rational subsystem* is the *confirmation of a proof of a mathematical theorem*.

The experiential subsystem is a preconscious emotionally-based subsystem that operates holistically, automatically, and rapidly, and demands minimal cognitive resources for its execution. Since it is nearly effortless, it is used most of the time. It is assumed that the experiential subsystem has access to a large coherent

Table 2.1 Intuitive experiential versus analytic rational. Problem solving strategy (Adapted from [Eps08, p. 26])

Intuitive experiential	Analytic rational
Holistic	Analytic
Emotional (what feels good)	Logical reason oriented (what is sensible?)
Unreflective associative connections	Cause and effect connections, causal chains
Outcome oriented	Process oriented
Behavior mediated by vibes from past experience	Behavior mediated by conscious appraisal of events
Encodes reality in concrete images, metaphors and narratives	Encodes reality in abstract symbols, words, and numbers
More rapid processing, immediate action	Slower processing, delayed action
Slow to change the fundamental structure: changes with repetitive or intense experience	Changes more rapidly, changes with the speed of thought
Experience processed passively and pre-consciously, seized by our emotions	Experience processed actively and consciously, in control of our thoughts
Self evidently valid: <i>seeing is believing</i>	Requires justification via logic and evidence

knowledge base that represents an implicit model of the world. This subjective knowledge base, which is one part of what we call the *conceptual landscape* of an individual, is mainly built up and maintained by experience and emotional events that are accumulated over the lifetime of an individual. Although this knowledge base is continually adapted and extended, its core structure is rather rigid and cannot be changed easily. *Experiential reasoning* is holistic and has the tendency to use limited information for general and broad classifications of scenarios and subjects (e.g., this is a *good* or *bad* person). The experiential system does assimilate the data about reality in a coherent stable conceptual framework. The concepts in this framework are mostly linked by *unconscious associative connections*, where the *source* of an association is often *unknown*.

The rational subsystem is a conscious analytic subsystem that operates according to the laws of causality and logic. Bunge [Bun08, p. 48] defines a *causality relationship* between a *cause* *C* and an *event* *E* as follows: *If C happens, then (and only then) E is always produced by it*. We try to get an understanding of a dynamic scenario by isolating a *primary cause*, suppressing seemingly irrelevant detail, and establishing a *unidirectional causal chain* between this primary cause and an observed *effect*. If cause and effect cannot be cleanly isolated, such as is the case in a feedback scenario, or if the relationship between cause and effect is *non-deterministic* (see also Sect. 5.6.1 on the definition of *determinism*), then it is more difficult to understand a scenario.

Example: Consider the analysis of a car accident that is caused by *the skidding* of a car. There are a number of conditions that must hold for skidding to occur: the speed of the car, the conditions of the road (e.g., icy road), the conditions of the tires, abrupt manoeuvre by the driver, the non-optimal functioning of the computer based skid-control system, etc. In order to *simplify* the model of the situation (the reality is not simplified) we often *isolate* a primary cause, e.g., the *speed*, and consider the other conditions as secondary.

The rational subsystem is a verbal and symbolic reasoning system, driven by a controlled and noticeable mental effort to investigate a scenario. Adult humans have a conscious *explicit model* of reality in their rational subsystem, in addition to their *implicit model* of reality in the experiential subsystem. These two models of reality coincide to different degrees and form jointly the *conceptual landscape* of an individual. There seem to be a nearly unlimited set of resources in the experiential subsystem, whereas the cognitive resources that are available to the rational subsystem are limited [Rei10].

There are many subtle interrelationships between these two problem-solving subsystems, which form the extremes of a continuum of problem solving strategies where both systems cooperate to arrive at a solution. It is not infrequent that, after unsuccessful tries by the rational subsystem, at first a solution to a problem is produced unconsciously by the experiential subsystem. Afterwards this solution is justified by analytical and logical arguments that are constructed by the rational subsystem.

Similarly, the significance of a new scenario is often recognized at first by the experiential subsystem. At a later stage it is investigated and analyzed by the rational subsystem and rational problem solving strategies are developed. Repeated

encounters of similar problems – the accumulation of *experience* – effortful learning and drill move the problem-solving process gradually from the rational subsystem to the experiential subsystem, thus freeing the cognitive resources that have previously been allocated to the problem solving process in the limited rational subsystem. There exist many practical examples that demonstrate this phenomenon: learning a foreign language, learning a new sport, or learning how to drive a car. It is characteristic for a *domain expert* that she/he has mastered this transition in her/his domain and mainly operates in the effortless experiential mode, where a fast, holistic and intuitive approach to problem solving dominates.

Example: A brain-imaging study of the chess-playing strategy of amateurs versus grandmasters investigated the activity in different sections of the brain immediately after a chess move by the partner. The amateurs displayed the highest activity in the *medial temporal lobe* of the brain, which is consistent with the interpretation that their mental activity is focused on the rational analysis of the new move. The highly skilled grandmasters showed more activity in the *frontal and parietal cortices*, indicating that they are retrieving stored information about previous games from expert memory in order to develop an *understanding of the scenario* [Ami01].

2.1.2 Definition of a Concept

In a changing world, *knowledge* about permanent and characteristic properties of objects and situations must be identified and maintained since such knowledge is of critical importance for survival. This knowledge is acquired by the process of *abstraction*, by which the particular is subordinated to the general, so that what is known about the general is applicable to many particulars. Abstraction is a fundamental task of the human cognitive system.

Example: Face recognition is an example for the powerful process of *abstraction*. Out of many particular images of the face of a person – varying angles of observation, varying distance, changing lighting conditions – characteristic permanent features of the face are identified and stored in order that they can be used in the future to recognize the face again. This demanding abstraction process is executed unconsciously, seemingly without effort, in the experiential subsystem. Only its results are delivered to the rational subsystem.

Abstraction forms categories, where a *category* is a set of elements that share common *characteristic features*. The notion of category is *recursive*: the *elements of a category* can themselves be *categories*. We thus arrive at a hierarchy of categories, going from the concrete to the abstract. At the lowest level we find immediate sensory experiences.

A *concept* is a category that is augmented by a *set of beliefs* about its relations to other categories [Rei10, pp. 261–300]. The set of beliefs relates a *new concept* to already *existing concepts* and provides for an *implicit theory* (a subjective mental model). As a new domain is penetrated, new concepts are formed and linked to the concepts that are already present in the conceptual landscape. A concept is a mental construct of the *generalizable aspects* of a known entity. It has an intension (*What is*

the essence?) and an extension, answering the question as to which *things* and *mental constructs* are exemplars of the concept. A concept can also be considered as a *unit of thought* [Vig62].

2.1.3 Cognitive Complexity

What do we mean when we say *an observer understands a scenario*? It means that the concepts and relationships that are employed in the representation of the scenario have been adequately linked with the conceptual landscape and the methods of reasoning of the observer. *The tighter the links are, the better is the understanding. Understanding* (and therefore *simplicity*) is thus a *relation* between an observer and a scenario, *not a property* of the scenario.

We take the view of Edmonds [Edm00] that *complexity* can only be assigned to *models of physical systems*, but not to the physical systems themselves, no matter whether these physical systems are natural or man made. A physical system has a nearly infinite number of properties – every single transistor of a billion-transistor *system-on-chip* consists of a huge number of atoms that are placed at distinct positions in space. We need to abstract, to build *models* that leave out the seemingly irrelevant detail of the micro-level, in order to be able to reason about properties of interest to us at the macro-level.

What then is a good measure for *the cognitive complexity* of a *model*? We are looking for a quantity that measures the cognitive effort needed to understand the model by a human observer. *We consider the elapsed time needed to understand a model by a given observer a reasonable measure for the cognitive effort and thus for the complexity of a model relative to the observer.* We assume that the *given observer* is representative for the intended user group of the model.

According to the *scientific tradition*, it would be desirable to introduce an objective notion of cognitive complexity without reference to the subjective human experience. However, this does not seem to be possible, since cognitive complexity refers to a *relation* between an objective external scenario and the subjective internal conceptual landscape of the observer.

The perceived complexity of a model depends on the relationship between the existing subjective conceptual landscape and the problem solving capability of the observer versus the concepts deployed in the representation of the model, the interrelations among these concepts and the notation used to represent these concepts. If the observer is an expert, such as the chess grandmaster in the previous example, the experiential subsystem provides an understanding of the scenario within a short time and without any real effort. According to our metric, the scenario will be judged as *simple*. An amateur has to go through a tedious cause-and-effect analysis of every move employing the rational subsystem that takes time and explicit cognitive effort. According to the above metric, the same chess scenario will be judged as *complex*.

There are models of behavior and tasks that are *intrinsically difficult to comprehend* under any kind of representation. The right column of Table 2.2 in Sect. 2.5 lists some characteristics of intrinsically difficult tasks. It may take a long time, even for an expert in the field, to gain an understanding of a model that requires the comprehension of the behavior of difficult tasks – if at all possible. According to the introduced metric, these models are classified as exceedingly complex.

In order to gain an understanding of a large system we have to understand many models that describe the system from different viewpoints at different abstraction levels (see also Sect. 2.3.1). The cognitive complexity of a large system depends on the number and complexity of the different models that must be comprehended in order to understand the complete system. The time it takes to understand all these models can be considered as a measure for the *cognitive complexity of a large system*.

Case studies about the understanding of the behavior of large systems have shown that the *perceptually available information* plays an important role for developing an understanding of a system [Hme04]. *Invisible information flows* between *identified subsystems* pose a considerable barrier to understanding.

If every embedded system is one of its kind and no relationships between different instances of systems can be established, then there is hardly a chance that experience-based expert knowledge can be developed and the transition from the tedious and effortful rational subsystem to the effortless experiential subsystem can take place.

One route to simplification is thus the development of a *generic model of an embedded system* that can be successfully deployed in many different domains at a proper level of abstraction. This model should contain few orthogonal mechanisms that are used recursively. The model must support simplification strategies and make public the internal information flow between identified subsystems, such that the process of gaining an understanding of the behavior is supported. By getting intimately acquainted with this model and gaining experience by using this model over and over again, the engineer can incorporate this model in the experiential subsystem and become an expert. It is one stated goal of this book to develop such a generic cross-domain model of embedded systems.

2.1.4 Simplification Strategies

The resources in the rational problem solving subsystem of humans, both in storage and processing capacity, are limited. The seminal work of Miller [Mil56] introduced a limit of five to seven chunks of information that can be stored in short-term memory at a given instant. Processing limitations are established by the *relational complexity theory* of Halford [Hal96]. Relational complexity is considered to correspond to the *arity* (number of arguments) of a relation. For example, binary relations have two arguments as in LARGER-THAN (elephant, mouse). The

relational complexity theory states that the upper limits of adult cognition seem to be relations at the quaternary level.

If a scenario requires cognitive resources that are beyond the given limits, then humans tend to apply simplification strategies to reduce the problem size and complexity in order that the problem can be tackled (possibly well, possibly inadequately) with the limited cognitive resources at hand. We know of four strategies to simplify a complex scenario in order that it can be processed by the limited cognitive capabilities of humans: *abstraction*, *partitioning*, *isolation*, and *segmentation*:

- *Abstraction* refers to the formation of a higher-level concept that captures the essence of the problem-at-hand and reduces the complexity of the scenario by omitting irrelevant detail that is not needed, given the purpose of the abstraction. Abstraction is applied *recursively*.
- *Partitioning* (also known as *separation of concerns*) refers to the *division* of the problem scenario into nearly independent parts that can be studied successfully in isolation. Partitioning is at the core of *reductionism*, the preferred simplification strategy in the natural sciences over the past 300 years. Partitioning is not always possible. It has its limits when *emergent properties* are at stake.
- *Isolation* refers to the suppression of seemingly irrelevant detail when trying to find a *primary cause*. The primary cause forms the starting point of the causal chain that links a sequence of events between this primary cause and the observed effect. There is a danger that the simplification strategy of *isolation* leads to a too simplistic model of reality (see the example on skidding of a car in Sect. 2.1.1).
- *Segmentation* refers to the *temporal decomposition* of intricate behavior into smaller parts that can be processed *sequentially*, one after the other. Segmentation reduces the amount of information that must be processed in parallel at any particular instant. Segmentation is difficult or impossible if the behavior is formed by highly concurrent processes, depends on many interdependent variables and is strongly non-linear, caused by positive or negative feedback loops.

2.2 The Conceptual Landscape

The notion of *conceptual landscape*, or the *image* [Bou61], refers to the *personal knowledge base* that is built up and maintained by an individual in the experiential and rational subsystem of the mind. The knowledge base in the experiential subsystem is *implicit*, while the knowledge base in the rational subsystem is *explicit*. The conceptual landscape can be thought of as a structured network of interrelated *concepts* that defines the *world model*, the *personality*, and the *intentions* of an individual. It is built up over the lifetime of an individual, starting from pre-wired structures that are established during the development of the *genotype* to the *phenotype*, and continually augmented as the individual interacts with its environment by exchanging messages via the sensory systems.

2.2.1 Concept Formation

The formation of concepts is governed by the following two principles [And01]:

- The *principle of utility* states that a new concept should encompass those properties of a scenario that are of utility in achieving a stated purpose. The purpose is determined by the human desire to fulfill basic or advanced needs.
- The *principle of parsimony* (also called *Occam's razor*) states that out of a set of alternative conceptualizations that are of comparable utility the one that requires the least amount of mental effort is selected.

There seems to be a *natural level of categorization*, neither too specific nor too general, that is used in human communication and thinking about a domain. We call the concepts at this natural level of categorization *basic-level concepts* [Rei01, p. 276].

Example: The basic level concept *temperature* is more fundamental than the sub-concept *oil-temperature* or the encompassing concept *sensor data*.

Studies with children have shown that basic-level concepts are *acquired earlier* than *sub-concepts* or *encompassing* concepts. As a child grows up it continually builds and adds to its *conceptual landscape* by observing regularities in the perceptions and utility in grouping properties of perceptions into new categories [Vig62]. These new categories must be interlinked with the already existing concepts in the child's mind to form a consistent *conceptual landscape*. By abstracting not only over perceptions, but also over already existing concepts, new concepts are formed.

A new concept requires for its formation a number of experiences that have something in common and form the basis for the abstraction. *Concept acquisition* is normally a bottom-up process, where sensory experiences or basic concepts are the starting point. Examples, prototypes and feature specification play an important role in concept formation. A more *abstract concept* is understood best *bottom up* by generalizations from a set of a suitable collection of examples of already acquired concepts. Abstract analysis and concrete interpretation and explanation should be intertwined frequently. If one remains only at a *low-level of abstraction* then the amount of non-essential detail is overwhelming. If one remains only at a *high-level of abstraction*, then relationships to the world as it is experienced are difficult to form.

In the *real world* (in contrast to an *artificial world*), a *precise definition* of a concept is often not possible, since many concepts become fuzzy at their boundaries [Rei10, p. 272].

Example: How do you define the concept of *dog*? What are its characteristic features?
Is a dog, which has lost a leg, still a dog?

Understanding a new concept is a matter of establishing connections between the new concept and already familiar concepts that are well embedded in the conceptual landscape.

Example: In order to understand the new concept of *counterfeit money*, one must relate this new concept to the following already familiar concepts: (1) the concept of *money*,

(2) the concept of a *legal system*, (3) the concept of a *national bank* that is *legalized* to print money and (4) the concept of *cheating*. A *counterfeit money bill* looks like an *authentic money bill*. In this situation, *examples* and *prototypes* are of limited utility.

In the course of cognitive development and language acquisition, *words (names)* are associated with concepts. The *essence of a concept* associated with a word can be assumed to be the *same* within a natural language community (*denotation*), but different individuals may associate different *shades of meaning* with a concept (*connotation*), dependent on their *individual existing conceptual landscape* and the differing personal emotional experiences in the acquisition of the concept.

Example: If communicating partners refer to different concepts when using a word or if the concept behind a word is not well established in the (scientific) language community, (i.e., does not have a well-defined denotation), then effective communication among partners becomes difficult to impossible.

If we change the language community, the names of concepts will be changed, although the *essence of the concept, its semantic content*, remains the same. The names of concepts are thus relative to the context of discourse, while the *semantic content* remains invariant.

Example: The semantic *content* of the concept *speed* is precisely defined in the realm of physics. Different language communities give different names to the same concept: in German *Geschwindigkeit*, in French *vitesse*, in Spanish *velocidad*.

2.2.2 Scientific Concepts

In the world of science, new concepts are introduced in many publications in order to be able to express new *units of thought*. Often these concepts are named by a *mnemonic*, leading to, what is often called, *scientific jargon*. In order to make an exposition *understandable*, new concepts should be introduced sparingly and with utmost care. A new scientific concept should have the following properties [Kop08]:

- *Utility*. The new concept should serve a useful well-defined purpose.
- *Abstraction and Refinement*. The new concept should abstract from lower-level properties of the scenario under investigation. It should be clear which properties *are not parts* of the concept. In the case of refinement of a *basic-level concept*, it should be clearly stated what additional aspects are considered in the refined concept.
- *Precision*. The characteristic properties of the new concept must be precisely defined.
- *Identity*. The new concept should have a distinct identity and should be significantly different from other concepts in the domain.
- *Stability*. The new concept should be usable uniformly in many different contexts without any qualification or modification.

- *Analogy*. If there is any concept in the existing *conceptual landscape* that is, in some respects, analogous to the new concept, this similarity should be pointed out. The analogy helps to establish links to the *existing conceptual landscape* of a user and facilitates understanding. According to [Hal96, p. 5]:

Analogical reasoning mechanisms are important to virtually every area of higher cognition, including language comprehension, reasoning and creativity. Human reasoning appears to be based less on an application of formal laws of logic than on memory retrieval and analogy.

The availability of a useful, well defined, and stable set of concepts and associated terms that are generally accepted and employed by the scientific community is a mark for the maturity of a scientific domain. An ontology is a shared taxonomy that classifies terms in a way useful to a specific application domain in which all participants share similar levels of understanding of the meaning of the terms [Fis06, p. 23]. Progress in a field of science is intimately connected with concept formation and the establishment of a well-defined ontology.

Example: The main contributions of Newton in the field of mechanics are not only in the formulation of the laws that bear his name, but also in the isolation and conceptualization of the abstract notions *power*, *mass*, *acceleration* and *energy* out of an unstructured reality.

Clear concept formation is an essential prerequisite for any formal analysis or formal verification of a given scenario. The mere replacement of fuzzy concepts by formal symbols will not improve the understanding.

2.2.3 *The Concept of a Message*

We consider a *message* as a *basic concept* in the realm of communication. A message is an *atomic unit* that captures the value domain and the temporal domain of a unidirectional information transport at a level of abstraction that is applicable in many diverse scenarios of human communication [Bou61] and machine communication. A basic message transport service (BMTS) transports a message from a sender to one or a set of receivers. The BMTS can be realized by different means, e.g., biological or electrical.

For example, the message concept can be used to express the information flow from the human sensory system to the conceptual landscape of an individual. The message concept can also model the indirect high-level interactions of a human with his environment that are based on the use of language.

Example: We can model the sensory perception, e.g. of temperature, by saying that a message containing the sensed variable (temperature) is sent to the *conceptual landscape*. A message could also contain verbal information about the temperature at a location that is outside the realm of direct sensory experience.

The message concept is also a *basic concept* in the domain of distributed embedded computer systems at the architecture level. If the BMTS between encapsulated sub-systems is based on *unidirectional temporally predictable multicast messages*, then

the data aspect, the timing aspect, the synchronization aspect, and the publication aspect are integrated in a single mechanism. The BMTS can be refined at a lower level of abstraction by explaining the transport mechanism. The transport mechanism could be wired or wireless. The information can be coded by different signals. These refinements are relevant when studying the implementation of the message mechanism at the physical level, but are irrelevant at a level where the only concern is the timely arrival of the information sent by one partner to another partner.

A *protocol* is an abstraction over a sequence of *rule-based* message exchanges between communicating partners. A protocol can provide additional services, such as flow control or error detection. A protocol can be understood by breaking it down to the involved messages without the need to elaborate on the concrete transport mechanisms that are used.

2.2.4 Semantic Content of a Variable

The concept of a *variable*, a fundamental concept in the domain of computing, is of such importance for the rest of the book that it justifies some special elaboration. A variable can be considered as a *language construct* that assigns an *attribute* to a *concept*. If the point in real-time, the *instant*, when this assignment is valid, is of relevance, then we call the *variable* a *state variable*. As time progresses, the attribute of a state variable may change, while the concept remains the same. A variable thus consists of two parts, a *fixed part*, the *variable name* (or the *identifier*), and a *variable part* called the *value of the variable* that is assigned to the variable. The variable name designates the concept that determines *what we are talking about*. In a given context, the variable name – which is analogous to the name of a concept in a natural language community – must be unique and point to the same concept at all communicating partners. The meaning that is conveyed by a variable is called the *semantic content* of the variable. As we will show in the latter part of this section, the semantic content of a variable is *invariant* to a change in representation. The requirement of *semantic precision* demands that the concept that is associated with a variable name and the domain of values of the variable are unambiguously defined in the model of the given application.

Example: Consider the variable name *engine-temperature* that is used in an automotive application. This concept is too abstract to be meaningful to an automotive engineer, since there are different temperatures in an automotive engine: the temperature of the oil, the temperature of the water, or the temperature in the combustion chamber of the engine.

The unambiguous definition of a concept does not only relate to the meaning of the concept associated with the variable, but also to the specification of the *domain of values* of the variable. In many computer languages, the *type of a variable*, which is introduced as an attribute of the variable name, specifies primitive attributes of the value domain of the variable. These primitive attributes, like *integer* or *floating point number*, are often not sufficient to properly describe all relevant attributes of the value domain. An extension of the type system will alleviate the problem.

Example: If we declare the *value domain* of the variable *temperature* to be *floating point* we still have not specified whether the temperature is measured in units of *Celsius*, *Kelvin* or *Fahrenheit*.

Example: The Mars Climate Orbiter crash occurred because the ground-based software used different system units than the flight software. The first of the recommendations in the report of the mishap investigation board was *that the MPL (Mars Polar Lander) project verify the consistent use of units throughout the MPL spacecraft design and operation* [NAS99].

In different language communities, different variable names may be used to point to the same concept. For example, in an English speaking language community *the temperature of the air* may be abbreviated by *t-air*, while a German speaking community may call it *t-luft*. If we change the representation of the value domain of a variable, e.g., if we replace the units for measuring the temperature from *Celsius* to *Fahrenheit* and adapt the value of the variable accordingly, the *semantic content* expressed by the variable remains the same.

Example: On the *surface* the two variables $t\text{-air} = 86$ and $t\text{-luft} = 30$ are completely different since they have different names and different values. If, however, *t-air* and *t-luft* refer to the same concept, i.e., the temperature of the air, and the value of *t-air* is expressed in degrees Fahrenheit and that of *t-luft* in degrees Celsius, then it becomes evident that the *semantic contents* of these two variables are the same.

These differences in the representations of the *semantic content* of a variable become important when we look at *gateway components* which link two subsystems of a *system of systems* that have been developed by two different organizations according to two different *architectural styles*. The term *architectural style* refers to all *explicit* and *implicit* principles, rules and conventions that are followed by an organization in the design of a system, e.g., the representation of data, protocols, syntax, naming, and semantics, etc.. The gateway component must translate the variable names and representations from one architectural style to the other architectural style, while keeping the semantic content *invariant*.

Data that describe the properties of (*object*) *data* is sometimes called *meta-data*. In our model of a variable, data that describes the properties of the *fixed parts of a variable* is *meta data*, while the *variable part* of a variable, the *value set*, is (*object*) *data*. *Meta data* thus describes the properties of the concept that is referred to by the variable name. Since *meta data* can become *object data* of another level, the distinction between *data* and *meta data* is relative to the viewpoint of the observer.

Example: The price of a product is *data*, while the currency used to denote the price, the time interval and the location where this price is applicable are *meta data*.

2.3 The Essence of Model Building

Given the rather limited cognitive capabilities of the rational subsystem of the human mind we can only develop a rational understanding of the world around us if we build *simple models* of those properties that are of relevance and interest to

us and disregard (abstract from) detail that proves to be irrelevant for the given purpose. *A model is thus a deliberate simplification of reality with the objective of explaining a chosen property of reality that is relevant for a particular purpose.*

Example: The purpose of a model in *Celestial Mechanics* is the explanation of the movements of the heavenly bodies in the universe. For this purpose it makes sense to introduce the abstract concept of a *mass point* and to reduce the whole diversity of the world to a single *mass point in space* in order that the interactions with other mass points (heavenly bodies) can be studied without any distraction by unnecessary detail.

When a new level of abstraction (a new model) is introduced that successfully conceptualizes the properties relevant for the given purpose and disregards the rest, *simplicity* emerges. Such simplicity, made possible by the formation of proper concepts, give rise to new insights that are at the roots of the *laws of nature*. As Popper [Pop68] points out, due to the inherent imperfection of the *abstraction* and *induction* process, laws of nature can only be falsified, but never be proven to be absolutely correct.

2.3.1 Purpose and Viewpoint

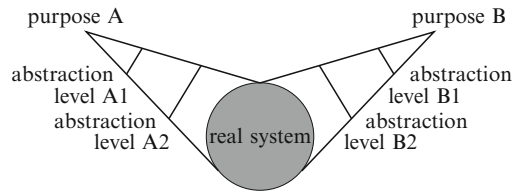
At the start of any modeling activity, a clear purpose of the model must be established. Formulating the precise questions the model must address helps to concretize the purpose of the model. If the purpose of a model is not crystal clear, or if there are multiple divergent purposes to satisfy, then it is not possible to develop a *simple* model.

Example: The purpose of a *model of behavior* of a real-time computer system is to provide answers to the question at *what points in real-time* will the computer system produce *what kind of outputs*. If our computer system is a *System-on-Chip (SoC)* with a billion transistors, then we must find a hierarchy of behavioral models to meet our purpose.

The recursive application of the *principles of abstraction* leads to such a hierarchy of models that Hayakawa [Hay90] calls the *abstraction ladder*. Starting with *basic-level concepts* that are essential for understanding a domain, more general concepts can be formed by *abstraction* and more concrete concepts can be formed by *refinement*. At the lowest level of the abstraction ladder are the direct sensory experiences.

Example: The *Four Universe Model* of Avizienis [Avi82] introduces a hierarchy of models in order to simplify the description of the behavior of a computer system. At the lowest level of the hierarchy, the *physical level*, the analog signals of the circuits are observed, such as the rise time of the voltages as a transistor performs a switching operation. The analysis of a circuit behavior at the physical (analog) level becomes difficult as soon as more and more transistors get involved (*emerging complexity*). The next higher level, the *digital logic level*, abstracts from the physical analog quantities and the dense time and introduces binary logic values (*high* or *low*) of signals at discrete instants, resulting in a much simpler representation of the behavior of an elementary circuit, e.g. an AND gate (*emerging simplicity*). Complexity creeps in again as we combine more and

Fig. 2.1 Purpose and abstraction level of a model



more logic circuits. The next higher level, the *information level*, lumps a (possible large) sequence of binary values into a meaningful data structure, (e.g., a pointer, a real-valued variable or a complete picture) and introduces powerful high-level operations on these data structures. Finally, at the *external level*, only the services of the computer system to the environment, as seen by an outside user, are of relevance.

A posed question about a distinct property of a *real system* gives rise to the construction of a hierarchy of models of that system that are intended to answer the posed question. Figure 2.1 depicts two hierarchies of models that are introduced to serve two purposes, purpose A and purpose B. Purpose A could refer to a hierarchy of behavioral models, while purpose B could refer to a hierarchy of dependability models of the same *real system*. At the top of each hierarchy is the stated purpose, i.e., the questions that must be answered. The different levels of the hierarchy – the abstraction levels – are introduced to support a stepwise refinement of the stated question considering more detail, where each step takes consideration of the limited cognitive capabilities of the human mind. At the low end of the hierarchy is the real system. The analysis is substantially simplified if the structure of the model corresponds with the structure of the system. Otherwise we have to resolve a structure clash that complicates the issues.

Example: The model for predicting the temporal properties of the behavior of a real-time computer system is straightforward if there is a predictable sequence of computational and communication actions between the start of a computation and the termination of a computation. Conversely, if the actual durations of the computational and communication actions depend on global system activity (e.g., arbitration for access to shared resources such as caches, communication links, etc.) then it will not be possible to construct a *simple model* for predicting the temporal properties of the behavior.

2.3.2 The Grand Challenge

Whereas the natural scientist must uncover the regularities in a given reality and find appropriate concepts at a suitable level of abstraction in order to formulate models and theories that explain the observed phenomena, the computer scientist is – *at least theoretically* – in a much better position: The computer scientist has the freedom to design the system – *an artifact* – which is the subject of his modeling. The requirement to build artifacts, the properties of which can be analyzed by *simple models*, should thus be an explicit design driver. In many areas of computer science this principle of building artifacts that can be modeled by *simple models* is violated.

For example, the temporal behavior of a modern pipelined microprocessor with multiple caches cannot be captured in a *simple model*.

The major challenge of design is the building of a software/hardware artifact (an embedded computer system) that provides the intended behavior (i.e. the service) under given constraints and where relevant properties of this artifact (e.g., the behavior) can be modeled at different levels of abstraction by models of adequate simplicity.

As stated before, there are many different purposes that give rise to a hierarchy of models of an artifact. Examples are: behavior, reliability, man–machine interaction, energy consumption, physical dimension, cost of manufacturing, or cost of maintenance, to name a few. Out of these, the most important one is the *model of behavior*. In the context of real-time systems, behavior specifies the output actions of a computer system as a consequence of the inputs, the state and the progression of real-time. Output actions and input can be captured in the concepts of *input messages* and *output messages*. In Chap. 4 of this book we present a cross-domain model for the behavior of a real-time computer system using these concepts.

2.4 Emergence

We speak of *emergence* when the interactions of subsystems give rise to unique global properties at the system level that are not present at the level of the subsystems [Mor07]. Non-linear behavior of the subsystems, feedback and feed forward mechanisms, and time delays are of relevance for the appearance of emergent properties. Up to now, the phenomenon of emergence is not fully understood and a topic of intense study.

2.4.1 Irreducibility

Emergent properties are irreducible, holistic, and novel – they disappear when the system is partitioned into its subsystem. Emergent properties can appear unexpectedly or they are planned. In many situations, the first appearance of the emergent properties is unforeseen and unpredictable. Often a fundamental revision of state-of-the-art models is required to get a better understanding of the conditions that lead to the intended emergence. In some cases, the emergent properties can be captured in a new conceptualization (model) at a higher level of abstraction resulting in an *abrupt simplification* of the scenario.

Example: The emergent properties of a *diamond*, such as *brilliance* and *hardness*, which are caused by the *coherent* alignment of the Carbon-atoms, are substantially different from the properties of graphite (which consists of the *same atoms*). We can consider the diamond with its characteristic properties a new concept, a *new unit of thought*, and forget about its composition and internal structure. Simplicity comes out as a result of the intricate interactions among the elements that help to generate a *new whole* with its new emergent properties.

2.4.2 Prior and Derived Properties

When dealing with emergence, it is helpful to distinguish between the *prior properties* of the components and the new *derived properties* that come about by the interactions of the components.

Example: The high reliability of the services of a fault-tolerant system (*derived property*) that is the result of the interactions of many unreliable components (*prior property*) is an emergent property.

In many cases the prior properties and the derived properties can be of a completely different kind. It often happens that the *derived properties* open a completely new domain of science and engineering. This new domain requires the formation of novel concepts that capture essential properties of this new domain.

Example: The property of *being able to fly* which comes about by the proper interaction of the subsystems of an airplane, such as the wings, the fuselage, the engines and the controls, is only present in the airplane as a whole but not in any of the isolated subsystems. *Being able to fly* has opened the domain of the air transportation industry with its own rules and regulations. For example, the subject of *air traffic control* is far removed from the prior properties of the components that make up an airplane.

Prior properties and derived properties are relative to the viewpoint of the observer. When climbing up the abstraction ladder, the derived properties at one level of abstraction become the prior properties at the next higher level of abstraction and so on, since a new form of emergence can appear at higher levels.

Example: In the evolution of the universe two very significant stages of emergence are the *appearance of life* and at a further stage the *appearance of consciousness* that forms the basis for the development of human culture. The realm of human culture has developed its own system of concepts in the arts, sciences etc., that are far removed from the biological prior properties that are characterizing the human brain.

Emergent behavior cannot be predicted analytically, but must be detected in an operating system. Thus control elements must incorporate hooks for monitoring system performance in real time [Par97, p. 7]. The multicast message concept, discussed in Sect. 2.2.3 provides the basis for the nonintrusive observation of system behavior.

2.4.3 Complex Systems

We classify a system as *complex* if we are not in the position to develop a set of models of *adequate simplicity* – commensurate to the rational capabilities of the human mind – to explain the structure and behavior of the system. In addition to *life* and *consciousness*, examples for complex systems are the earth's climate and weather, the global economy, living organisms, and many large computer systems, to name a few.

We hold the opinion that a fundamental understanding of a complex system can only be achieved by a *proper conceptualization* and not by the execution of

elaborate computer simulations. This view is also shared by Mesarovic et al. [Mes04, p.19] when he speaks about biology:

We further argue that for a deeper understanding in systems biology investigations should go beyond building numerical mathematical or computer models – important as they are Such a categorical perspective led us to propose that the core of understanding in systems biology depends on the search for organizing principles rather than solely on construction of predictive descriptions (i.e. models) that exactly outline the evolution of systems in space and time. The search for organizing principles requires an identification/discovery of new concepts and hypotheses.

Maybe, sometimes in the future, we will form appropriate concepts that will lead to an abrupt simplification of some of today’s complex systems. If this happens, the system will not be classified as complex any more.

Whereas system biology deals with a natural system, a large computer system is an *artifact* developed by humans. When designing such an artifact, we should take consideration of the limited rational problem solving capability of humans in order that we can describe the behavior of the artifact by models of adequate simplicity. These models should guide the design process, such that a structure clash between the model and the artifact is avoided.

Example: Let us look at the technical example of designing the on-chip communication infrastructure for the communication among IP-cores on a system-on-chip. There are basically two technical alternatives, the provision of a *shared memory* that can be accessed by all IP-cores or the provision of *local memory to each one of the IP-cores* and the design of a message-passing subsystem that enables the exchange of messages among IP-cores [Pol07, Lev08]. The message-passing subsystem isolates and makes explicit the global communication among subsystems and thus supports the introduction of a new level in the hierarchy where a distinction is made between the *intra-IP core* interactions and the *inter-IP core* interactions. The common memory intermixes global intra-IP-core and local inter-IP-core interactions and makes it very difficult to separate global and local concerns, leading to a more complex system model.

2.5 How Can We Achieve Simplicity?

Cognitive scientists have studied how students learn and understand different tasks [Fel04]. They have identified a set of task characteristics that require a disproportional mental effort for understanding the task. Table 2.2 compares the characteristics of *simple* tasks versus *difficult* tasks. We thus need to design a generic model for expressing the behavior of an embedded system that avoids the characteristics of difficult tasks. It should be possible to apply the model *recursively*, such that large systems can be modeled at different levels of abstraction using the same modeling mechanisms.

The model of a real-time system, presented in Chap. 4, tries to reach this goal. Simplicity is achieved by adhering to the following seven design principles:

1. *Principle of Abstraction.* The introduction of a component (a hardware/software unit) as a basic structural and computational unit makes it possible to use

Table 2.2 Characteristics of simple versus difficult tasks (Adapted from [Fel04, p. 91])

Characteristics of a simple task	Characteristics of a difficult task
Static: The properties of the task do not change over time.	Dynamic: The properties of the task are time dependant.
Discrete: The variables that characterize the task can only take values from discrete sets.	Continuous: The domain of the variables is continuous.
Separable: Different subtasks are nearly independent. There is only a weak interaction among tasks.	Non-separable: Different subtasks are highly interactive. It is difficult to isolate the behavior of a single task.
Sequential: Behavior can be understood by a sequential step-by-step analysis.	Simultaneous: Many concurrent processes interact in generating visible behavior. Step-by-step analysis is difficult.
Homogeneous: Components, explanatory schemes, and representations are alike.	Heterogeneous: Many different components, explanatory schemes, and representations.
Mechanism: Cause and effect relations dominate.	Organicism: Behavior characterized by a multitude of feedback mechanisms.
Linear: Functional relationships are linear.	Non-linear: Functional relationships are non-linear.
Universal: Explanatory principles do not depend on context.	Conditional: Explanatory principles are context dependent.
Regular: Domain characterized by a high regularity of principles and rules.	Irregular: Many different context dependent rules.
Surface: Important principles and rules are apparent by looking at observable surface properties.	Deep: Important principles are covert and abstract and not detectable when looking at surface properties.

- the component on the basis of its precise interface specifications without any need to understand the internals of the component operation. In order to maintain the abstraction of a component even in the case that faults are occurring, a component should be a fault-containment unit (Sect. 6.1.1). If components stand in a hierarchical relationship to each other, different *levels of abstraction* can be distinguished. At a high level of abstraction, the behavior of a complete autonomous *constituent system* (consisting of many clusters of components) of a system-of-systems (SoS) is captured in the precise linking interface specification of its gateway component (see Sects. 4.6 and 4.7.3).
2. *Principle of Separation of Concerns*. This principle helps to build simple systems by disentangling functions that are separable in order that they can be grouped in self-contained architectural units, thus generating *stable intermediate forms* [Sim81]. This principle is sometimes called *principle of partitioning* [Ses08]. An example is the strict separation of computational activities from communication activities such that the communication system and the computational components can be developed independently (Sect. 4.1.1).
 3. *Principle of Causality*. The analytical-rational problem solving subsystem of humans excels in reasoning along causal chains. The *deterministic behavior* of basic mechanisms makes it possible that a causal chain between a cause and the consequent effect can be established without a doubt (Sect. 5.6).

4. *Principle of Segmentation.* This principle suggests that hard-to-understand behavior should be decomposed, wherever possible, into a serial behavioral structure such that a sequential step-by-step analysis of the behavior becomes possible. Each step requires only the investigation of the limited context that is of relevance at this step.
5. *Principle of Independence.* This principle suggests that the interdependence of architectural units (*components* or *clusters*, see Sect. 1.1) should be reduced to the necessary minimum that is required by the application. An example is the provision of a single unidirectional primitive for the communication among components such that any low-level dependency of the sender of a message on the correct operation of the receiver is eliminated by design. This principle is of paramount importance in the design of fault-tolerant systems to ensure that back-propagation of failures is avoided and the independence of failures of fault-containment units can be assumed (Sect. 6.4).
6. *Principle of Observability.* Non-visible communication channels among architectural units pose a severe impediment for the understanding of system behavior. This can be avoided by supporting a multicast topology in the basic message passing primitive. It is then possible to observe the external behavior of any component without a *probe* effect (Sect. 12.2).
7. *Principle of a Consistent Time.* The progression of real-time is an important independent variable in any behavioral model of the physical subsystem of an embedded system. This principle suggests that a *global time base* should be introduced in the distributed computer system such that system-wide consistent temporal relations (e.g., simultaneity) and temporal distances among events can be established on the basis of global time-stamps (Sect. 3.3). The availability of a global time simplifies the solution of many problems in distributed systems (see Sect. 14.2.1).

Points to Remember

- Humans have two quite different mental subsystems for solving problems: the *intuitive-experiential subsystem* and the *analytic-rational subsystem*.
- The *experiential subsystem* is a preconscious emotionally-based subsystem that operates holistically, automatically, and rapidly, and demands minimal cognitive resources for its execution.
- The *rational subsystem* is a conscious analytic subsystem that operates according to the laws of logic. It is well equipped to handle deterministic relations and *causality*.
- Adult humans have a conscious *explicit model* of reality in their rational subsystem, in addition to their *implicit model* of reality in the experiential subsystem. These two models of reality coincide to different degrees and form jointly the *conceptual landscape* of an individual.
- Knowledge is acquired by the process of *abstraction*, by which the particular is subordinated to the general, so that what is known about the general is applicable to many particulars.

- A *concept* is a category that is augmented by a *set of beliefs* about its relations to other categories. The set of beliefs relates a *new concept* to already *existing concepts* and provides for an *implicit theory* (a subjective mental model).
- *Understanding* means that the concepts and relationships that are employed in the representation of a scenario have been adequately linked with the conceptual landscape and the methods of reasoning of the observer. *The tighter the links are, the better is the understanding.* *Understanding* (and therefore *simplicity*) is thus a *relation* between an observer and a scenario, *not a property* of the scenario.
- The elapsed time needed to understand a model by an intended observer is a reasonable measure for the cognitive effort and thus for the *complexity* of a *model relative to the observer*.
- *Complexity* can only be assigned to models of *physical systems*, but not to the physical systems themselves, no matter whether these physical systems are natural or man made.
- The complexity of a large system depends on the number and complexity of the models that must be comprehended in order to understand the complete system. The time it takes to understand all these models can be considered as a measure for the *cognitive complexity of a large system*.
- Invisible information flows between *identified subsystems* pose a considerable barrier for understanding.
- The resources in the rational problem solving subsystem of humans, both in storage and processing capacity, are limited.
- The four strategies to simplify a complex scenario in order that it can be processed by the limited cognitive capabilities of humans are *abstraction, partitioning, isolation, and segmentation*.
- The formation of concepts is governed by the following two principles the *principle of utility* and the *principle of parsimony* (also called *Occam's razor*).
- The *essence of a concept*, i.e., the *semantic content of a concept*, associated with a *name*, can be assumed to be the *same* within a natural language community (*denotation*), but different individuals may associate different *shades of meaning* with a concept (*connotation*).
- A *variable* is a *language construct* that assigns an *attribute* to a *concept* at the given *instant*. A variable thus consists of two parts, a *fixed part*, the *variable name*, and a *variable part* called the *value of the variable* that is assigned to the variable at a particular instant.
- Differences in the representations of the *semantic content* of a variable become important when we look at *gateway components* which link two subsystems that have been developed by two different organizations according to two different *architectural styles*.
- A *model* is a deliberate simplification of reality with the objective of explaining a chosen property of reality that is relevant for a particular purpose.
- If the purpose of a model is not crystal clear, or if there are multiple divergent purposes to satisfy, it is not possible to develop a *simple* model.

- The recursive application of the *principles of abstraction* leads to such a hierarchy of models. More general models can be formed by *abstraction* and more concrete models can be formed by *refinement*.
- The major challenge of design is the building of a software/hardware artifact (an embedded computer system) that provides the intended behavior (i.e. the service) under given constraints and where relevant properties of this artifact (e.g., the behavior) can be modeled at different levels of abstraction by models of adequate simplicity.
- We talk about *emergence* when the interactions of subsystems give rise to unique global properties at the system level that are not present at the level of the subsystems. Emergent properties are irreducible, holistic, and novel – they disappear when the system is partitioned into its subsystems.
- We classify a system as *complex* if we are not in the position to develop a set of models of *adequate simplicity* – commensurate to the rational capabilities of the human mind – to explain the structure and behavior of the system.

Bibliographic Notes

The textbook by Reisberg [Rei10] gives a good overview of the state-of-the-art in the field of cognition and introduces many of the terms that have been used in this chapter. Epstein [Eps08] discusses the characteristics of the *intuitive-experiential subsystem* and the *analytic-rational subsystem* of problem solving. Boulding [Bou61] elaborates extensively on the notion of *conceptual landscape* (which he calls the *Image*) and the role of the *message metaphor* in all types of communication. The hierarchy of concepts, the *abstraction ladder*, is taken from Hayakawa [Hay90]. The relational complexity theory of Halford [Hal96] establishes limits for the rational reasoning capability of humans, while Miller [Mil56] elaborates on the limits of the human short-term memory capacity. Popper [Pop68] and Edmonds [Edm00] discuss the relevance and limitations of model building for understanding physical systems. The book by Bedau [Bed08] is devoted to the topic of emergence. The comparison of simple versus difficult tasks is taken from [Fel04]. The PhD thesis by Rumpler [Rum08] deals with *design comprehension* of embedded real-time systems. Roger Session's book [Ses08] *Simple Architectures for Complex Enterprises* contains practical guidelines for designing understandable Enterprise Information Architectures.

Review Questions and Problems

- 2.1 What are the distinguishing characteristics of the *intuitive-experiential* and the *analytic-rational* subsystems for human problem solving?
- 2.2 Give concrete examples for typical tasks for the *intuitive-experiential* and the *analytic-rational* problem solving subsystems!

- 2.3 How is a *concept* defined? What are the principles that guide *concept formation*? What is the *conceptual landscape*? What are basic level concepts?
- 2.4 What is characteristic for a *domain expert*?
- 2.5 What do we mean when we say we *understand a scenario*? How is *cognitive complexity* defined? Give an example of a *barrier to understanding*!
- 2.6 Which are known *simplification strategies*?
- 2.7 What are the characteristics of scientific concepts?
- 2.8 What is the concept of a message? What is a *protocol*?
- 2.9 What is the *semantic content* of a variable? What is the relationship between *the representation of a variable* and its *semantic content*?
- 2.10 What is the essence of model building?
- 2.11 Explain the *four-universe model* of a computer system!
- 2.12 What makes a task *simple* or *complex*?
- 2.13 What do we mean by *emergence*? What are *prior* and *derivative properties*?
- 2.14 What are the advantages and disadvantages of message-based inter IP-core communication on an MP-SoC (Multiprocessor system on chip)?

Chapter 3

Global Time

Overview This chapter starts with a general discussion on time and order. The notions of causal order, temporal order, and delivery order and their interrelationships are elaborated. The parameters that characterize the behavior and the quality of a digital clock are investigated. Section 3.2 proceeds along the positivist tradition by introducing an omniscient external observer with an absolute reference clock that can generate precise time-stamps for all relevant events. These absolute time-stamps are used to reason about the precision and accuracy of a global time base, and to expose the fundamental limits of time measurement in a distributed real-time system.

In Sect. 3.3, the model of a *sparse time base* is introduced to establish a consistent view of the order of computer-generated events in a distributed real-time system without having to execute an agreement protocol. The *cyclic model of time* presented in this section is well suited to deal with the progression of time in cyclic systems, such as in many control and multimedia systems.

The topic of internal clock synchronization is covered in Sect. 3.4. First, the notions of convergence function and drift offset are introduced to express the synchronization condition that must be satisfied by any synchronization algorithm. Then, the simple central-master algorithm for clock synchronization is presented, and the precision of this algorithm is analyzed. Section 3.4.3 deals with the more complex issue of fault-tolerant distributed clock synchronization. The jitter of the communication system is a major limiting factor that determines the precision of the global time base.

The topic of external synchronization is studied in Sect. 3.5. The role of a time gateway and the problem of faults in external synchronization are discussed. Finally, the network time protocol (NTP) of the Internet, the time format of the IEEE 1588 clock synchronization protocol, and the time format of the TTA are presented.

3.1 Time and Order

Applying the principles of *utility* and *parsimony* (Sect. 2.2.1), we base our model of time on Newtonian physics, because the models of *Newtonian physics* are simpler than the models of *relativistic physics* and sufficient to deal with most temporal phenomena in embedded systems. In many engineering disciplines (e.g., Newtonian mechanics), time is introduced as an independent variable that determines the sequence of states of a system. The basic constants of physics are defined in relation to the standard of time, the physical second. This is why the global time base in a cyber-physical real-time system should be based on the metric of the physical second.

In a typical real-time application, the distributed computer system performs a multitude of different functions concurrently, e.g., the monitoring of real-time (RT) entities (both their value and rate of change), the detection of alarm conditions, the display of the observations to the operator, and the execution of control algorithms to find new set-points for many distinct control loops. These diverse functions are normally executed at different nodes. In addition, replicated nodes are introduced to provide fault tolerance by active redundancy. To guarantee a consistent behavior of the entire distributed system, it must be ensured that all nodes process all events in the same consistent order, preferably in the same temporal order in which the events occurred (see also the example in Sect. 5.5) in the controlled object. A proper global time base helps to establish such a consistent temporal order on the basis of the time-stamps of the events.

3.1.1 Different Orders

Temporal Order. The continuum of Newtonian real time can be modeled by a *directed timeline* consisting of an infinite set $\{T\}$ of *instants* (or *points in time*) with the following properties [Wit90, p. 208]:

1. $\{T\}$ is an ordered set, that is, if p and q are any two instants, then either p is simultaneous with q , or p precedes q , or q precedes p , where these relations are mutually exclusive. We call the order of instants on the timeline the *temporal order*.
2. $\{T\}$ is a dense set. This means that there is at least one q between p and r iff p is not the same instant as r , where p , q , and r are instants.

A section of the time line between two different instants is called a *duration*. In our model, an *event* takes place at an instant of time and does not have a duration. If two events occur at the *same* instant, then the two events are said to occur *simultaneously*. Instants are totally ordered; however, events are only partially ordered, since simultaneous events are not in the order relation. Events can be totally ordered if another criterion is introduced to order events that occur simultaneously, e.g., in a distributed computer system, the number of the node at which the event occurred can be used to order events that occur simultaneously [Lam78].

Causal Order. In many real-time applications, the causal dependencies among events are of interest. The computer system must assist the operator in identifying the *primary event* of an *alarm shower* (see Sect. 1.2.1). Knowledge of the exact temporal order of the events is helpful in identifying this primary event. If an event e_1 occurs after an event e_2 , then e_1 cannot be the cause of e_2 . If, however, e_1 occurs before e_2 , then it is possible, but not certain, that e_1 is the cause of e_2 . The *temporal* order of two events is necessary, but not sufficient, for their *causal* order. Causal order is *more* than temporal order.

Reichenbach [Rei57, p. 145] defined causality by a mark method without reference to time: If event e_1 is a cause of event e_2 , then a small variation (a mark) in e_1 is associated with small variation in e_2 , whereas small variations in e_2 are not necessarily associated with small variations in e_1 .

Example: Suppose there are two events e_1 and e_2 :

e_1 Somebody enters a room.

e_2 The telephone starts to ring.

Consider the following two cases

1. e_2 occurs after e_1 .
2. e_1 occurs after e_2 .

In both cases the two events are temporally ordered. However, while it is unlikely that there is a causal order between the two events of case (1), it is likely that such a causal order exists between the two events of case (2), since the person might enter the room to answer the telephone.

If the (partial) temporal order between alarm events has been established, it is possible to exclude an event from being the *primary event* if it *definitely occurred later* than another alarm event. Subsequently, we will show that a precise global time base helps to determine the event set that is in this *definitely-occurred-later-than* relation (see also the example in Sect. 1.2.1).

Delivery Order. A weaker order relation that is often provided by distributed communication systems is a consistent *delivery order*. The communication system guarantees that all nodes see a defined *set of related events* in the same delivery order. This delivery order is not necessarily related to the temporal order of event occurrences or the causal relationship between events. Some distributed algorithms, e.g., *atomic broadcast algorithms* require a consistent delivery order.

3.1.2 Clocks

In ancient history, the measurement of durations between events was mainly based on subjective judgment. With the advent of modern science, objective methods for measuring the progression of time by using *physical clocks* have been devised.

Digital Physical Clock. A (*digital physical*) *clock* is a device for measuring time. It contains a *counter* and a *physical oscillation mechanism* that periodically generates

an event to increase the counter. The periodic event is called the *microtick* of the clock. (The term *tick* is introduced in Sect. 3.2.1 to denote the events generated by the global time).

Granularity. The duration between two consecutive microticks of a digital physical clock is called a *granule* of the clock. The granularity of a given clock can be measured only if there is a clock with a finer granularity available. The granularity of any digital clock leads to a digitalization error in time measurement.

There also exist analog physical clocks, e.g., *sundials* that do not have granularity. In the following, we only consider digital physical clocks.

In subsequent definitions, we use the following notation: clocks are identified by natural numbers $1, 2, \dots, n$. If we express properties of clocks, the property is identified by the clock number as a superscript with the microtick or tick number as a subscript. For example, microtick i of clock k is denoted by microtick_i^k .

Reference Clock. Assume an *omniscient external observer* who can observe all events that are of interest in a given context (remember that relativistic effects are disregarded). This observer possesses a *unique reference clock* z with frequency f^z , which is in perfect agreement with the international standard of time. The counter of the reference clock is always the same as that of the international time standard. We call $1/f^z$ the *granularity* g^z of clock z . Let us assume that f^z is very large, say 10^{15} microticks/s, so that the granularity g^z is 1 fs (10^{-15} s). Since the granularity of the reference clock is so small, the digitalization error of the reference clock is considered a second order effect and disregarded in the following analysis.

Absolute Time-Stamp. Whenever the omniscient observer perceives the occurrence of an event e , she/he will instantaneously record the current state of the reference clock as the time of occurrence of this event e , and, will generate a *time-stamp* for e . $\text{Clock}(e)$ denotes the time-stamp generated by the use of a given *clock* to time-stamp an event e . Because z is the single reference clock in the system, $z(e)$ is called the *absolute time-stamp* of the event e .

The *duration* between two events is measured by counting the microticks of the reference clock that occur in the interval between these two events. The *granularity* g^k of a given clock k can now be measured and is given by the nominal number n^k of microticks of the reference clock z between 2 microticks of this clock k .

The temporal order of events that occur between any two consecutive microticks of the reference clock, i.e., within the granularity g^z , cannot be reestablished from their absolute time-stamps. This is a fundamental limit in time measurement.

Clock Drift. The *drift* of a physical clock k between microtick i and microtick $i + 1$ is the frequency ratio between this clock k and the reference clock, at the instant of microtick i . The drift is determined by measuring the duration of a granule of clock k with the reference clock z and dividing it by the nominal number n^k of reference clock microticks in a granule:

$$\text{drift}_i^k = \frac{z(\text{microtick}_{i+1}^k) - z(\text{microtick}_i^k)}{n^k}$$

Because a good clock has a drift that is very close to 1, for notational convenience the notion of a *drift rate* ρ_i^k is introduced as

$$\rho_i^k = \left| \frac{z(\text{microtick}_{i+1}^k) - z(\text{microtick}_i^k)}{n^k} - 1 \right|.$$

A perfect clock will have a drift rate of 0. Real clocks have a varying drift rate that is influenced by environmental conditions, e.g., a change in the ambient temperature, a change in the voltage level that is applied to a crystal oscillator, or aging of the crystal. Within specified environmental parameters, the drift rate of an oscillator is bounded by the *maximum drift rate* $\rho_{\max}^k$, which is documented in the data sheet of the resonator. Typical maximum drift rates $\rho_{\max}^k$ are in the range of 10^{-2} to 10^{-7} s/s, or better, depending on the quality (and price) of the oscillator. Because every clock has a non-zero drift rate, *free-running* clocks, i.e., clocks that are never resynchronized, leave any bounded relative time interval after a finite time, even if they are fully synchronized at startup.

Example: During the Gulf war on February 25, 1991, a Patriot missile defense system failed to intercept an incoming Scud rocket. The clock drift over a 100-h period (which resulted in a tracking error of 678 m) was blamed for the Patriot missing the Scud missile that hit an American military barracks in Dhahran, killing 29 and injuring 97. The original requirement was a 14-h mission. The clock drift during a 14-h mission could be handled [Neu95, p. 34].

Failure Modes of a Clock. A physical digital clock can exhibit two types of failures. The counter could be mutilated by a fault so that the counter value becomes erroneous, or the drift rate of the clock could depart from the specified drift rate (the shaded area of Fig. 3.1) because the clock starts ticking faster (or slower) than specified.

3.1.3 Precision and Accuracy

Offset. The *offset* at microtick i between two clocks j and k with the same granularity is defined as

$$\text{offset}_i^{jk} = |z(\text{microtick}_i^j) - z(\text{microtick}_i^k)|$$

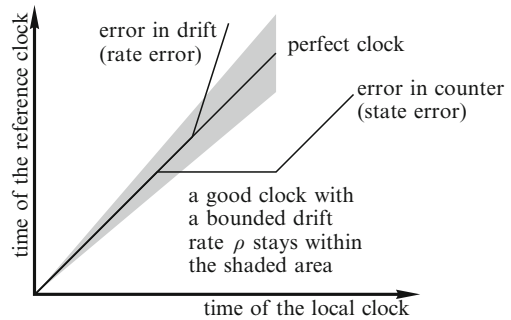


Fig. 3.1 Failure modes of a physical clock

The offset denotes the time difference between the respective microticks of the two clocks, measured in the number of microticks of the reference clock.

Precision. Given an ensemble of n clocks $\{1, 2, \dots, n\}$, the maximum offset between any two clocks of the ensemble

$$\Pi_i = \max_{\forall 1 \leq j, k \leq n} \{offset_i^{jk}\}$$

is called the *precision* Π of the ensemble at microtick i . The maximum of Π_i over an *interval of interest* is called the *precision* Π of the ensemble. The precision denotes the *maximum offset* of respective microticks of any two clocks of the ensemble during a duration of interest. The precision is expressed in the number of microticks of the reference clock.

Because of the drift rate of any physical clock, the clocks of an ensemble will drift apart if they are not resynchronized periodically (i.e., brought closer together). The process of mutual resynchronization of an ensemble of clocks to maintain a bounded precision is called *internal synchronization*.

Accuracy. The offset of clock k with respect to the reference clock z at microtick i is called the *accuracy* y_i^k . The maximum offset over all microticks i that is *of interest* is called the *accuracy* y^k of clock k . The accuracy denotes the maximum offset of a given clock from the external time reference during a duration of interest.

To keep a clock within a bounded interval of the reference clock, it must be periodically resynchronized with an external time reference. This process of resynchronization of a clock with an external time reference is called *external synchronization*.

If all clocks of an ensemble are externally synchronized with an accuracy A , then the ensemble is also internally synchronized with a precision of at most $2A$. The converse is not true. An ensemble of internally synchronized clocks will drift from the external time if the clocks are never resynchronized with the external time base.

3.1.4 Time Standards

In the last decades, a number of different time standards have been proposed to measure the time difference between any two events and to establish the position of an event relative to some commonly agreed origin of a time base, the *epoch*. Two of these time bases are relevant for the designer of a distributed real-time computer system, the *International Atomic Time* (TAI) and the *Universal Time Coordinated* (UTC).

International Atomic Time (TAI – *Temps Atomique Internationale*). The need for a time standard that can be generated in a laboratory gave birth to the International Atomic Time (TAI). TAI defines the second as the duration of 9,192,631,770 periods of the radiation of a specified transition of the cesium

atom 133. The intention was to define the duration of the TAI second so that it agrees with the second derived from astronomical observations. TAI is a *chronoscopic* timescale, i.e., a timescale without any discontinuities (e.g., leap seconds). The epoch of TAI starts on January 1, 1958 at 00:00 h Greenwich Mean Time (GMT). The time base of the global positioning system GPS is based on TAI with the epoch starting on January 6, 1980 at 00:00 h.

Universal Time Coordinated (UTC). UTC is a time standard that has been derived from astronomical observations of the rotation of the earth relative to the sun. It is the basis for the time on the *wall-clock*. However, there is a known offset between the local wall-clock time and UTC determined by the time zone and by the political decisions about when daylight savings time must be used. The UTC time standard was introduced in 1972, replacing the Greenwich Mean Time (GMT) as an international time standard. Because the rotation of the earth is not smooth, but slightly irregular, the duration of the GMT second changes slightly over time. In 1972, it was internationally agreed that the duration of the second should conform to the TAI standard, and that the number of seconds in an hour would have to be modified occasionally by inserting a *leap second* into the UTC to maintain synchrony between the UTC (wall-clock time) and astronomical phenomena, like day and night. Because of this leap second, the UTC is not a chronoscopic timescale, i.e., it is not free of discontinuities. It was agreed that on January 1, 1958 at midnight, both the UTC and the TAI had the same value. Since then the UTC has deviated from TAI by about 30 s. The point in time when a leap second is inserted into the UTC is determined by the Bureau International de l'Heure and publicly announced, so that the current offset between the UTC and the TAI is always known.

Example: In *Software Engineering Notes* of March 1996 [Pet96, p. 16] was the following story:

Ivan Peterson reported on a problem that occurred when a leap second was added at midnight on New Year's Eve 1995. The leap second was added, but the date inadvertently advanced to Jan. 2. Ivars heard from a source at AP radio that the synchronization of their broadcast networks depends on the official time signal, and this glitch affected their operation for several hours until the problem was corrected. You can't even count on the national timekeepers to get it right all the time. Bob Huey responded that making corrections at midnight is obviously risky: (1) The day increments to January 1, 1996, 00:00:00. (2) You reset the clock to 23:59:59, back one second. (3) The clock continues running. (4) The day changes again, and it is suddenly January 2, 1996, 00:00:00. No wonder they had problems.

3.2 Time Measurement

If the real-time clocks of all nodes of a distributed system were perfectly synchronized with the reference clock z , and all events were time-stamped with this reference time, then it would be easy to measure the interval between any two events or to reconstruct the temporal order of events, even if variable

communication delays generated differing delivery orders. In a loosely coupled distributed system where every node has its own local oscillator, such a tight synchronization of clocks is not possible. A weaker notion of a universal time reference, the concept of *global time*, is therefore introduced into a distributed system.

3.2.1 Global Time

Suppose a set of nodes exists, each one with its own local physical clock c^k that ticks with granularity g^k . Assume that all of the clocks are internally synchronized with a precision Π , i.e., for any two clocks j, k , and all microticks i

$$\left| z(\text{microtick}_i^j) - z(\text{microtick}_i^k) \right| < \Pi.$$

(In Sect. 3.4, methods for the internal synchronization of the clocks are presented). It is then possible to select a *subset of the microticks* of each local clock k for the generation of the local implementation of a global notion of time. We call such a selected local microtick i a *macrotick* (or a *tick*) of the global time. For example, every tenth microtick of a local clock k may be interpreted as the global tick, the *macrotick* t_i^k , of this clock (see Fig. 3.2). If it does not matter at which clock k the (macro)tick occurs, we denote the tick t_i without a superscript. A global time is thus an *abstract notion* that is *approximated* by properly selected microticks from the synchronized local physical clocks of an ensemble.

Reasonableness Condition. The global time t is called *reasonable*, if all local implementations of the global time satisfy the condition

$$g > \Pi$$

the *reasonableness condition* for the global granularity g . This reasonableness condition ensures that the synchronization error is *bounded* to less than one *macrogranule*, i.e., the duration between two (macro) ticks. If this reasonableness condition is satisfied, then for a single event e , that is observed by any two different clocks of the ensemble,

$$\left| t^j(e) - t^k(e) \right| \leq 1,$$

i.e., the global time-stamps for a single event can differ by at most one tick. *This is the best we can achieve.* Because of the impossibility of synchronizing the clocks

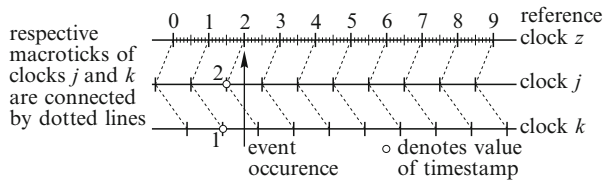
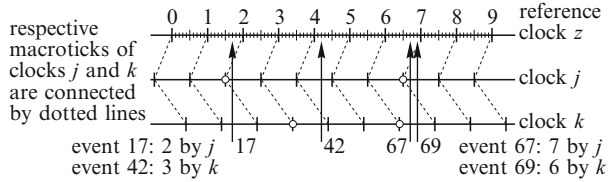


Fig. 3.2 Time-stamps of a single event

Fig. 3.3 Temporal order of two events with a difference of one tick



perfectly, and the granularity of any digital time there is always the possibility of the following sequence of events: clock j ticks, event e occurs, clock k ticks. In such a situation, the single event e is time-stamped by the two clocks j and k with a difference of one tick (Fig. 3.2).

One Tick Difference – What Does It Mean? What can we learn about the temporal order of two events, observed by different nodes of a distributed system with a reasonable global time, given that the global time-stamps of these two events differ by one tick?

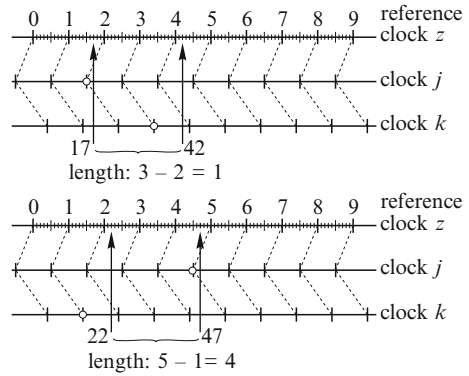
In Fig. 3.3, four events are depicted, *event 17*, *event 42*, *event 67* and *event 69* (time-stamps from the reference clock). Although the duration between *event 17* and *event 42* is 25 microticks, and the duration between *event 67* and *event 69* is only 2 microticks, both durations lead to the *same* measured difference of one macrogranule. The global time-stamp for *event 69* is *smaller* than the global time-stamp for *event 67*, although *event 69* occurred *after* *event 67*. Because of the accumulation of the synchronization error and the digitalization error, it is not possible to reconstruct the temporal order of two events from the knowledge that the global time-stamps differ by one tick. However, if the time-stamps of two events differ by two ticks, then the temporal order can be reconstructed because the sum of the synchronization and digitalization error is always less than two granules in a clocking system with a reasonable global time-base.

This fundamental limitation in time measurement limits the *faithfulness* of the digital computer model of a controlled physical subsystem. The time-base in the physical part of a cyber-physical system is dense, while the time base in the computer system is discrete. Whenever two events in the physical subsystem occur close together, compared to the granularity of the global time, it is not possible to reconstruct the physical temporal order of the events in the computer system faithfully. The only way out of this dilemma is the provision of a global time base with a smaller granularity, such that temporal errors are reduced [Kop09].

3.2.2 Interval Measurement

An interval is delimited by two events, the start event of the interval and the terminating event of the interval. The measurement of these two events relative to each other can be affected by the synchronization error and the digitalization error. The sum of these two errors is *less than* $2g$ because of the reasonableness

Fig. 3.4 Errors in interval measurement



condition, where g is the granularity of the global time. It follows that the true duration d_{true} of an interval is bounded by

$$(d_{obs} - 2g) < d_{true} < (d_{obs} + 2g)$$

where d_{obs} is the observed difference between the start event and the terminating event of the interval. Figure 3.4 depicts how the observed duration of an interval of length 25 microticks can differ, depending on which node observes the start event and the terminating event. The global tick, assigned by an observing node to an event delimiting the interval is marked by a small circle in Fig. 3.4.

3.2.3 π/Δ -Precedence

Consider a distributed computer system that consists of three nodes j , k , and m that support a global time. Every node is to generate an event at its view of the global instants 1, 5, and 9. An omniscient outside observer will see the scenario depicted in Fig. 3.5.

All events that are generated locally at the same global clock tick will occur within a small interval π , where $\pi \leq \Pi$, the precision of the ensemble (because of the reasonableness condition). Events that occur at different ticks will be at least Δ apart (Fig. 3.5). The outside observer should not order the events that occur within π , because these events are *supposed* to occur at the same instant. Events that occur at different ticks should be ordered. How many granules of silence must exist between the event subsets such that an outside observer or another cluster will always recover the temporal order intended by the sending cluster? Before we can answer this question (in Sect. 3.3.2) we must introduce the notion of π/Δ precedence.

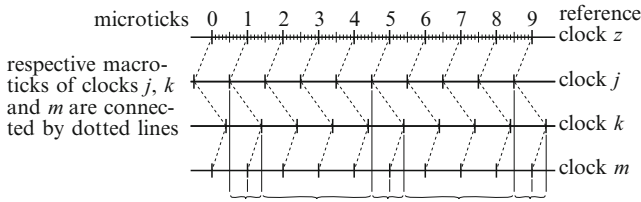


Fig. 3.5 π/Δ precedence

Given a set of events $\{E\}$ and two durations π and Δ where $\pi \ll \Delta$, such that for any two elements e_i and e_j of this set, the following condition holds:

$$[|z(e_i) - z(e_j)| \leq \pi] \vee [|z(e_i) - z(e_j)| > \Delta]$$

where z is the reference clock. Such an event set is called π/Δ -precedent. π/Δ -precedence means that a subset of the events that happen at about the same time (and that are therefore close together within π) is separated by a substantial interval (at least Δ) from the elements in another subset. If π is zero, then any two events of the $0/\Delta$ -precedent event set occur either at the same instant or are at least a duration Δ apart.

Assume a distributed system with a reasonable global time base with granularity g and two events, e_1 and e_2 , that are produced at the same locally generated global tick of two different nodes. Due to the synchronization error, these events can differ by up to but less than one granule. These events are observed by some of the other nodes.

Because of the synchronization and digitalization error, the two (simultaneous by intention) events can be time-stamped by the observers with two ticks difference. In order to be able to establish the intended temporal order of events from their time-stamps a sufficient duration of silence is needed before the next event may occur in order to ensure that the intended simultaneity of the events can always be recovered by all observers [Ver94].

3.2.4 Fundamental Limits of Time Measurement

The above analysis leads to the following four fundamental limits of time measurement in distributed real-time systems with a reasonable global time base with granularity g .

1. If a single event is observed by two different nodes, there is always the possibility that the time-stamps differ by one tick. A one-tick difference in the time-stamps of two events is not sufficient to reestablish the temporal order of the events from their time-stamps.

2. If the observed duration of an interval is d_{obs} , then the true duration d_{true} is bounded by

$$(d_{obs} - 2g) < d_{true} < (d_{obs} + 2g)$$

3. The temporal order of events can be recovered from their time-stamps if the difference between their time-stamps is equal to or greater than 2 ticks.
4. The temporal order of events can *always* be recovered from their time-stamps, if the event set is at least $0/3g$ precedent.

These fundamental limits of time measurement are also the *fundamental limits to the faithfulness* of a digital model of a physical system.

3.3 Dense Time Versus Sparse Time

Example: It is known a priori that a particular train will arrive at a train station every hour. If the train is always on time and all clocks are synchronized, it is possible to uniquely identify each train by its time of arrival. Even if the train is slightly off, say, by 5 min, and the clocks are slightly out of synchronization, say, by 1 min, there will be no problem in uniquely identifying a train by its time of arrival. What are the limits within which a train can still be uniquely identified by its time of arrival?

Assume a set $\{E\}$ of events that are of interest in a particular context. This set $\{E\}$ could be the ticks of all clocks, or the events of sending and receiving messages. If these events are allowed to occur at any instant of the timeline, then, we call the time base *dense*. If the occurrence of these events is restricted to some *active intervals* of duration ε , with an interval of silence of duration Δ between any two active intervals, then we call the time base ε/Δ -*sparse*, or simply *sparse* for short (Fig. 3.6). If a system is based on a sparse time base, there are time intervals during which no significant event is allowed to occur. Events that occur only in the active intervals are called *sparse events*.

It is evident that the occurrences of events can only be restricted if the given system has the authority to control these events, i.e., these events are in the sphere of control of the computer system [Dav79]. The occurrence of events outside the sphere of control of the computer system cannot be restricted. These external events are based on a dense time base and cannot be forced to be *sparse events*.

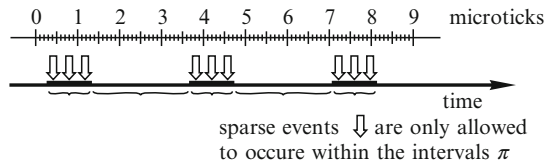


Fig. 3.6 Sparse time-base

Example: Within a distributed computing system, the sending of messages can be restricted to some intervals of the timeline and can be forbidden at some other intervals – they can be designed to be *sparse events*.

3.3.1 Dense Time-Base

Suppose that we are given two events $e1$ and $e2$ that occur on a dense time base. If these two events are closer together than $3g$, where g is the granularity of the global time, then, it is not always possible to establish the temporal order, or even a consistent order of these two events on the basis of the time-stamps generated by the different nodes if no *agreement protocol* (see below) is applied.

Example: Consider the scenario of Fig. 3.7 with two events, $e1$ and $e2$ which are 2.5 granules apart. Event $e1$ is observed by node j at time 2 and by node m at time 1, while $e2$ is only observed by node k that reports its observation “ $e2$ occurred at 3” to node j and node m . Node j calculates a time-stamp difference of one tick and concludes that the events occurred at about the same time and *cannot* be ordered. Node m calculates a time-stamp difference of 2 ticks and concludes that $e1$ has *definitely* occurred before $e2$. The two nodes j and m have an *inconsistent* view about the order of event occurrence.

Agreement Protocol. To arrive at a *consistent view* of the order of *non-sparse events* within a distributed computer system (which does not necessarily reflect the temporal order of event occurrence), the nodes must execute an *agreement protocol*. The first phase of an agreement protocol requires an information interchange among the nodes of the distributed system with the goal that every node acquires the differing local views about the state of the world from every other node. At the end of this first phase, every correct node possesses exactly the same information as every other node. In the second phase of the agreement protocol, each node applies a deterministic algorithm to this consistent information to reach the same conclusion about the assignment of the event to an active interval of the sparse time base—the commonly agreed value. In the fault-free case, an agreement algorithm requires an additional round of information exchange as well as the resources for executing the agreement algorithm.

Agreement algorithms are costly, both in terms of communication requirements, processing requirements, and – worst of all – in terms of the additional delay they introduce into a control loop. It is therefore expedient to look for solutions to the consistent temporal ordering problem in distributed computer systems that do not

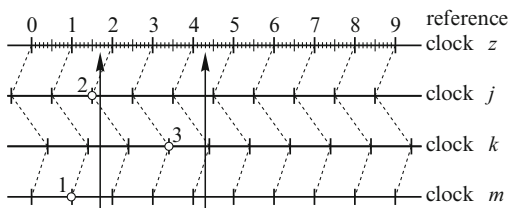


Fig. 3.7 Different observed order of two events $e1$ and $e2$

require these additional overheads. The sparse time model, introduced below, provides for such a solution.

3.3.2 *Sparse Time-Base*

Consider a distributed system that consists of two clusters: cluster A generates events, and cluster B observes these generated events. Each one of the clusters has its own cluster-wide synchronized time with a granularity g , but these two cluster-wide time bases are not synchronized with each other. Under what circumstances is it possible for the nodes in the observing cluster to reestablish consistently the *intended temporal order* of the generated events without the need to execute an agreement protocol?

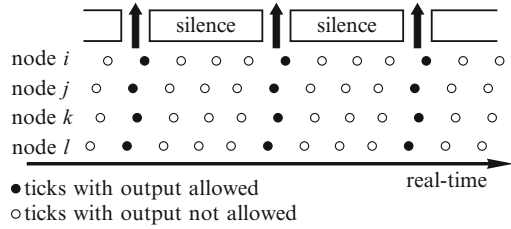
If two nodes, nodes j and k of cluster A , generate two events at the same cluster-wide tick t_i , i.e., at tick t_i^j and at tick t_i^k , then these two events can be, at most, a distance Π apart from each other, where $g > \Pi$, the granularity of the cluster-wide time. Because there is no intended temporal order among the events that are generated at the same cluster-wide tick of cluster A , the observing cluster B should *never* establish a temporal order among the events that have occurred at about the same time. On the other hand, the observing cluster B should *always* reestablish the temporal order of the events that have been occurred at different cluster-wide ticks. Is it sufficient if cluster A generates a $1g/3g$ precedent event set, i.e., after every cluster-wide tick at which events are allowed to be generated there will be silence for at least three granules?

If cluster A generates a $1/3g$ precedent event set, then it is possible that two events that are generated at the same cluster-wide granule at cluster A will be time-stamped by cluster B with time-stamps that differ by 2 ticks. The observing cluster B should not order these events (although it could), because they have been generated at the same cluster-wide granule. Events that are generated by cluster A at different cluster-wide granules ($3g$ apart) and therefore should be ordered by cluster B , could also obtain time-stamps that differ by 2 ticks. Cluster B cannot decide whether or not to order events with a time-stamp difference of 2 ticks. To resolve this situation, cluster A must generate a $1/4g$ precedent event set. Cluster B will not order two events if their time-stamps differ by ≤ 2 ticks, but will order two events if their time-stamps differ by ≥ 3 ticks, thus reestablishing the temporal order that has been intended by the sender.

3.3.3 *Space-Time Lattice*

The ticks of the global clock can be seen as generating a space-time lattice, as depicted in Fig. 3.8. A node is allowed to generate an event (e.g., send a message) at the filled dots and must be silent at the empty dots. This rule makes it possible for

Fig. 3.8 Sparse time base



the receiver to establish a consistent temporal order of events without executing an agreement protocol. Although a sender is allowed to generate an event only at the filled dots, this is still much faster than executing an agreement protocol, provided a global time base of sufficient precision is available. Events that are generated at the filled dots of the sparse time lattice are called *sparse events*.

Events that occur outside the sphere of control of the computer system cannot be confined to a sparse time base: they happen on a dense time base and are therefore not *sparse events*. To generate a consistent view of events that occur in the controlled object, and that are observed by more than one node of the distributed computer system, the execution of an agreement protocol is unavoidable at the interface between the computer system and the controlled object or other systems that do not participate in the global time. Such an agreement protocol transforms a *non-sparse event* into a *sparse event*.

3.3.4 Cyclic Representation of Time

Many processes in the technical and biological world are cyclic [Win01]. A cyclic process is characterized by a regular behavior, where a similar set of action patterns is repeated in every cycle.

Example: In a typical control system, real-time is partitioned into a sequence of control cycles (Fig. 3.9). Every control cycle starts with reading the state variables of the controlled object, proceeds with the execution of the control algorithm, and finishes with the output of new set-points to the actuators at the interface between the computer system and the controlled object.

In the cyclic representation of time, the linear time is partitioned into cycles of equal duration. Every cycle is represented by a circle, where an instant within a cycle is denoted by the *phase*, i.e., the angular deviation of the instant from the beginning of the cycle. *Cycle* and *phase* thus denote an instant in a cyclic representation. In the cyclic representation of sparse time, the circumference of the circle is not a *dense* line, but a *dotted* line, where the size and the distance between dots is determined by the precision of the clock synchronization.

A sequence of consecutive processing and communication actions, such as the actions in Fig. 3.9, are *phase-aligned*, if the termination of one action is immediately followed by the start of the next consecutive action. If the actions

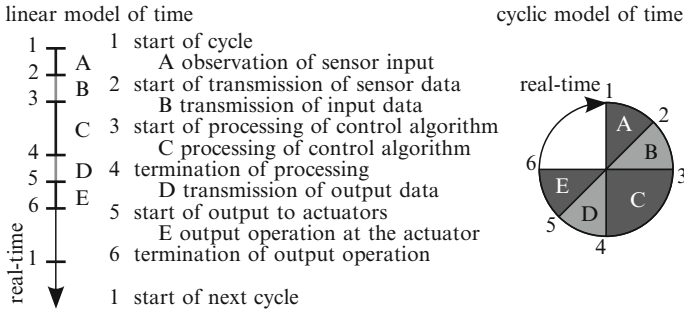


Fig. 3.9 Linear versus cyclic representation of time in a control system

within a *RT-transaction* (see Sect. 1.7.3) are phase-aligned, then the overall duration of the RT transactions is minimized.

If we look at Fig. 3.9, we see that communication services in a typical control loop are periodically required only in the intervals *B* and *D* of a cycle. The *shorter* these intervals *B* and *D*, the *better*, since the dead time of the control loop is reduced. This requirement leads to the model of *pulsed data streams*, where, in a time-triggered system, the highest possible bandwidth is allocated periodically in the intervals *B* and *D*, while, during the rest of the cycle, the communication bandwidth can be allocated to other requests [Kop06].

An extension of the cyclic representation is the *spiral representation of time*, where a third axis is introduced to depict the linear progression of the cycles.

3.4 Internal Clock Synchronization

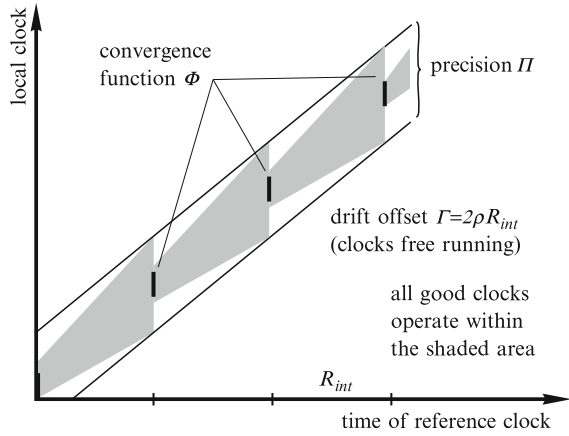
The purpose of internal clock synchronization is to ensure that the global ticks of all correct nodes occur within the specified *precision* Π , despite the varying drift rate of the local real-time clock of each node. Because the availability of a proper global time base is crucial for the operation of a distributed real-time system, the clock synchronization should not depend on the correctness of a single clock, i.e., it should be fault-tolerant.

Every node of a distributed system has a local oscillator that (micro)ticks with a frequency determined by the physical parameters of the oscillator. A subset of the local oscillator's microticks called the ticks (or macroticks – see Sect. 3.2.1) is interpreted as the global time ticks at the node. These global time ticks increment the local node's global time counter.

3.4.1 The Synchronization Condition

The global time ticks of each node must be periodically resynchronized within the ensemble of nodes to establish a global time base with specified precision.

Fig. 3.10 Synchronization condition



The period of resynchronization is called the *resynchronization interval* R_{int} . At the end of each resynchronization interval, the clocks are adjusted to bring them into better agreement with each other. The *convergence function* Φ denotes the offset of the time values immediately after the resynchronization. Then, the clocks drift again apart until they are resynchronized at the end of the next resynchronization interval R_{int} (Fig. 3.10). The *drift offset* Γ indicates the maximum accumulated divergence of any two good clocks from each other during the resynchronization interval R_{int} , where the clocks are free running. The drift offset Γ depends on the length of the resynchronization interval R_{int} and the maximum specified drift rate ρ of the clock:

$$\Gamma = 2\rho R_{\text{int}}$$

An ensemble of clocks can only be synchronized if the following *synchronization condition* holds between the *convergence function* Φ , the *drift offset* Γ and the *precision* Π :

$$\Phi + \Gamma \leq \Pi$$

Assume that at the end of the resynchronization interval, the clocks have diverged so that they are at the edge of the precision interval Π (Fig. 3.10). The synchronization condition states that the synchronization algorithm must bring the clocks so close together that the amount of divergence during the next free-running resynchronization interval will not cause a clock to leave the precision interval.

Byzantine Error. The following example explains how, in an ensemble of three nodes, a malicious node can prevent the other two nodes from synchronizing their clocks since they cannot satisfy the synchronization condition [Lam82]. Assume an ensemble of three nodes, and a convergence function where each of the three nodes sets its clock to the average value of the ensemble. Clocks *A* and *B* are good, while clock *C* is a malicious *two-faced* clock that disturbs the other two good clocks in

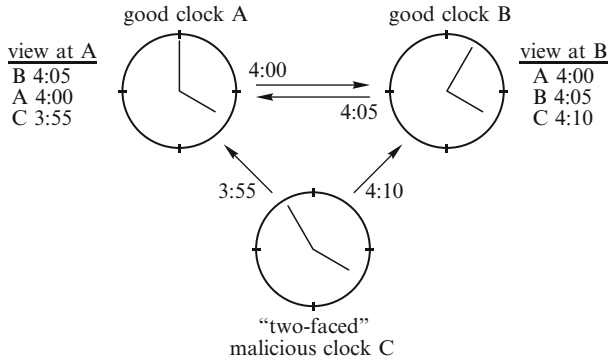


Fig. 3.11 Behavior of a malicious clock

such a manner that neither of them will ever correct their time value (Fig. 3.11), and will thus eventually violate the synchronization condition.

Such a malicious, *two-faced* manifestation of behavior is sometimes called a *malicious error* or a *Byzantine error* (see also Sect. 6.1.3). During the exchange of the synchronization messages, a Byzantine error can lead to inconsistent views of the state of the clocks among the ensemble of nodes. A special class of algorithms, the *interactive-consistency algorithms* [Pea80], inserts additional rounds of information exchanges to agree on a consistent view of the time values at all nodes. These additional rounds of information exchanges increase the quality of the precision at the expense of additional communication overhead. Other algorithms work with inconsistent information, and establish bounds for the maximum error introduced by the inconsistency. An example of such an algorithm is the Fault-Tolerant-Average algorithm, described later in this section. It has been shown [Lam85] that clock synchronization can only be guaranteed in the presence of Byzantine errors if the total number of clocks $N \geq (3k + 1)$, where k is the number of Byzantine faulty clocks.

3.4.2 Central Master Synchronization

This is a simple non-fault tolerant synchronization algorithm. A unique node, the central master, periodically sends the value of its time counter in synchronization messages to all other nodes, the slave nodes. As soon as a slave node receives a new synchronization message from the master, the slave records the time-stamp of message arrival. The difference between the master's time, contained in the synchronization message, and the recorded slave's time-stamp of message arrival, corrected by the known latency of the message transport, is a measure of the deviation of the clock of the master from the clock of the slave. The slave then corrects its clock by this deviation to bring it into agreement with the master's clock.

The convergence function Φ of the central master algorithm is determined by the difference between the fastest and slowest message transmission to the slave nodes of the ensemble, i.e., the *latency jitter* ε between the event of writing the synchronization time value by the master and the events of message arrival time-stamping at all slaves.

Applying the *synchronization condition*, the precision of the central master algorithm is given by:

$$\Pi_{central} = \varepsilon + \Gamma$$

The central master synchronization is often used in the startup phase of a distributed system. It is simple, but not fault tolerant, since a failure of the master ends the resynchronization, causing the free-running clocks of the slaves to leave the precision interval soon thereafter. In a variant of this algorithm, a multi-master strategy is followed: if the active master fails silently and the failure is detected by a local time-out at a *shadow master*, one of the *shadow masters* assumes the role of the master and continues the resynchronization.

3.4.3 Fault-Tolerant Synchronization Algorithms

Typically, distributed fault-tolerant clock resynchronization proceeds in three distinct phases. In the first phase, every node acquires knowledge about the state of the global time counters in all the other nodes by the exchange of messages among the nodes. In the second phase, every node analyzes the collected information to detect errors, and executes the convergence function to calculate a correction value for the local global time counter. A node must deactivate itself if the correction term calculated by the convergence function is larger than the specified precision of the ensemble. Finally, in the third phase, the local time counter of the node is adjusted by the calculated correction value. Existing algorithms differ in the way in which the time values are collected from the other nodes, in the type of convergence function used, and in the way in which the correction value is applied to the local time counter.

Reading the Global Time. In a local-area network, the most important term affecting the precision of the synchronization is the jitter of the time messages that carry the current time values from one node to all the other nodes. The known minimal delay for the transport of a time message between two nodes can be compensated by an a priori known delay-compensation term [Kop87] that compensates for the delay of the message in the transmission channel and in the interface circuitry. The delay jitter depends more than anything else on the system level, at which the synchronization message is assembled and interpreted. If this is done at a high level of the architecture, e.g., in the application software, all random delays caused by the scheduler, the operating system, the queues in the protocol software, the message

Table 3.1 Approximate jitter of the synchronization message

Synchronization message assembled and interpreted	Approximate range of jitter
At the application software level	500 μ s–5 ms
In the kernel of the operating system	10–100 μ s
In the hardware of the communication controller	Less than 1 μ s

retransmission strategy, the media-access delay, the interrupt delay at the receiver, and the scheduling delay at the receiver, accumulate and degrade the quality of the time values, thus deteriorating the precision of the clock synchronization. Table 3.1 gives approximate value ranges for the jitter that can be expected at the different levels [Kop87]:

Since a small jitter is important to achieve high precision in the global time, a number of special methods for jitter reduction have been proposed. Christian [Cri89] proposed the reduction of the jitter at the application software level using a probabilistic technique: a node queries the state of the clock at another node by a query-reply transaction, the duration of which is measured by the sender. The received time value is corrected by the synchronization message delay that is assumed to be half the round-trip delay of the query-reply transaction (assuming that the delay distribution is the same in both directions). A different approach is taken in the Time-Triggered Architecture. A special clock synchronization unit has been implemented to support the segmentation and assembly of synchronization messages at the hardware level, thereby reducing the jitter to a few microseconds. The new IEEE 1588 standard for clock synchronization limits the jitter by hardware assisted time stamping [Eid06].

Impossibility Result. The important role of the latency jitter ε for internal synchronization is emphasized by an impossibility result by Lundelius and Lynch [Lun84]. According to this result, it is not possible to internally synchronize the clocks of an ensemble consisting of N nodes to a better precision than

$$\Pi = \varepsilon \left(1 - \frac{1}{N} \right)$$

(measured in the same units as ε) even if it is assumed that all clocks have perfect oscillators, i.e., the drift rates of all the local clocks are zero.

The Convergence Function. The construction of a convergence function is demonstrated by the example of the distributed Fault-Tolerant-Average (FTA) algorithm in a system with N nodes where k Byzantine faults should be tolerated. The FTA algorithm is a one-round algorithm that works with inconsistent information and bounds the error introduced by the inconsistency. At every node, the N measured time differences between the node's clock and the clocks of all other nodes are collected (the node considers itself a member of the ensemble with time difference zero). These time differences are sorted by size. Then the k largest and the k smallest time differences are removed (assuming that an erroneous time value is

either larger or smaller than the rest). The remaining $N - 2k$ time differences are, by definition within the precision window definition (since only k values are assumed to be erroneous and an erroneous value is larger or smaller than a good value). The average of these remaining $N - 2k$ time differences is the correction term for the node's clock.

Example: Figure 3.12 shows an ensemble of seven nodes and one tolerated Byzantine fault. The FTA takes the average of the five accepted time values shown.

The worst-case scenario occurs if all good clocks are at opposite ends of the precision window Π , and the Byzantine clock is seen at different corners by two nodes. In the example of Fig. 3.13, node j will calculate an average value of $4\Pi/5$ and node k will calculate an average value of $3\Pi/5$; the difference between these two terms, caused by the Byzantine fault, is thus $\Pi/5$.

Precision of the FTA. Assume a distributed system with N nodes, each one with its own clock (all time values are measured in seconds). At most k out of the N clocks behave in a Byzantine manner.

A single Byzantine clock will cause the following difference in the calculated averages at two different nodes in an ensemble of N clocks:

$$E_{byz} = \Pi / (N - 2k).$$

In the worst case a total of k Byzantine errors will thus cause an error term of

$$E_{k-byz} = k\Pi / (N - 2k).$$

Considering the jitter of the synchronization messages, the convergence function of the FTA algorithm is given by

$$\Phi(N, k, \varepsilon) = (k\Pi / (N - 2k)) + \varepsilon.$$

Fig. 3.12 Accepted and rejected time values

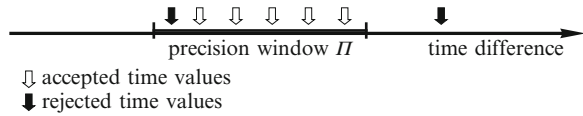


Fig. 3.13 Worst possible behavior of a malicious (Byzantine) clock

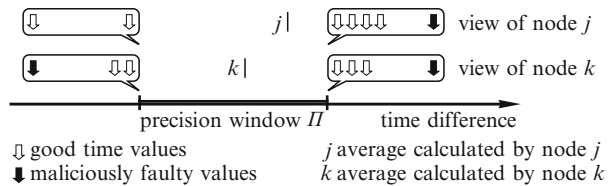


Table 3.2 Byzantine error term $\mu(N, k)$

Faults	Number of nodes in the ensemble							
	4	5	6	7	10	15	20	30
1	2	1.5	1.33	1.25	1.14	1.08	1.06	1.03
2				3	1.5	1.22	1.14	1.08
3					4	1.5	1.27	1.22

Combining the above equation with the synchronization condition (Sect. 3.4.1) and performing a simple algebraic transformation, we get the precision of the FTA algorithm:

$$\Pi(N, k, \varepsilon, \Gamma) = (\varepsilon + \Gamma) \frac{N - 2k}{N - 3k} = (\varepsilon + \Gamma) \mu(N, k).$$

where $\mu(N, k)$ is called the *Byzantine error term* and is tabulated in Table 3.2.

The Byzantine error term $\mu(N, k)$ indicates the loss of quality in the precision due to the inconsistency arising from the Byzantine errors. In a real environment, at most one Byzantine error is expected to occur in a synchronization round (and even this will happen very, very infrequently), and thus, the consequence of a Byzantine error in a properly designed synchronization system is not serious.

The drift-offset Γ is determined by the quality of the selected oscillator and the length of the resynchronization interval. If a standard quartz oscillator with a nominal drift rate of 10^{-4} s/s is used, and the clocks are resynchronized every second, then Γ is about 100 μ s. Because the stochastic drift rate of a crystal is normally two orders of magnitude smaller than the nominal drift rate that is determined by the systematic error of the quartz oscillator, it is possible to reduce the drift offset Γ by up to two orders of magnitude by performing systematic error compensation.

Many other convergence functions for the internal synchronization of the clocks have been proposed and analyzed in the literature [Sch88].

3.4.4 State Correction Versus Rate Correction

The correction term calculated by the convergence function can be applied to the local-time value immediately (*state correction*), or the rate of the clock can be modified so that the clock speeds up or slows down during the next resynchronization interval to bring the clock into better agreement with the rest of the ensemble (*rate correction*).

State correction is simple to apply, but it has the disadvantage of generating a discontinuity in the time base. If clocks are set backwards and the same nominal-time value is reached twice, then, pernicious failures can occur within the real-time software (see the example in Sect. 3.1.4). It is therefore advisable to implement rate

correction with a bound on the maximum value of the clock drift so that the error in interval measurements is limited. The resulting global time base then maintains the chronoscopy property despite the resynchronization. Rate correction can be implemented either in the digital domain by changing the number of microticks in some of the (macro)ticks, or in the analog domain by adjusting the voltage of the crystal oscillator. To avoid a common-mode drift of the complete ensemble of clocks, the average of the rate correction terms among all clocks in the ensemble should be close to zero.

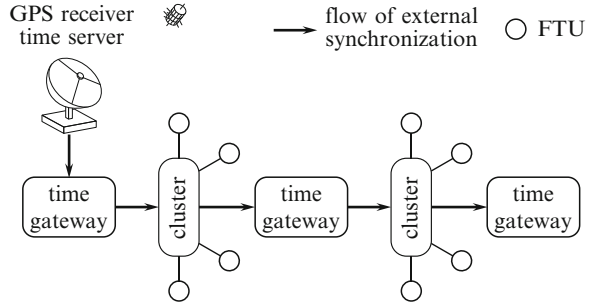
3.5 External Clock Synchronization

External synchronization links the global time of a cluster to an external standard of time. For this purpose it is necessary to access a *timeserver*, i.e., an external time source that periodically broadcasts the current reference time in the form of a *time message*. This time message must raise a synchronization event (such as the beep of a wrist watch) in a designated node of the cluster and must identify this synchronization event on the agreed time scale. Such a time scale must be based on a widely accepted measure of time, e.g., the physical second, and must relate the synchronization event to a defined origin of time, the *epoch*. The interface node to a timeserver is called a *time gateway*. In a fault-tolerant system, the time-gateway should be a fault-tolerant unit (FTU – see Sect. 6.4.2).

3.5.1 External Time Sources

Assume that the time gateway is connected to a GPS (Global Positioning System). The accuracy of a GPS receiver is better than 100 ns and it has an authoritative long-term stability – in some sense, GPS is the worldwide measurement standard for measuring the progression of time. Alternatively, the external time source can be a temperature compensated crystal oscillator (TCXO) with a drift rate of better than 1 ppm, causing a *drift offset* of better 1 $\mu\text{s/s}$ or an atomic clock, e.g., a Rubidium clock with a drift rate in the order of 10^{-12} , causing a *drift offset* of about 1 μs in 10 days (more expensive atomic clocks are even better). The time gateway periodically broadcasts time messages containing a synchronization event, as well as the information to place this synchronization event on the TAI scale. The time gateway must synchronize the global time of its cluster with the time received from the external time source. This synchronization is unidirectional, and therefore asymmetric, as shown in Fig. 3.14. It can be used to adjust the rate of the clocks without any concern for the occurrence of emergent instability effects.

If another cluster is connected to this *primary* cluster by a secondary time gateway, then, the unidirectional synchronization functions in the same manner. The secondary time gateway considers the synchronized time of the primary cluster as its time reference, and synchronizes the global time of the secondary cluster.

Fig. 3.14 Flow of external synchronization

While internal synchronization is a cooperative activity among all the members of a cluster, external synchronization is an authoritarian process: the time gateway forces its view of external time on all its subordinates. From the point of view of fault tolerance, such an authoritarian regime introduces a problem: if the authority sends an incorrect message, then all its *obedient* subordinates will behave incorrectly. However, for external clock synchronization, the situation is under control because of the *inertia* of time. Once a cluster has been synchronized, the fault-tolerant global time base within a cluster acts as a *monitor* of the time gateway. An external synchronization message will only be accepted if its content is sufficiently close to the cluster's view of the external time. The time gateway has only a limited authority to correct the drift rate of a cluster. The enforcement of a *maximum common-mode correction rate* is required to keep the error in relative time-measurements small. The software in each node of the cluster checks the maximum correction rate.

The implementation must guarantee that it is impossible for a faulty external synchronization to interfere with the proper operation of the internal synchronization, i.e., with the generation of global time within a cluster. The worst possible failure scenario occurs if the external timeserver fails maliciously – a very low probability failure mode if the external timeserver is GPS. This leads to a common-mode deviation of the global time from the external time base with the *maximum permitted deviation rate*. In a properly designed synchronization system, this drift from the external time base will not affect the internal synchronization within a cluster.

3.5.2 Time Gateway

The time gateway must control the timing system of its cluster in the following ways:

1. It must initialize the cluster with the current external time.
2. It must periodically adjust the rate of the global time in the cluster to bring it into agreement with the external time and the standard of time measurement, the second.

3. It must periodically send the current external time in a time message to the nodes in the cluster so that a reintegrating node can reinitialize its external time value.

The time gateway achieves this task by periodically sending a time message with a rate-correction byte. This rate-correction byte is calculated in the time gateway’s software. First, the difference between the occurrence of a significant event, e.g., the exact start of the full second in the timeserver, and the occurrence of the related significant event in the global time of the cluster, is measured by using the local time base (microticks) of the gateway node. Then, the necessary rate adjustment is calculated, bearing in mind the fact that the rate adjustment is bounded by the agreed maximum rate correction. This bound on the rate correction is necessary to keep the maximum deviation of relative time measurements in the cluster below an agreed threshold, and to protect the cluster from faults of the server.

3.5.3 Time Formats

Over the last few years, a number of external-time formats have been proposed for external clock synchronization. The most important one is the standard for the time format proposed in the Network Time Protocol (NTP) of the Internet [Mil91]. This time format (Fig. 3.15) with a length of 8 B contains two fields: a 4 B full seconds field, where the seconds are represented according to UTC, and a fraction of a second field, where the fraction of a second is represented as a binary fraction with a resolution of about 232 ps. On January 1, 1972, at midnight the NTP clock was set to 2,272,060,800.0 s, i.e., the number of seconds since January 1, 1900 at 00:00 h.

The NTP time is not chronoscopic because it is based on UTC which has to accommodate for the switching second. The occasional insertion of a leap second into UTC can disrupt the continuous operation of a time-triggered real-time system.

Another time format is the IEEE 1588 standard time format [Eid06]. In this time format the epoch starts on January 1, 1970 at 00:00 h or is user defined. The full seconds are counted according to TAI, while the unit of the fraction of a second is the nanosecond. This leads to abrupt change in the representation whenever a full second is reached.

The Time-Triggered Architecture (TTA) uses a time format that is a combination of IEEE 1588 and NTP. The full seconds are counted as in TAI (such as IEEE 1588), but parts of a second are represented in a binary fraction of the full second (such as NTP). It is thus chronoscopic and conforms fully to the dual system.

Fig. 3.15 Time format in the Network Time Protocol (NTP)

full seconds UTC, 4 bytes

binary fraction of second, 4 bytes

range up to the year 2036. i.e., 136 years wrap-around cycle

Points to Remember

- An event happens at an *instant*, i.e., at a point of the timeline. A *duration* is a section of the timeline delimited by two instants.
- A consistent delivery order of a set of events in a distributed system does not necessarily reflect the temporal or causal order of the events.
- A *physical clock* is a device for time measurement that contains a counter and a physical oscillation mechanism that periodically generates an event to increase the counter.
- Typical maximum drift rates ρ of physical clocks are in the range from 10^{-2} to 10^{-7} s/s, or lower, depending on the quality (and price) of the resonator.
- The *precision* denotes the maximum offset of respective ticks of any two clocks of an ensemble during the time interval of interest.
- The *accuracy* of a clock denotes the maximum offset of a given clock from the external time reference during the time interval of interest.
- TAI is a *chronoscopic* timescale, i.e., a timescale without any discontinuities, that is derived from the frequency of the radiation of a specified transition of the cesium atom 133.
- UTC is a non-chronoscopic timescale that is derived from astronomical observations of the rotation of the earth in relation to the sun.
- A *global time* is an abstract notion that is approximated by properly selected microticks from the synchronized local physical clocks of an ensemble.
- The *reasonableness condition* ensures that the synchronization error is always less than one granule of the global time.
- If the difference between the time-stamps of two events is *equal to or larger than* 2 ticks, then that temporal order of events can be recovered, provided the global time is reasonable.
- The temporal order of events can *always* be recovered from their time-stamps if the event set is at least 0/3g precedent.
- If events happen only at properly selected points of a *sparse time base*, then it is possible to recover the temporal order of the events without the execution of an agreement protocol.
- The *convergence function* Φ denotes the offset of the time values immediately after the resynchronization.
- The *drift offset* Γ indicates the maximum divergence of any two good clocks from each other during the resynchronization interval R_{int} , in which the clocks are free running.
- The synchronization condition states that the synchronization algorithm must bring the clocks so close together that the amount of divergence during the next free-running resynchronization interval will not cause a clock to leave the precision interval.
- Clock synchronization is only possible if the total number of clocks N is larger or equal to $(3k + 1)$, if k is the number of clocks behaving *maliciously* faulty.

- The most important term affecting the precision of the synchronization is the *latency jitter* of the synchronization messages that carry the current time values from one node to all other nodes of an ensemble.
- When applying the fault-tolerant average algorithm, the *Byzantine error factor* $\mu(N, k)$ denotes the loss of quality in the precision caused by the Byzantine errors.
- *State correction* of a clock has the disadvantage of generating a discontinuity in the time base.
- While internal synchronization is a cooperative activity among all members of a cluster, external synchronization is an authoritarian process: the timeserver forces its view of external time on all its subordinates.
- The *NTP time*, based on UTC, is not chronoscopic. The occasional insertion of a leap second can disrupt the continuous operation of a time-triggered real-time system.
- The *time gateway* maintains the external synchronization by periodically sending a time message with a rate correction byte to all the nodes of a cluster.

Bibliographic Notes

The problem of generating a global time base in a distributed system has first been analyzed in the context of the SIFT [Wen78] and FTMP [Hop78] projects. A VLSI chip for clock synchronization in distributed systems was developed by Kopetz and Ochsenreiter [Kop87]. The Network Time Protocol of the Internet was published in 1991 by Mills [Mil91]. Kopetz presented the concept of a sparse time model first in [Kop92]. The excellent book by Eidson [Eid06] covers the IEEE 1588 protocol for clock synchronization in detail. The integration of internal and external clock synchronization is discussed in Kopetz et al. [Kop04]. For a more philosophical treatment of the problem of time, the reader is advised to study the excellent book by Withrow in [Wit90] entitled *The Natural Philosophy of Time*.

Review Questions and Problems

- 3.1 What is the difference between an *instant* and an *event*?
- 3.2 What is the difference between *temporal* order, *causal* order, and a consistent *delivery* order of messages? Which of the orders implies another?
- 3.3 How can clock synchronization assist in finding the *primary* event of an alarm shower?
- 3.4 What is the difference between UTC and TAI? Why is TAI better suited as a time base for distributed real-time systems than UTC?
- 3.5 Define the notions of *offset*, *drift*, *drift rate*, *precision*, and *accuracy*.
- 3.6 What is the difference between *internal* synchronization and *external* synchronization?
- 3.7 What are the fundamental limits of time measurement?
- 3.8 When is an event set ε/Δ -*precedent*?

- 3.9 What is an *agreement protocol*? Why should we try to avoid agreement protocols in real-time systems? When is it impossible to avoid agreement protocols?
- 3.10 What is a *sparse time base*? How can a sparse time base help to avoid agreement protocols?
- 3.11 Give an example that shows that, in an ensemble of three clocks, a Byzantine clock can disturb the two good clocks such that the synchronization condition is violated.
- 3.12 Given a clock synchronization system that achieves a precision of $90\ \mu\text{s}$, what is a reasonable granularity for the global time? What are the limits for the observed values for a time interval of $1.1\ \text{ms}$?
- 3.13 What is the role of the *convergence function* in internal clock synchronization?
- 3.14 Given a latency jitter of $20\ \mu\text{s}$, a clock drift rate of $10^{-5}\ \text{s/s}$, and a resynchronization period of $1\ \text{s}$, what precision can be achieved by the central master algorithm?
- 3.15 What is the effect of a Byzantine error on the quality of synchronization by the FTA algorithm?
- 3.16 Given a latency jitter of $20\ \mu\text{s}$, a clock drift rate of $10^{-5}\ \text{s/s}$ and a resynchronization period of $1\ \text{s}$, what precision can be achieved by the FTA algorithm in a system with ten clocks where one clock could be malicious?
- 3.17 Discuss the consequences of an error in the external clock synchronization. What effect can such an error have on the internal clock synchronization in the worst possible scenario?

Chapter 4

Real-Time Model

Overview The objective of this chapter is to introduce the reader to a cross-domain architecture model of the behavior of a real-time system. This model will be used throughout the rest of the book. The model is based on three basic concepts, the concept of a *computational component*, the concept of *state*, and the concept of a *message*. Large systems can be built by the recursive composition of components that communicate by the exchange of messages. Components can be reused on the basis of their interface specification without having to understand the component internals. Concerns about the understandability have been of utmost importance in the development of this model.

The chapter is structured as follows. In Sect. 4.1 we give a broad outline of the model, describing the essential characteristics of a *component* and a *message*. Related components that work towards a joint objective are grouped into *clusters*. The differences between *temporal control* and *logical control* are explained. The following section elaborates on the close relationship between *real-time* and the *state of a component*. The importance of a well-defined *ground state* for the dynamic reintegration of a component is highlighted. Section 4.3 refines the message concept and introduces the notions of event-triggered messages, time-triggered messages, and data streams. Section 4.4 presents the four interfaces of a component, two operational interfaces and two control interfaces. Section 4.5 deals with the concept of a *gateway component* that links two clusters that adhere to different *architectural styles*. Section 4.6 deals with the specification of the linking interface of a component. The linking interface is the most important interface of a component. It is relevant for the integration of a component within a cluster and contains all the information that is needed for the use of a component. The linking interface specifications consists of three parts: (1) the transport specification that contains the information for the transport of the messages, (2) the operational specification that is concerned with interoperability of components and the establishment of the message variables, and (3) the meta-level specification that assigns meaning to the message variables. Points to consider when composing a set of components to build *systems of subsystems* or *system of systems* are discussed in Sect. 4.6. In this section the four principles of composability are introduced and the notion of a multilevel system is explained.

4.1 Model Outline

Viewed from the perspective of an outside observer, a real-time (RT) system can be decomposed into three communicating subsystems: a *controlled object* (the *physical* subsystem, the behavior of which is governed by the laws of physics), a “*distributed*” *computer subsystem* (the *cyber system*, the behavior of which is governed by the programs that are executed on digital computers), and a *human user or operator*. The distributed computer system consists of computational nodes that interact by the exchange of messages. A *computational node* can host one or more *computational components*.

4.1.1 Components and Messages

We call the process of executing an algorithm by a processing unit a *computation or task*. Computations are performed by *components*. In our model, a component is a self-contained hardware/software unit that interacts with its environment exclusively by the exchange of messages. We call the *timed sequence of output messages* that a component produces at an interface with its environment the *behavior* of the component at that interface. The *intended behavior* of a component is called its *service*. An unintended behavior is called a *failure*. The internal structure of a component, whether complex or simple, is neither visible, nor of concern, to a user of a component.

A component consists of a *design* (e.g., the software) and an *embodiment* (e.g., the hardware, including a processing unit, memory, and an I/O interface). A *real-time component* contains a real-time clock and is thus aware of the progression of real-time. After *power-up*, a component enters a *ready-for-start* state to wait for a triggering signal that indicates the start of execution of the component’s computations. Whenever the triggering signal occurs, the component starts its predefined computations at the *start instant*. It then reads input messages and its internal state, produces output messages and an updated internal state, and so on until it terminates its computation – if ever – at a *termination instant*. It then enters the *ready-for-start* state again to wait for the next triggering signal. In a cyclic system, the real-time clock produces a triggering signal at the start of the next cycle.

An important principle of our model is the *consequent separation of the computational components* from the *communication infrastructure* in a distributed computer system. The communication infrastructure provides for the transport of unidirectional messages from a sending component to one or more receiving components (*multicasting*) within a given interval of real-time. *Unidirectionality* of messages supports the unidirectional reasoning structure of *causal chains* and eliminates any dependency of the sender on the receiver(s). This property of *sender independence* is of utmost importance in the design of fault-tolerant systems, because it avoids error back-propagation from a faulty receiving component to a correct sending component by design.

Multicasting is required for the following reasons:

1. Multicasting supports the *non-intrusive observation* of component interactions by an independent observer component, thus making the interactions of components perceptually accessible and removing the barrier to understanding that has its origin in *hidden interactions* (see Sect. 2.1.3).
2. Multicasting is required for the implementation of fault-tolerance by active redundancy, where each single message has to be sent to a set of replicated components.

A message is sent at a *send instant* and arrives at the receiver(s) at some later instant, the *receive instant*. The message-paradigm combines the temporal-control and the value aspect of an interaction into a single concept. The temporal properties of a message include information about the send instants, the temporal order, the inter-arrival time of messages (e.g., periodic, sporadic, aperiodic recurrence), and the latency of the message transport. Messages can be used to synchronize a sender and a receiver. A message contains a data-field that holds a data structure that is transported from the sender to the receiver. The communication infrastructure is *agnostic* about the contents of the data field. The message concept supports *data atomicity* (i.e., atomic delivery of the complete data structure contained in a message). A single well-designed message-passing service provides a simple interface of a component to other components inside and outside a node and to the environment of a component. It facilitates encapsulation, reconfiguration, and the recovery of component services.

4.1.2 Cluster of Components

A cluster is a *set of related components* that have been grouped together in order to achieve a common objective (Fig. 4.1). In addition to the set of components,

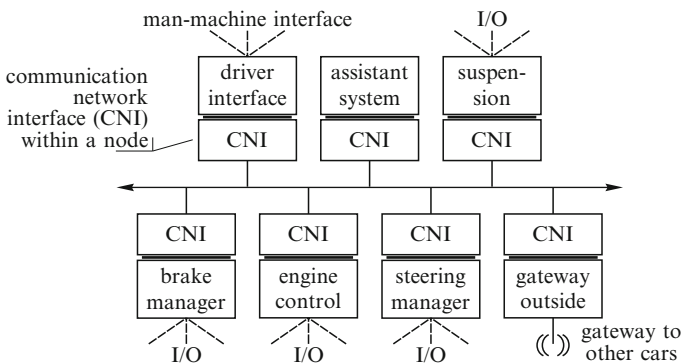


Fig. 4.1 Example of an *in-car cluster*

a cluster must contain an intra-cluster communication system that provides for the transport of messages among the components of the cluster. The components that form a computational cluster agree on a common *architectural style* (see the last paragraphs in Sect. 2.2.4).

Example: Figure 4.1 depicts an example of a computational cluster within a car. This cluster consists of a computational component, the *assistant system*, and gateway components to the man-machine interface (the driver), the physical subsystems of the car, and a gateway to other cars via a wire-less vehicle-to-vehicle communication link.

4.1.3 Temporal Control Versus Logical Control

Let us revisit the rolling mill example of Fig. 1.9 of Chap. 1 and specify a relation between measured variables that must be monitored by an alarm-monitoring task in the MMI component. Assume that the pressures p_1 , p_2 , and p_3 , between the roles of the three drives are measured by the three controller components of Fig. 1.9. The measurements are sent to the man-machine interface (MMI) component for checking the following alarm condition:

```

when(( $p_1 < p_2$ )  $\wedge$  ( $p_2 < p_3$ ))
then everything ok
else raise pressure alarm;
```

This looks like a reasonable specification at the user level. Whenever the pressure between the rolls does not satisfy the specified condition, a pressure alarm must be raised.

During the refinement of this specification by a system architect, four different tasks (three *measurement* tasks in the three control nodes and one *alarm-monitoring* task in the MMI node of Fig. 1.9) must be designed. The following questions concerning the temporal activation of these tasks arise:

1. What is the maximum tolerable time interval between the occurrence of the alarm condition in the controlled object and the raising of the alarm at the MMI? Because the communication among the components takes a finite amount of time, some time intervals are unavoidable!
2. What are the maximum tolerable time intervals between the three pressure measurements at the three different control nodes? If these time intervals are not properly controlled, false alarms will be generated or important alarms will be missed.
3. When and how often do we have to activate the pressure measurement tasks at the three control nodes?
4. When do we have to activate the alarm-monitoring task at the alarm-monitoring component (the MMI component in Fig. 1.9)?

Because these questions are not answered by the given specification, it is evident that this specification lacks precise information concerning the architectural requirements in the temporal domain. The temporal dimension is buried in the ill-specified

semantics of the **when** statement. In this example, the **when** statement is intended to serve two purposes. It is specifying

1. the point in time when the alarm condition must be raised, and
2. the conditions in the value domain that must be monitored.

It thus intermingles two separate issues, the behavior in the time domain and the behavior in the value domain. A clean distinction between these two issues requires a careful definition of the concepts of *temporal control* and *logical control*.

Temporal control is concerned with determining the instants in the domain of real time when computations must be performed, i.e., when tasks must be activated. These instants are derived from the dynamics of the application. In the above example, the decision regarding the instants at which the pressure measuring tasks and the alarm-monitoring task must be activated is an issue of temporal control. Temporal control is related to the progression of *real-time*.

Logical control is concerned with the control flow *within* a task that is determined by the given task structure and the particular input data, in order to realize the desired computation. In the above example, the evaluation of the branch condition and the selection of one of the two alternatives is an example of *logical control*. The time interval needed for the execution of a task that performs the logical control is determined by the frequency of the oscillator that drives the processing unit – we call this time-base the *execution time*. The execution time is determined by the given implementation and will change if we replace the given processor by a faster one.

Since *temporal control* is related to *real time*, while *logical control* is related to *execution time*, a careful distinction must be made between these two types of control (see also Sect. 8.3.4). A good design will separate these two control issues in order to decouple the reasoning about temporal constraints dictated by the application, from the reasoning about logical issues inside the algorithmic part of a program. Synchronous real-time languages, such as LUSTRE [Hal92], ESTEREL [Ber85], and SL [Bou96] distinguish cleanly between logical control and temporal control. In these languages, the progression of real-time is partitioned into an (infinite) sequence of intervals of specified real-time duration, which we call steps. Each step begins with a tick of a real-time clock that starts a computational task (logical control). The computational model assumes that a task, once activated by the tick of a real-time clock (temporal control), finishes its computation *quasi immediately*. Practically this means that a task must terminate its executions before the next triggering signal (the next tick of the real-time clock) initiates the next execution of the task.

The periodic finite state machine (PFSM) model [Kop07] extends the classic FSM, which is concerned with *logical control*, by introducing a new dimension for the progression of a global sparse time to cover *temporal control* issues.

If the issues of temporal control and logical control are intermingled in a program segment, then it is not possible to determine the worst-case execution time (WCET – see Sect. 10.2) of this program segment without analyzing the behavior of the environment of this program segment. This is a violation of the design principle *Separation of Concerns* (see Sect. 2.5).

Example: A *semaphore wait statement* is a temporal control statement. If a *semaphore wait statement* is contained in a program segment that also includes logical control (algorithmic) statements, then the temporal behavior of this program segment depends on both, the progress of *execution time* and the progress of *real-time* (see also Sects. 9.2 and 10.2).

4.1.4 Event-Triggered Control Versus Time-Triggered Control

In Sect. 4.1.1, we introduced the notion of a *triggering signal*, i.e., a control signal that indicates the instant when an activity should start in the temporal domain. What are the possible origins of such a triggering signal? The triggering signal can be associated either with the occurrence of a *significant event* – we call this *event-triggered control* – or with the arrival of a *specified instant* on the time line – we call this *time-triggered control*.

The significant events that form the basis of event-triggered control can be the arrival of a particular message, the completion of an activity inside a component, the occurrence of an external interrupt, or the execution of a *send message statement* by the application software. Although the occurrences of significant events are normally sporadic, there should be a minimal real-time interval between two successive events so that an overload of the communication system and the receiver of the events can be avoided. We call such an event-stream, for which a minimum inter-arrival time between events is maintained, a *rate-controlled* event stream.

Time-triggered control signals are derived from the progression of the global time that is available in every component. Time-triggered control signals are normally cyclic. A cycle can be characterized by its *period*, i.e., the real-time interval between two successive *cycle starts*, and by its *phase*, that is the interval between the start of the period, expressed in the global time, and the *cycle start* (see also Sect. 3.3.4). We assume that a cycle is associated with every time-triggered activity.

4.2 Component State

The concept of *state* of a component is introduced in order to separate *past behavior* from *future behavior* of a real-time component. The concept of state requires a clear distinction between past events and future events, i.e., there must be a consistent temporal order among the events of significance (refer to Sect. 3.3.2).

4.2.1 Definition of State

The notion of *state* is widely used in the computer science literature, albeit sometimes with meanings that are different from the meaning of *state* that is useful

in a real-time system context. In order to clarify the situation, we follow the precise definition of Mesarovic [Mes89, p. 45], which is the basis for our elaborations:

The state enables the determination of a future output solely on the basis of the future input and the state the system is in. In other words, the state enables a “decoupling” of the past from the present and future. The state embodies all past history of a system. Knowing the state “supplants” knowledge of the past. . . . Apparently, for this role to be meaningful, the notion of past and future must be relevant for the system considered.

The sparse time model introduced in Sect. 3.3.2 makes it possible to establish the consistent system-wide separation of the past from the future that is necessary to define a *consistent system state* in a distributed real-time computer system.

4.2.2 The Pocket Calculator Example

Let us look at the familiar example of a pocket calculator to investigate the concept of *state* in more detail. An operand, i.e., a number of keyboard digits, must be entered into the calculator before the selected operator, e.g., a key for the trigonometric function *sine*, can be pressed to initiate the computation of the selected function. After the computation terminates, the result is displayed at the calculator display. If we consider the computation to be an atomic operation and observe the system immediately before or after the execution of this atomic operation, the internal state of this simple calculator device is empty at the points of observation

Let us now observe the pocket calculator (Fig. 4.2) during the interval between the *start* of the computation and the *end* of the computation. If the internals of the device can be observed, then a number of intermediate results that are stored in the local memory of the pocket calculator can be traced during the series expansion of the *sine* function. If the computation is interrupted at an instant between the instants *start* and *end*, the contents of the program counter and all memory cells that hold the intermediate results form the *state at this instant* of interruption. After the end instant of the computation, the contents of these intermediate memory cells are no longer relevant, and the state is empty again. Figure 4.3 depicts a typical *expansion* and *contraction* of the state during a computation.

Let us now analyze the state (sometimes also called history state or h-state) of a pocket calculator used to sum up a set of numbers. When entering a new number, the sum of the previously entered numbers must be stored in the device. If we interrupt the work after having added a subset of numbers and continue the addition

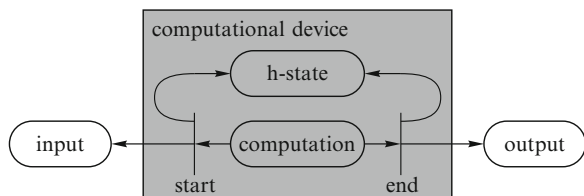
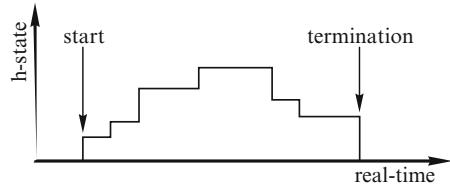


Fig. 4.2 Model of a pocket calculator

Fig. 4.3 Expansion and contraction of the *state* during a computation



with a new calculator, we first have to input the intermediate result of the previously added numbers. At the user level, the state consists of the intermediate result of the previous additions. At the end of the operation, we receive the final result and clear the memory of the calculator. The state is empty again.

From this simple example we can conclude that the size of the state of a system depends on the *instant of observation* of the system. If the granularity of observations is increased, and if the observation points are selected immediately before or after an atomic operation at the chosen level of abstraction, then, the size of the state can be reduced.

The state at any instant of interruption is contained in the contents of the program counter and all *state variables* that must be loaded into a *virgin* hardware device to resume the operation from the instant of interruption onward. If an interruption is caused by a failure of a component and we must reintegrate the component into a running system, then the size of the state that must be reloaded into the repaired component is of concern.

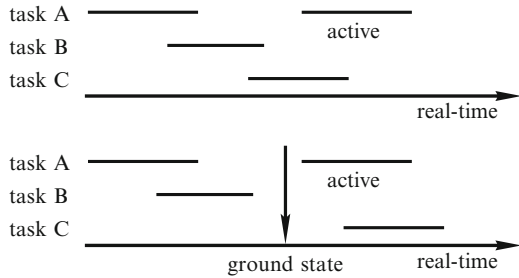
If our hardware device is a programmable computer, we must first load the software, i.e., operating system, the set of application programs, and the initial values for all state variables, into a *virgin* hardware device before we can start a computation. We call the *totality of software* that has to be loaded into a virgin hardware device the *core image* or the *job*. Normally, the job is a data structure that is static, i.e., it is not changed during the execution of the software. In some embedded hardware devices the job is stored in a ROM (read-only memory) and thus the *software becomes literally a part of the hardware*.

4.2.3 Ground State

In order to facilitate the dynamic reintegration of a component into a running system, it is necessary to design periodic *reintegration instants* into the behavior, where the size of a component's state at the reintegration instant contained is a small set of well-defined application specific *state variables*. We call the state at the reintegration instant the *ground state* (*g-state*) of a component and the temporal interval between two reintegration points the *ground cycle*.

The ground state at the reintegration point is stored in a declared *g-state data structure*. Designing a minimal ground state data structure is the result of an explicit design effort that involves a semantic analysis of the given application. The designer

Fig. 4.4 Task executions: without (above), and with (below) ground state



has to find periodic instants where there is a maximum decoupling of future behavior from past behavior. This is relatively easy in *cyclic applications*, such as in control applications and multimedia applications. In these applications, a natural reintegration instant is immediately after the termination of one cycle and before the beginning of the next cycle. Design techniques for the minimization of the ground state are discussed in Sect. 6.6.

At the reintegration instant, no task may be active and all communication channels must be flushed, i.e., there are no messages in transit [Ahu90]. Consider a node that contains a number of concurrently executing tasks that exchange messages among each other and with the environment of the node. Let us choose a level of abstraction that considers the execution of a task as an *atomic action*. If the execution of the tasks is asynchronous, then, the situation depicted in the upper part of Fig. 4.4, can arise; at every instant, there is at least one active task, thus implying that there is no instant when the ground state of the node can be defined.

In the lower part of Fig. 4.4, there is an instant when no task is active and when all channels are empty, i.e., when the system is in the *g-state*. If a node is in the *g-state*, then the entire state that is essential for the future operation of the node is contained in the declared ground state data structure.

Example: Consider the relation between the size of the *g-state* and the duration of the ground (*g*) cycle in the design of a clock. If the *g-cycle* is 24 h and the start of a new day is the reintegration instant, then the *g-state* is empty. If every complete hour is a reintegration instant, then the *g-state* contains 5 bits (to identify one out of 24 h per day). If every complete minute is a reintegration instant, then the *g-state* is 11 bits (to identify one of 1,440 min per day). If every complete second is a reintegration instant, then the *g-state* is 17 bits (to identify one out of 86,400 s per day). It depends on the application characteristics to determine which one of the above alternatives is optimal for a given application. If the clock is an alarm clock that can store up to five alarms and the accuracy of an alarm is 5 min, then the *g-state* for every alarm is 9 bits (8 bit for the alarm and 1 bit to denote whether the alarm is *on* or *off*). If we assume that the reintegration cycle is 1 s and 5 alarms must be supported, then the *g state* message in this simple example is 62 bits in lengths. This *g-state* can be stored in an 8-byte data structure. In the restart message, the time field must be corrected such that it contains the precise time value at the restart instant.

Table 4.1 shows that *g-state recovery* is substantially different from *checkpoint recovery* that is used to establish a consistent state after a failure in a non-real-time data-intensive system.

Table 4.1 Comparison of g-state recovery and checkpoint recovery

	G-state recovery	Checkpoint recovery
Data selection	Application specific small data set that is essential for the future operation of the system.	All data elements that have been modified since the start of the computation.
Data modification	G-state data is modified to establish consistency between the g-state and the state of the environment at the future reintegration instant. Rollback of the environment is not possible in a real-time system.	No modification of checkpoint data. Consistency is established by rolling the (data) environment back to the instant when the checkpoint data was captured.

4.2.4 Database Components

We call a component where the number of *dynamic data elements*, i.e., data elements that are modified by the computations, is too large for storing them in a single ground state message a *database component*. The data elements that are contained in a database component can be either part of the *state* or *archival data*.

The term *archival data* refers to data that has been collected for archival purposes and does and not have a direct influence on the future behavior of the component. Archival data is needed to record the history of production process variables in order to be able to analyze a production process at some later time line. It is a good practice to send archival data to a remote storage site for archival data as soon as possible.

Example: The legendary *black box* in an airplane contains archival data. One could send these data immediately after collection via a satellite link to a storage site on the ground in order to avoid the problems of having to recover a black box after an accident.

4.3 The Message Concept

The concept of a *message* is the third basic concept of our model. A *message* is an atomic data structure that is formed for the purpose of communication, i.e., data transmission and synchronization among components.

4.3.1 Message Structure

The concept of a *message* is related to the concept of a *letter* in the postal system. A message consists of a header, a data field, and a trailer. The header, corresponding to the envelope of a letter, contains the *port address* of the receiver (the mailbox) where the message must be delivered, information about how the message must be

handled (e.g., a registered letter), and may contain the address of the sender. The data field contains the application specific data of the message, corresponding to the content of a letter. The trailer, corresponding to the signature in a letter, contains information that allows the receiver to detect whether the data contained in the message is uncorrupted and authentic. There are different types of trailers in use: the most common trailer is a CRC-field that allows the receiver to determine whether the data field has been corrupted during transport. A message may also contain an *electronic signature* in the trailer that makes it possible to determine whether the *authenticated contents* of the message have not been *altered* (see Sect. 6.2). The notion of *atomicity* implies that a message is delivered either in its entirety or not at all. If a message is corrupted or only parts of the message arrive at the receiver's site, the whole message is discarded.

The temporal dimension of the message concept relates to the instants when a message is sent by the sender and received by the receiver, and consequently how long the message has been in transit. We call the interval between the *send instant* and the *receive instant* the *transport delay*. A second aspect of the temporal dimension relates to the rate of message production by the sender and message consumption by the receiver. If the sending rate is constrained, then we speak about a *rate-constrained* message system. In case the sender's rate is not constrained, the sender may overload the transport capacity of the communication system (we call this *congestion*) or the processing capacity of the receiver. In case the receiver cannot keep up with the message production rate of the sender, the receiver can send a control message to the sender telling the sender to slow down (*back pressure flow control*). Alternatively, the receiver or the communication system may simply discard messages that exceed its processing capacity.

4.3.2 Event Information Versus State Information

The state of a dynamic system changes as real-time progresses. Let us assume that we periodically observe the state variables of a system with a duration d between two successive observation instants. If we observe that the value of all state variables is the same in two successive observations, then we infer that no *event*, i.e., *change of state*, has occurred in the last observation interval d . This conclusion is only valid, if the dynamics of the system is slow compared to our observation interval d (refer to *Shannon's theorem* [Jer77]). If two successive observations of the values of some state variables differ, then we conclude that at least one event has occurred in the last observation interval d . We can report about the occurrence of an event, i.e., a change of state, in two different ways: either by sending a single message containing *event information* or by sending a sequence of messages containing *state information*.

We talk about *event information* if the information conveys the *difference in values* of the previous state observation and the current state observation.

The instant of the current (later) observation is postulated to be the instant of event occurrence. This assumption is not fully accurate, since the event may have occurred at any instant during the last interval of duration d . We can reduce this temporal observation error of an event by making the interval d smaller, but we cannot fully eliminate the temporal uncertainty about the observation of events. This holds true even if we use the interrupt system of a processor to report about an event. The input signal that relays the occurrence of an interrupt is not sensed continuously by the processor, but only after the termination of the execution of an instruction. This delay is introduced in order to reduce the amount of *processor state* that has to be saved and restored in the processor in order to be able to continue the interrupted task after the interrupt has been served. As outlined in Sect. 4.2.3, the state is minimal immediately before or after the execution of an atomic operation – in this case, the execution of a complete instruction by a processor.

If the precise timing of an event is critical, we can provide a separate dedicated hardware device to time-stamp the observed state-change of the interrupt line immediately and thus reduce the temporal observation error to a value that is in the order of magnitude of the cycle time of the hardware. Such hardware devices are introduced to achieve precise clock synchronization in distributed systems, where the precision of the distributed clocking system must be in the nanosecond range.

Example: The IEEE 1588 standard for clock synchronization suggests the implementation of a separate hardware device to precisely capture the arrival instant of a clock synchronization message.

We talk about *state information* if the information conveys the *values* of the current state variables. If the data field of a message contains *state information*, it is up to the receiver to compare two successive state observations and determine whether an event has occurred or not. The temporal uncertainty about the event occurrence is the same as above.

4.3.3 Event-Triggered Message

A message is called *event-triggered (ET)* if the triggering signal for sending the message is derived from the occurrence of a significant event, such as the execution of a *send message* command by the application software.

ET messages are well suited to transport *event information*. Since an event refers to a *unique* change of state, the receiver must consume every single event message and it is not allowed to duplicate an event message. We say that an event message must adhere to the *exactly-once* semantics. The event message model is the standard model of message transmission that is followed in most non-real time systems.

Example: The event message *valve must be closed by 5°* means that the new intended position of the valve equals the current position plus 5°. If this event message is lost or duplicated, then the image of the state of the valve position in the computer will differ from

the actual state of the valve position in the environment by 5° . This error can be corrected by *state alignment*, i.e., the (full) state of the intended valve position is sent to the valve.

In an event-triggered system, error detection is in the responsibility of the sender who must receive an explicit acknowledgment message from the receiver telling the sender that the message has arrived correctly. The receiver cannot perform error detection, because the receiver cannot distinguish between *no activity by the sender* and *loss of message*. Thus the control flow must be *bidirectional* control flow, even if the data flow is only *unidirectional*. The sender must be *time-aware*, because it must decide within a finite interval of real time that the communication has failed. This is one reason why we cannot build fault-tolerant system that are unaware of the progression of *real time*.

4.3.4 Time-Triggered Message

A message is called *time-triggered* (TT) if the triggering signal for sending the message is derived from the progression of real-time. There is a *cycle*, characterized by its *period* and *phase*, assigned to every time-triggered message before the system starts operating. At the instant of cycle start, the transmission of the message is initiated automatically by the operating system. There is no *send message command* necessary in TT message transmission.

TT messages are well suited to transport *state information*. A TT message that contains state information is called a *state message*. Since a new version of a state observation normally replaces the existing older version, it is reasonable that a new state message *updates-in-place* the older message. On reading, a state message is *not consumed*; it remains in the memory until it is updated by a new version. The semantics of state messages is similar to the semantics of a *program variable* that can be read many times without consuming it. Since there are no queues involved in state message transmissions, queue overflow is no issue. Based on the *a priori* known cycle of state messages, the *receiver can perform error detection autonomously* to detect the loss of a state message. State messages support the *principle of independence* (refer to Sect. 2.5) since sender and receiver can operate at different (independent) rates and there is no means for a receiver to influence the sender.

Example: A temperature sensor observes the state of a temperature sensor in the environment every second. A state-message is well suited to transport this observation to a user that stores this observation in a program variable named *temperature*. The user-program can read this variable *temperature* whenever it needs to refer to the current temperature of the environment, knowing that the value stored in this variable is *up to date* to within about 2 s. If a single state message is lost, then for one cycle the value stored in this variable is *up to date* to only within about 3 s. Since, in a time-triggered system, the communication system knows *a priori* when a new state message must arrive, it can associate a *flag* with the variable *temperature* to inform the user if the variable *temperature* has been properly updated in the last cycle.

4.4 Component Interfaces

Let us assume that the design of a large component-based system is partitioned into two distinct design phases, *architecture design* and *component design* (see also Sect. 11.2 on *system design*). At the end of the architecture design phase, a *platform independent model* (PIM) of a system is available. The PIM is an executable model that partitions the system into clusters and components and contains the precise interface specification (in the domains of value and time) of the linking interfaces of the components. The linking interface specification of the PIM is *agnostic* about the component implementation and can be expressed in a high-level executable system language, e.g., in *System C*. A PIM component that is transformed to a form that can be executed on the final execution platform is called a *platform-specific model* (PSM) of the component. The PSM has the same interface characteristics as the PIM. In many cases, an appropriate compiler can transform the PIM to the PSM automatically.

An interface should serve a *single well-defined purpose* (Principle of *separation of concerns*, see Sect. 2.5). Based on *purpose*, we distinguish between the following four message interfaces of a component (Fig. 4.5):

- The *Linking Interface* (LIF) provides the specified service of the component at the considered level of abstraction. This interface is agnostic about the component implementation. It is the same for the PIM and the PSM.
- The *Technology Independent Control Interface* (TII) is used to configure and control the execution of the component. This interface is agnostic about the component implementation. It is the same for the PIM and the PSM.
- The *Technology Dependent Debug Interface* (TDI) is used to provide access to the internals of a component for the purpose of maintenance and debugging. This interface is implementation specific.
- The *Local Interface* links a component to the external world that is the external environment of a cluster. This interface is syntactically specified at the PSM level only, although the *semantic content* of this interface is contained in the LIF.

The LIF and the *local interface* are *operational interfaces*, while the TII and TDI are *control interfaces*. The control interfaces are used to control, monitor, or debug

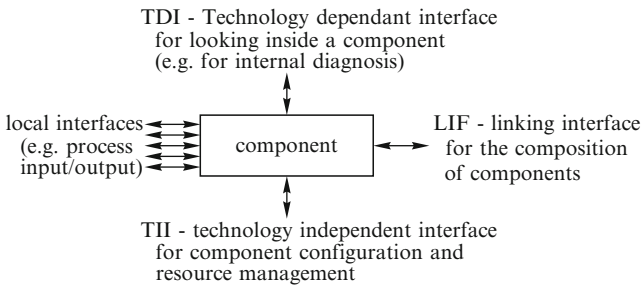


Fig. 4.5 The four interfaces of a component

a component, while the operational interfaces are in use during the normal operation of a component. Before discussing these four interfaces in detail, we elaborate on some general properties of message interfaces.

4.4.1 Interface Characterization

Push versus Pull Interface. There are two options to handle the arrival of a new message at the interface of a receiving component:

- *Information push.* The communication system raises an interrupt and forces the component to immediately act on the message. Control over the temporal behavior of the component is delegated to the environment outside of the component.
- *Information pull.* The communication system puts the message in an intermediate storage location. The component looks periodically if a new message has arrived. Temporal control remains inside the component.

In real-time systems, the information pull strategy should be followed whenever possible. Only in situations when an immediate action is required and the delay of one cycle that is introduced by the information pull strategy is not acceptable, one should resort to the information push strategy. In the latter case, mechanisms must be put into place to protect the component from erroneous interrupts caused by failures external to the component (see also Sect. 9.5.3). The information push strategy violates the principles of independence (see Sect. 2.5).

Example: An engine control component for an automotive engine worked fine as long as it was not integrated with the theft avoidance system. The message interface between the engine controller and the theft avoidance system was designed as a push interface, causing the sporadic interruption of a time-critical engine control task when a message arrived from the theft avoidance system at an ill-timed instant. As a consequence the engine controller sporadically missed a deadline and failed. Changing the interface to a *pull interface* solved the problem.

Elementary versus Composite Interface. In a distributed real-time system, there are many situations where a simple unidirectional data flow between a sending and a receiving component must be implemented. We call such an interface *elementary* if both the data flow and the control flow are unidirectional. If, in a unidirectional data flow scenario, the control flow is bidirectional we call the interface *composite* (Fig. 4.6) [Kop99].

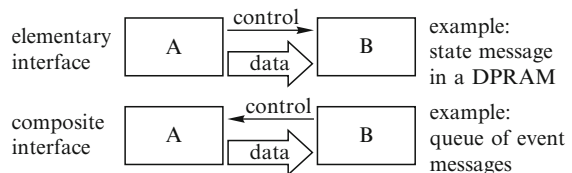


Fig. 4.6 Elementary vs. composite interface

Elementary interfaces are *inherently simpler* than composite interfaces, because there is *no dependency* of the behavior of the sender on the behavior of the receiver. We can reason about the correctness of the sender without having to consider the behavior of the receiver. This is of particular importance in safety-critical systems.

4.4.2 Linking Interface

The *services* of a component are accessible at its *cluster LIF*. The cluster LIF of a component is an *operational message-based* interface that interconnects a component with the other components of the cluster and is thus the interface for the integration of components into the cluster. The LIF of a component abstracts from the internal structure and the local interfaces of the component. The specification of the LIF must be self-contained and cover not only the functionality and timing of the component itself, but also the semantics of its local interfaces. The LIF is *technology agnostic* in the sense that the LIF does not expose implementation details of the internals of the component or of its local interfaces. A technology agnostic LIF ensures that different implementations of computational components (e.g., general purpose CPU, FPGA, ASIC) and different local Input/Output subsystems can be connected to a component without any modification to the other components that interact with this component across its message based LIF.

Example: In an input/output component, the external input and output signals are connected by a local point-to-point wiring interface. The introduction of a bus system, e.g., a CAN bus, will not change the cluster LIF of the input/output component, as long as the temporal properties of the data appearing at the LIF are the same.

4.4.3 Technology Independent Control Interface

The technology independent interface is a *control interface* that is used to configure a component, e.g., assign the proper names to a component and its input/output ports, to *reset*, *start*, and *restart* a component and to monitor and control the resource requirements (e.g., power) of a component during run time, if required. Furthermore, the TII is used to configure and reconfigure a component, i.e., to assign a specific *job* (i.e., core image) to the programmable component hardware.

The messages that arrive at the TII communicate either directly with the component hardware (e.g., *reset*), with the component's operating system (e.g., *start a task*), or with the *middleware* of the component, but not with the application software. The TII is thus orthogonal to the LIF. This strict separation of the application specific message interfaces (LIF) from the system control interface of a component (TII) simplifies the application software and reduces the overall complexity of a component (see also the *principle of separation of concerns* in Sect. 2.5).

4.4.4 *Technology Dependent Debug Interface*

The TDI is a *special control interface* that provides a means to look inside a component and to observe the internal variables of a component. It is related to the *boundary scan interface* that is widely used for testing and debugging large VLSI chips and has been standardized in the IEEE standard 1149.1 (also known as the JTAG Standard). The TDI is intended for the person who has a deep understanding of the component internals. The TDI is of no relevance for the user of the LIF services of the component or the system engineer who configures a component. The precise specification of the TDI depends on the technology of the component implementation, and will be different if the same functionality of a component is realized by software running on a CPU, by an FPGA or by an ASIC.

4.4.5 *Local Interfaces*

The local interfaces establish a connection between a component and its outside environment, e.g., the sensors and actuators in the physical plant, the human operator, or another computer system. A component that contains a local interface is called a *gateway component* or an *open component*, in contrast to a component that does not contain a local interface, which is called a *closed component*. The distinction between open and closed components is important from the point of view of the specification of the semantics of the LIF of the component. Only closed components can be fully specified without knowing the *context of use* of the component.

From the point of view of the *cluster LIF*, only the *timing* and the *semantic content*, i.e., the meaning of the information exchanged across a local interface is of relevance, while the detailed structure, naming, and access mechanisms of the local interface is *intentionally left unspecified* at the cluster level. A modification of the local access mechanisms, e.g., the exchange of a CAN Bus by Ethernet, will not have any effect on the LIF specification, and consequently on the users of the LIF specification, as long as the *semantic content* and the *timing* of the relevant data items are the same (see also the *principle of abstraction* in Sect. 2.5).

Example: A component that calculates a trigonometric function is a *closed component*. Its functionality can be formally specified. A component that reads a temperature sensor is an *open component*. The meaning of *temperature* is application specific, since it depends on the position where the sensor is placed in the physical plant

4.5 Gateway Component

Viewed from the perspective of a cluster, a gateway component is an *open component* that links two worlds, the *internal world* of the cluster and the *external world* of the cluster environment. A gateway component acts as a *mediator* between these two

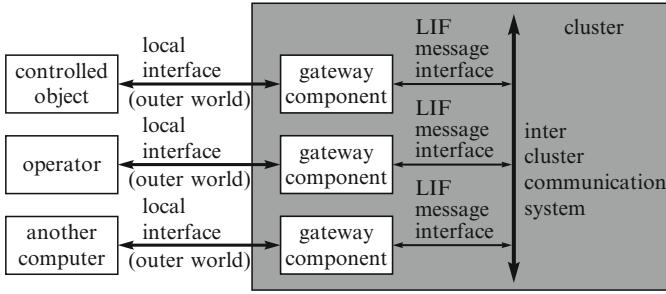


Fig. 4.7 Gateway components between the LIF and the local external world interface

worlds. It has two operational interfaces, the LIF message interface to the cluster and the local interface to the external world, which can be the physical plant, a man-machine interface, or another computer system (Fig. 4.7). Viewed from the outside of a cluster, the *role of the interfaces is reversed*. The previous local interface becomes the new LIF and the previous LIF becomes the new local interface.

4.5.1 Property Mismatches

Every system is developed according to an *architectural style*, i.e., a set of adopted rules and conventions for the concept formation, representation of data, naming, programming, interaction of components, semantics of the data, and many more. The architectural style characterizes the *properties* of the entities that represent the design. The details of the architectural style are sometimes explicitly documented, but more often only implicitly followed by the development team, reflecting the unstated conventions that govern the development community (see also the last paragraph of Sect. 2.2.4).

Whenever a communication channel links two systems, developed by two different organizations, it is highly probable that some of the properties of the messages that are exchanged across this channel are in disagreement. We call any disagreement in a property of the data or the protocol between the sender and the receiver of a message a *property mismatch*. It is up to a gateway component to resolve property mismatches.

Example: Assume that there is a difference in *endianness*, i.e., the byte ordering of data, between the sender and the receiver of a message. If the sender assumes *big endian*, i.e., the most significant byte is first, and the receiver assumes *little endian*, i.e., the least significant byte is first, then this *property mismatch* has to be resolved either by the sender, or by the receiver, or by an intermediate connector system, i.e., the gateway component.

Property mismatches occur at the borders where systems interact, not inside a well-designed system. Inside a system, where all partners respect the rules and constraints of the architectural style, property mismatches are normally no issue.

Property mismatches should be resolved in the gateway components that link two systems in order to maintain the integrity of the architectural styles within each of the interacting subsystems.

4.5.2 LIF Versus Local Interface of a Gateway Component

As noted before, the set of related components that form a cluster share a common *architectural style* at their cluster LIFs. This means that *property mismatches* among LIF-cluster messages are rare.

This is in contrast to the messages that cross a gateway component. These messages come from two different worlds that are characterized by two different architectural styles. Property mismatches, syntactic incompatibility, incoherent naming, and differences in representation between these two sets of messages are the rule rather than the exception. It is the main task of a gateway component to translate the architectural style of one world to the architectural style of the other world without changing the *semantic content* of the message variables.

There are situations when the architectural styles of the two interfacing worlds are based on a different *conceptualization* of reality. i.e., there are not only differences in the *names* and the *representations* of the same concepts (as shown in the example in Sect. 2.2.4), but the concepts themselves, i.e., the semantic contents are dissimilar.

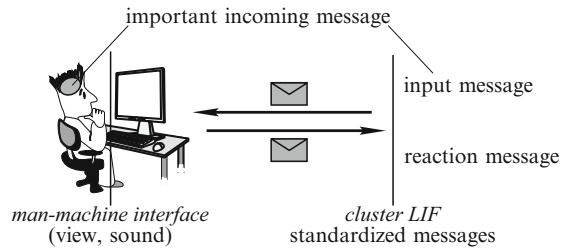
Viewed from a given cluster, the gateway component hides the details of the external world (the local interface) from the standardized message formats within the computational cluster and filters the incoming information: only the information that is relevant for the operation of the cluster under consideration is exposed in the form of cluster-standard messages at the cluster LIF of the gateway component.

An important special case of an external-world interface is a process I/O interface, which establishes a link between the cyber world and the physical world. Table 4.2 depicts some of the differences between the LIF messages and a process control I/O interface to the physical plant.

Table 4.2 Characteristics of a LIF versus a local process I/O interface

Characteristic	Local process I/O interface	LIF message interface
Information Representation	Unique, determined by the given physical interface device	Uniform within the whole cluster
Coding	Analog or digital, unique	Digital, uniform codes
Time-base	Dense	Sparse
Interconnection Pattern	One-to-one	One-to-many
Coupling	Tight, determined by the specific hardware requirements and the I/O protocol of the connected device	Weaker, determined by the LIF message communication protocol

Fig. 4.8 Standardized LIF versus concrete MMI



Example: In the external world of a process plant the value of the process variable *temperature* may be coded in an analog 4–20 mA sensor signal, where 4 mA means 0% and 20 mA means 100% of the selected measurement range, i.e., between 0° and 100°C. This analog representation must be converted to the standard representation of temperature that has been defined in the architectural style of the given cluster, which might be degrees *Kelvin*.

Example: Let us look at an important interface, the man–machine interface (MMI), in order to distinguish between the *local interface* and the *LIF message interface* of a gateway MMI component (Fig. 4.8). At the level of architectural modeling, we are not interested in the representational details of the local external world interface, but only in the *semantic content* and temporal properties of the message variables at the *LIF message interface*. An important message is sent to the MMI component. It is somehow relayed to the operator’s mind. A response message from the operator is expected within a given time interval at the *LIF message interface*. All intricate issues concerning the representation of the information at the graphic user interface (GUI) of the operator terminal are irrelevant from the point of view of architecture modeling of the interaction patterns between the operator and the cluster. If the purpose of our model were the study of human factors governing the specific man–machine interaction, then the form and attributes of the information representation at the GUI (e.g., shape and placement of symbols, color, and sound) would be relevant and could not be disregarded.

A gateway component can link two different clusters that have been designed by two different organizations using two different architectural styles. Which one of the two interfaces of such a gateway component is considered the LIF and which one is the local interface depends on the view taken. As already mentioned, if we change the view, the LIF becomes the local interface and the local interface becomes the LIF.

4.5.3 Standardized Message Interface

To improve the compatibility between components designed by different manufacturers, to enhance the interoperability of devices, and to avoid property mismatches, some international standard organizations have attempted to standardize message interfaces. An example of such a standardization effort is the SAE J 1587 Message Specification. The Society of Automotive Engineers (SAE) has standardized

the message formats for heavy-duty vehicle applications in the J 1587 Standard. This standard defines message names and parameter names for many data elements that occur in the application domain of heavy vehicles. Besides data formats, the range of the variables and the update frequencies are also covered by the standard.

4.6 Linking Interface Specification

As noted in Sect. 4.1.1, the *timed sequence of messages* that a component exchanges across an interface with its environment defines the *behavior* of the component at that interface [Kop03]. The *interface behavior* is thus determined by the properties of all messages that cross an interface. We distinguish three parts of an interface specification: (1) the *transport specification* of the messages, (2) the *operational specification* of the messages, and the (3) the *meta-level specification* of the messages.

The *transport specification* describes all properties of a message that are needed to transport the message from the sender to the receiver(s). The transport specification covers the addressing and temporal properties of a message. If two components are linked by a communication system, the transport specification suffices to describe the requested services from the communication system. The communication system is *agnostic* about the contents of the data field of a message. For the communication system it does not matter whether the data field contains multimedia data, such as voice or video, numerical data or any other data type.

Example: The Internet provides a defined message transport service between two end systems, not knowing what types of digital data are transported.

In order to be able to interpret the data field of a message at the end points of the communication, we need the *operational* and the *meta-level specification*. The *operational specification* informs about the syntactic structure of the message that is exchanged across the LIF and establishes the *message variables*. Both, the transport and the operational specification must be *precise* and *formal* to ensure the *syntactic interoperability* of components. The *meta-level specification* of a LIF assigns meaning to the *message variable names* introduced by the operational specification. It is based on an interface model of the user environment. Since it is impossible to formalize all aspects of a real-world user environment, the meta-level specification will often contain natural language elements, which lack the precision of a formal system. Central concepts of the application domains and applications can be specified using *domain specific ontologies*.

4.6.1 Transport Specification

The *transport specification* contains the information needed by the communication system to transport a message from a sender to the receiver(s). An interface contains

a number of ports where the messages destined for this interface arrive or where messages are placed for sending. The following attributes must be contained in the transport specification:

- Port address and direction
- Length of the data field of the message
- Type of message (e.g., *time-triggered* or *event-triggered* or *data stream*)
- Cycle of a time-triggered message
- Queue depth for an event-triggered message or a data stream

It is a design decision whether these attributes are linked with the message or are associated with the port where the message is handled.

As noted above, the transport specification must contain the information about the temporal properties of a message. For time-triggered messages, the temporal domain is precisely specified by the cycle that is associated with every time-triggered message. For event-triggered messages, the temporal specification is more difficult, particularly if event-triggered messages can arrive in bursts. In the long run, the message arrival rate must not be larger than the message consumption rate by the receiver, since an event-triggered message must conform to the *exactly once semantics* (see Sect. 4.3.3). In the short run, an arriving message burst can be buffered in the receiver queue until the queue is full. The specification of the proper queue length for bursty event-triggered messages is very important.

4.6.2 Operational Specification

From the point of view of communication, the data field of an arriving message can be considered as an *unstructured bit vector*. At the endpoints of the communication, the operational specification determines how this bit vector must be structured into *message variables*. A *message variable* is a syntactic unit that consists of a fixed part and a variable part (see Sect. 2.2.4). The information about how the data field of a message is structured in syntactic units is contained in a *message-structure declaration (MSD)*. The MSD contains the *message variable names* (i.e., the fixed part of the message variable) that point to the relevant concepts on one side and, on the other side, specifies which part of the unstructured bit vector denotes *the value (the variable part) of a message variable*. In addition to the structure information, the MSD may contain *input assertions* for checking the validity of incoming data (e.g., to test if the data is within a permitted data domain that the receiving component is capable to handle) and *output assertions* for checking outgoing data. An incoming data element that passes the input assertion is called a *permitted data element*. An outgoing data element that passes the output assertion is called a *checked data element*. The formalism used for specifying the data structures and the assertions in the MSD depends on the available programming environment.

In many real-time systems the MSD is static, i.e., it does not change over the lifetime of the system. In these systems, for performance reasons, the MSD is not transmitted in the message but stored in the memories of the communicating partners where the data fields must be handled.

The link between the *unstructured bit vector* arriving at a port and the associated MSD can be established by different means:

- The *MSD name* is assigned to the input port name. In this case only a single message type can be accepted at a port.
- The *MSD name* is contained in the data field of the message. In this case different message types can be received at the same port. This approach is followed in CAN, (see Sect. 7.3.2).
- The *MSD name* is assigned to the cyclic arrival instant of a time-triggered message. In this case different message types can be received at the same port, without the need to store the MSD name in the data field of the message. This approach is followed in TTP see Sect. 7.5.1.
- The *MSD name* is stored in a server that can be accessed by the receiver of a message. This approach is followed in CORBA [Sie00].
- The MSD itself is part of the message. This is the most flexible arrangement, at the cost of having to send the full MSD in every message. This approach is followed in service-oriented architectures (SOA) [Ray10].

4.6.3 Meta-Level Specification

The meta-level LIF specification assigns a meaning to the message variables exchanged between two communicating LIFs at the operational level and thus establishes *semantic interoperability*. It thus bridges the gap between the syntactic units and the user's mental model of the service provided at the interface. Central to this meta-level specification is the LIF service model. The LIF service model defines the concepts that are associated with the *message variable names* contained in the operational specification. These concepts will be qualitatively different for *closed components* and *open components* (see Sect. 4.4.5).

The LIF service model for a *closed* component can be formalized, since a closed component does not interact with the external environment. The relationship between the LIF inputs and LIF outputs depends on the discrete algorithms implemented within the closed component. There is no input from the external environment that can bring unpredictability into the component behavior. The sparse time-base within a cluster is discrete and supports a consistent temporal order of all events.

The LIF service model for an *open* component is fundamentally different since it must encompass the inputs from the external environment, the local interfaces of the component in its interface specification. *Without knowing the context of use of an open component, only the operational specification of an open component can be*

provided. Since the external physical environment is not rigorously definable, the interpretation of the external inputs depends on human understanding of the natural environment. The concepts used in the description of the LIF service model must thus fit well with the accustomed concepts within a user's *internal conceptual landscape* (see Sect. 2.2); otherwise the description will not be understood.

The discussion that follows focuses on LIFs of open components, since the systems we are interested in must interact with the external environment. The LIF service model of an open component must meet the following requirements:

- *User orientation*. Concepts that are familiar to a prototypical user must be the basic elements of the LIF service model. For example, if a user is expected to have an engineering background, terms and notations that are *common knowledge* in the chosen engineering discipline should be used in presenting the model.
- *Goal orientation*. A user of a component employs the component with the intent to achieve a goal, i.e., to contribute to the solution of her/his problem. The relationship between user intent and the services provided at the LIF must be exposed in the LIF service model.
- *System view*. A LIF service user (the system architect) needs to consider the system-wide effects of an interaction of the component with the external physical environment, i.e., effects that go beyond the component. The LIF service model is different from the model describing the algorithms implemented within a component, since these algorithms are within the component's boundaries.

Example: Let us analyze the simple case of a variable that contains a temperature. As any variable, it consists of the two parts, the *static variable name* and the *dynamic variable value*. The MSD contains the *static variable name* (let us assume the variable is named *Temperature-11*) and the position where the *dynamic variable value* is placed in an arriving bit stream of the message. The meta-level specification explains the meaning of *Temperature-11* (see also the examples in Sect. 2.2.4).

4.7 Component Integration

A component is a self-contained validated unit that can be used as a building block in the construction of larger systems. In order to enable a straightforward composition of a component into a cluster of components, the following four *principles of composability* should be observed.

4.7.1 Principles of Composability

1. *Independent Development of Components*: The architecture must support the precise specification of the linking interface (LIF) of a component in the domains

of value and time. This is a necessary prerequisite for the independent development of components on one side and the reuse of existing components that is based solely on their LIF specification on the other side. While the operational specification of the value domain of interacting messages is *state-of-the-art* in embedded system design, the temporal properties of these messages are often ill defined. Many of the existing architectures and specification technologies do not deal with the temporal domain with the appropriate care. Note that the transport specification and the operational LIF specification are independent of the context of use of an open component, while the meta-level LIF specification of an open component depends on the context of use. *Interoperability* of open components is thus not the same as *interworking* of open component, since the latter assumes the compatibility of the meta-level specifications.

2. *Stability of Prior Services*: The *stability of prior services principle* states that the services of a component that have been validated in isolation (i.e., prior to the integration of the component into the larger system) remain intact after the integration (see the example in Sect. 4.4.1).
3. *Non-Interfering Interactions*: If there exist two disjoint subgroups of cooperating components that share a common communication infrastructure, then the communication activities within one subgroup must not interfere with the communication activities within the other subgroup. If this principle is not satisfied, then the integration within one component-subgroup will depend on the proper behavior of the other (functionally unrelated) component-subgroups. These global interferences compromise the composability of the architecture.

Example: In a communication system where a single communication channel is shared by all components on a *first-come first-serve basis*, a *critical instant* is defined as an instant, when all senders start sending messages simultaneously. Let us assume that in such a system ten components are to be integrated into a cluster. A given communication system is capable to handle the critical instant if eight components are active. As soon as the ninth and tenth component are integrated, sporadic timing failures are observed.

4. *Preservation of the Component Abstraction in the Case of Failures*: In a composable architecture, the introduced abstraction of a component must remain intact, even if a component becomes faulty. It must be possible to diagnose and replace a faulty component without any knowledge about the component internals. This requires a certain amount of redundancy for error detection within the architecture. This principle constrains the implementation of a component, because it restricts the implicit sharing of resources among components. If a shared resource fails, more than one component can be affected by the failure.

Example: In order to detect a faulty component that acts like a *babbling idiot*, the communication system must contain information about the permitted temporal behavior of every component. If a component fails in the temporal domain, the communication system cuts off the component that violates its temporal specification, thus maintaining the timely communication service among the correct components.

4.7.2 Integration Viewpoints

In order to bring an understandable structure into a large system, it makes sense to view – as seen from an integration viewpoint – a (original) cluster of components as a single gateway component. The integration viewpoint establishes a new cluster that consists of the respective gateway components of the original clusters. Viewed from an original cluster, the external (local) interface of the gateway become the LIFs of the new cluster, while, viewed from the new cluster, the LIF of the gateway to the original cluster is the local interface of the new cluster (see Sect. 4.5.2). The gateways to the new cluster make only those information items available to the new cluster that are of relevance for the operation of the new cluster.

Example: Figure 4.1 depicts a cluster of components that form the control system within an automobile. The vehicle-to-vehicle gateway component (the right lower component in Fig. 4.1) establishes a wireless link to other vehicles. In this example, we distinguish the following two levels of integration: (1) the integration of components into the cluster depicted in Fig. 4.1 and (2) the integration of a car into a *dynamic system of cars* that is achieved via the car-to-car (C2C) gateway component. If we look at the integration of components within the cluster of Fig. 4.1, then the communication network interface (CNI) of the C2C gateway component is the *cluster LIF*. From the C2C communication viewpoint, the cluster LIF is the (unspecified) local interface of the C2C gateway component (see also the last paragraph of Sect. 4.5.2).

The hierarchical composition of components and clusters that leads to *distinct integration levels* is an important special case of the integration of components. Multi-levelness is an important organizing principle in large systems. At the lowest integration level primitive components (i.e., components that are considered to be atomic units and are not composed any further) are integrated to form a cluster. One distinct component of this cluster is a gateway component that forms, together with distinct gateway components of other clusters a *higher level cluster*. This process of integration can be continued recursively to construct a hierarchical system with distinct levels of integration (see also Sect. 14.2.2 on the recursive integration of components in the time-triggered architecture).

Example: In the GENESYS [Obm09, p. 44] architecture, three integration levels are introduced. At the lowest level, the chip level, the components are IP cores of an MPSoC that interact by a network on chip. At the next higher level, chips are integrated to form a device. At the third integration level devices are integrated to form *closed* or *open* systems. A *closed* system is a system in which the subsystems that form the systems are known *a priori*. In an open system subsystems (or devices) join and leave the system dynamically, leading to a system-of-systems.

4.7.3 System of Systems

There are two reasons for the rising interest in *systems-of-systems*: (1) the realization of new functionality and (2) the control of the complexity growth of large

systems caused by their continuous evolution. The available technology (e.g., the Internet) makes it possible to interconnect *independently developed systems (legacy systems)* to form new *system-of-systems (SoS)*. The integration of different *legacy systems* into an SoS promises more efficient economic processes and improved services.

The continuous adaptations and modifications that are necessary to keep the services of a large system relevant in a dynamic business environment brings about a growing complexity that is hard to manage in a monolithic context [Leh85]. One promising technique to attack this complexity problem is to break a single large monolithic system up into a set of nearly autonomous constituent systems that are connected by well-defined message interfaces. As long as the *relied-upon properties* at these message interfaces meet the user intentions, the internal structure of the constituent systems can be modified without any adverse effect on the global system level services. Any modification of the *relied-upon properties* at the message interfaces is carefully coordinated by a coordination-entity that monitors and coordinates the system evolution. The SoS technology is thus introduced to get a handle on the complexity growth, caused by the necessary evolution of large systems, by introducing structure and applying the simplification principles of *abstraction, separation of concerns, and observability of subsystem interactions* (see Sect. 2.5).

The distinction between a *system of sub-systems* and a *system of systems* is thus based on the degree of autonomy of the constituent systems [Mai98]. We call a monolithic system made out of subsystems that are designed according to a master plan and are in the *sphere of control* of a single development organization a *system of sub-system*. If the *systems* that cooperate are in the *sphere of control* of different development organization we speak of a *system of (constituent) systems*. Table 4.3 compares the different characteristics of a *monolithic system* versus a *system of systems*. The interactions of the autonomous constituent systems can evoke planned or unanticipated emergent behavior, e.g., a *cascade effect* [Fis06], that must be detected and controlled (see also Sect. 2.4).

In many distributed real-time applications it is not possible to bring *temporally accurate real-time information* to a central point of control within the available time interval between the *observation of the local environment* and the need to *control the local environment*. In these applications central control by a monolithic control system is not possible. Instead, the autonomous distributed controllers must cooperate to achieve the desired effects.

Example: It is not possible to control the movement of the cars in an open road system, where cyclists and pedestrians interfere with the traffic flow of the cars, by a monolithic central control system because the amount and timeliness of the real-time information that must be transported to and processed by the central control system is not manageable within the required response times. Instead, each car performs autonomous control functions and cooperates with other cars in order to maintain an efficient flow of traffic. In such a system a *cascade effect* of a traffic jam can occur due to emergent behavior if the traffic density increases beyond a tipping point.

Any ensemble of constituent systems that form an SoS must agreed on a shared purpose, a chain of trust, and a shared ontology on the semantic level. These global

Table 4.3 Comparison of a *monolithic system* and a *system of systems*

Monolithic system	System of systems (SoS)
<i>Sphere of control</i> and system responsibility within a single development organization. Subsystems are obedient to a central authority.	Constituent systems are in the sphere of control of different development organizations. Subsystems are autonomous and can only be influenced, but not controlled, by other subsystems.
The architectural styles of the subsystems are aligned. Property mismatches are the exception.	The architectural styles of the constituent systems are different. Property mismatches are the rule, rather than the exception.
The LIFs that effectuate the integration are controlled by the responsible system organization.	The LIFs that effectuate the integration are established by international standard organizations and outside the control of a single system supplier.
Normally hierarchical composition that leads to levels of integration.	Normally the interactions among the constituent systems follow a mesh network structure without clear integration levels.
Subsystems are designed to interact in order to achieve the system goal: <i>Integration</i> .	Constituent systems have their own goals that are not necessarily compatible with the SoS goal. Voluntary cooperation of systems to achieve a common purpose: <i>Interoperation</i> .
Evolution of the components that form the subsystems is coordinated.	Evolution of the constituent systems that form the SoS is uncoordinated.
Emergent behavior controlled.	Emergent behavior often planned, but sometimes unanticipated.

properties must be established at the meta-level and are subject of a carefully managed continuous evolution. A new entity must be established at the meta-level that monitors and coordinates the activities of the constituent systems in order that the shared purpose can be achieved.

An important characteristic of an SoS is the independent development and uncoordinated evolution of the constituent systems (Table 4.3). The focus in SoS design is on the linking interface behavior of the monolithic systems. The monolithic system themselves can be heterogeneous. They are developed according to different architectural styles by different organizations. If the monolithic systems are interconnected via open communication channels, then the topic of security is of utmost concern, since an outside attacker can interfere with the system operation, e.g., by executing a *denial-of-service* attack (see Sect. 6.2.2).

[Sel08, p. 3] discusses two important properties of an *evolutionary architecture*: (1) The complexity of the overall framework does not grow as constituent systems are added or removed and (2) a given constituent system does not have to be reengineered if other constituent systems are added, changed, or removed. This implies a precise specification and continuous revalidation of the relied upon interface properties (in the domains of value and time) of the constituent systems. The evolution of a constituent system will have no adverse effect on the overall behavior if the *relied-upon interface properties* of this constituent system are

not modified. Since the precise specification of the temporal dimension of the relied-upon interface properties requires a time-reference, the availability of a synchronized global time in all constituent systems of a large SoS is helpful, leading to a *time-aware architecture* (TAA, see Sect. 14.2.5). Such a global time can be established by reference to the global GPS signals (see Sect. 3.5). We call an SoS where all constituent systems have access to a synchronized global time a *time-aware SoS*.

The preferred interconnection medium for the construction of *systems of systems* is the Internet, leading to the Internet of Things (IoT). Chapter 13 is devoted to the topic of the *Internet of Things*.

Points to Remember

- A real-time component consists of a *design* (e.g., the software), an *embodiment* (e.g., the hardware, including a processing unit, memory, and an I/O interface), and a real-time clock that makes the component aware of the progression of real-time.
- The *timed sequence of output messages* that a component produces at an interface with its environment is the *behavior* of the component at that interface. The *intended behavior* is called the *service*. An unintended behavior is called a *failure*.
- *Temporal control* is concerned with determining the instants in the domain of real time when tasks must be activated while *logical control* is concerned with the control flow *within* a task.
- *Synchronous programming languages* distinguish cleanly between temporal control, which is related to the progression of real time, and logical control, which is related to execution time.
- A cycle, characterized by its *period* and *phase*, is associated with every time-triggered activity.
- At a given instant, the *state of a component* is defined as a data structure that contains the information of the past that is relevant for the future operation of the component.
- In order to enable the dynamic reintegration of a component into a running system, it is necessary to design periodic *reintegration instants* into the behavior, where the state at the reintegration instant is called the *ground state* of the component.
- A *message* is an atomic data structure that is formed for the purpose of communication, i.e., data transmission and synchronization, among components.
- *Event information* conveys the *difference* of the previous state observation and the current state observation. Messages that contain event information must adhere to the *exactly-once* semantic.
- *State messages* support the *principle of independence* because sender and receiver can operate at different (independent) rates and there is no danger of buffer overflow.

- In real-time systems, the *information pull strategy* should be followed whenever possible.
- *Elementary interfaces* are *inherently simpler* than composite interfaces, because there is *no dependency* of the behavior of the sender on the behavior of the receiver.
- The *services* of a component are offered at its *cluster LIF* to the other components of the cluster. The cluster LIF is an *operational message-based* interface that is relevant for the integration of components into the cluster. The detailed structure, naming, and access mechanisms of the local interface of a component is *intentionally left unspecified* at its cluster LIF.
- Every system is developed according to an *architectural style*, i.e., a set of adopted rules and conventions for the conceptualization, representation of data, naming, programming, interaction of components, semantics of the data, and many more.
- Whenever a communication channel links two systems developed by two different organizations, it is highly probable that some of the properties of the messages that are exchanged across this channel are in disagreement because of the differences in architectural styles.
- A *gateway component* resolves property mismatches and exposes the external-world information in the form of cluster-standard messages at the cluster LIF of the gateway components.
- We distinguish between three parts of a LIF specifications: (1) the *transport specification* of the messages (2) the *operational specification* of the messages and the (3) the *meta-level specification* of the messages.
- Only the *operational specification* of an open component can be provided without knowing the context of use of the open component.
- The information on how the data field of a message is structured into syntactic units is contained in a *message-structure declaration (MSD)*. The MSD establishes the *message variable names* (i.e., the fixed part of the message variable) that point to the respective concepts and specify which part of the unstructured bit vector denotes the *variable part of a message variable*.
- The four principles of composability are (1) independent development of components, (2) stability of prior services, (3) non-interfering interactions, and (4) preservation of the component abstraction in case of failures.
- Multi-levelness is an important organizing principle in large systems.
- The distinction between a *system of sub-systems* and a *system of systems* is based more on organizational than on technical grounds.

Bibliographic Notes

The presented real-time model of computation has been developed over the past 25 years and is documented in a number of publications, starting with *The Architecture of Mars* [Kop85] and further in the following publications: *Real-time Object Model* [Kim94], the *Time-Triggered Model of Computation* in [Kop98], *Elementary versus Composite Interfaces in Distributed Real-Time Systems* [Kop99], and *Periodic Finite State Machines* [Kop07].

Review Questions and Problems

- 4.1 How is a *real-time system component* defined? What are elements of a component? How is the behavior of a component specified?
- 4.2 What are the advantages of separating computational components from the communication infrastructure? List some of the consequences of this separation.
- 4.3 What is the difference between *temporal control* and *logical control*?
- 4.4 What is the definition of the *state* of a real-time system? What is the relationship between *time* and *state*? What is the *ground state*? What is a database component?
- 4.5 What is the difference between *event information* and *state information*? What is the difference in the handling of an *event message* from the handling a *state message*?
- 4.6 List and describe the properties of the four interfaces of a component? Why are the local interfaces of a component intentionally left *unspecified* at the architectural level?
- 4.7 What are the differences between an *information push interface* and an *information pull interface*? What are the differences between an *elementary interface* and a *composite interface*?
- 4.8 What do we mean by the term *architectural style*? What is a *property mismatch*?
- 4.9 What are the characteristics of a local process I/O interface and the LIF message interface?
- 4.10 What is the role of a gateway component?
- 4.11 What are the three parts of a linking interface specification?
- 4.12 What is the *message-structure declaration (MSD)*? How do we associate the MSD with the bit-vector contained in a message?
- 4.13 List the *four principles of composability*?
- 4.14 What is an integration level? How many integration levels are introduced in the GENESYS architecture?
- 4.15 Assume that the pressures p_1 and p_2 between the first two pairs of rolls in Fig. 1.9 are measured by the two controller nodes and sent to the man-machine interface (MMI) node for verifying the following alarm condition:

```

when( $p_1 < p_2$ )
    then everything ok
    else raise pressure alarm;

```

The rolling mill is characterized by the following parameters: maximum pressure between the rolls of a stand = $1,000 \text{ kp cm}^{-2}$ [kp is kilopond], absolute pressure measurement error in the value domain = 5 kp cm^{-2} , maximum rate of change of the pressure = $200 \text{ kp cm}^{-2} \text{ s}^{-1}$. It is required that the error due to the imprecision of the points in time when the pressures are measured at the different rolls should be of the same order of magnitude as the

measurement error in the value domain, i.e., 0.5% of the full range. The pressures must be *continuously* monitored, and the first alarm must be raised by the alarm monitor within 200 ms (at the latest) after a process has possibly left the normal operating range. A second alarm must be raised within 200 ms after the process has definitely entered the alarm zone.

1. Assume an event-triggered architecture. Each node contains a local real-time clock, but no global time is available. The minimum time d_{min} for the transport of a single message by the communication system is 1 ms. Derive the temporal control signals for the three tasks.
2. Assume a time-triggered architecture. The clocks are synchronized with a precision of 10 μ s. The time-triggered communication system is characterized by a TDMA round of 10 ms. The time for the transport of a single message by the communication system is 1 ms. Derive the temporal control signals for the three time-triggered tasks.
3. Compare the solutions of 4.16.(a) and 4.16.(b) with respect to the generated computational load and the load on the communication system. How sensitive are the solutions if the parameters, e.g., the jitter of the communication system or the duration of the TDMA round, are changed?

Chapter 5

Temporal Relations

Overview The behavior of a real-time cluster must be based on timely information about the state of its physical environment and the state of other cooperating clusters. Real-time data is *temporally accurate* for a limited real-time interval only. If real-time data is used outside this application specific time interval, the system will fail. It is the objective of this chapter to investigate the temporal relations among state variables in the different parts of a cyber-physical system.

In this chapter, the notions of a *real-time (RT) entity*, a *real-time (RT) image*, and a *real-time (RT) object* are introduced and the concept of *temporal validity* of an RT image is established. The temporal validity of the RT image can be extended by state estimation. A real-time clock is associated with every RT object. The object's clock provides periodic temporal control signals for the execution of the object procedures, particularly for state estimation purposes. The granularity of the RT object's clock is aligned with the dynamics of the RT entity in the controlled object that is associated with the RT object. The notions of parametric and phase-sensitive observations of RT entities are introduced, and the concept of *permanence* of an observation is discussed. The duration of the action delay, which is the time interval between the transmission of a message and the instant when this message becomes *permanent*, is estimated.

The final section of this chapter is devoted to an elaboration of the concept of determinism. Determinism is a desired property of a computation that is needed if fault-tolerance is to be achieved by the replication of components. Determinism is also helpful for testing and for understanding the operation of a system. A *set of replicated RT objects is replica determinate* if the objects visit the same state at approximately the same future point in time. The main causes for a loss of determinism are failures that the fault-tolerance mechanisms are intended to mask and *non-deterministic design constructs* which must be avoided in the design of deterministic systems.

5.1 Real-Time Entities

A *real-time (RT) entity* is a *state variable* of relevance for the given purpose. It is located either in the environment of the computer system or in the computer system itself. Examples of RT entities are the flow of a liquid in a pipe, the setpoint of a control loop that is selected by the operator, or the intended position of a control valve. An RT entity has static attributes that do not change during the lifetime of the RT entity, and dynamic attributes that change with the progression of real-time. Examples of static attributes are the name, the type, the value domain, and the maximum rate of change. The value set at a particular instant is the most important dynamic attribute. Another example of a dynamic attribute is the rate of change at a chosen instant.

5.1.1 Sphere of Control

Every RT entity is in the sphere of control (SOC) of a subsystem that has the authority to set the value of the RT entity [Dav79]. Outside its SOC, the RT entity can only be observed, but the *semantic content* of the RT entity cannot be modified. At the chosen level of abstraction, syntactic transformations of the representation of the value of an RT entity that do not change its *semantic content* (see Sect. 2.2.4) are disregarded.

Example: Figure 5.1 shows another view of Fig. 1.8 and represents the small control system that controls the flow of a liquid in a pipe according to a setpoint selected by the operator. In this example, there are three RT entities involved: the *flow in the pipe* is in the SOC of the controlled object, the *setpoint* for the flow is in the SOC of the operator, and the *intended position* of the control valve is in the SOC of the computer system.

5.1.2 Discrete and Continuous Real-Time Entities

An RT entity can have a discrete value set (*discrete RT entity*) or a continuous value set (*continuous RT entity*). If the timeline proceeds from left to right, then the

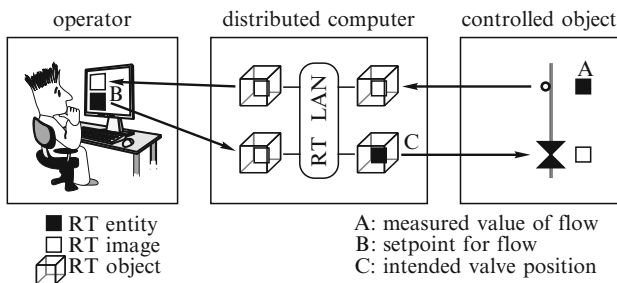
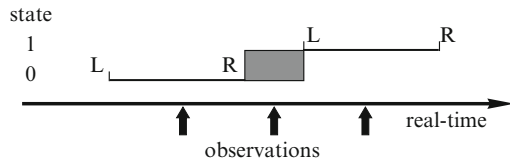


Fig. 5.1 RT entities, RT images, and RT objects

Fig. 5.2 Discrete RT entity

value set of a discrete RT entity is constant during an interval that starts with a left event (L_event) and ends with a right event (R_event) – see Fig. 5.2.

In the interval between an R_event and the next L_event , the set of values of a discrete RT entity is undefined. In contrast, the set of values of a continuous RT entity is always defined.

Example: Consider a garage door. Between the defined states specified by *door closed* and *door open*, there are many intermediate states that can be classified neither as *door open* nor as *door closed*.

5.2 Observations

The information about the state of an RT entity at a particular instant is captured by the notion of an *observation*. An observation is an *atomic data structure*

$$Observation = \langle Name, t_{obs}, Value \rangle$$

consisting of the name of the RT entity, the instant when the observation was made (t_{obs}), and the observed value of the RT entity. A continuous RT entity can be observed at any instant while the observation of a discrete RT entity gives a meaningful value only in the interval between a L_event and an R_event (see Fig. 5.2).

We assume that an *intelligent sensor node* is associated with a *physical sensor* to capture the physical signal, to generate the timestamp, and to transform the physical signal to meaningful digital technical units. An observation should be transported in a single message from this sensor node to the rest of the system because the message concept provides for the needed atomicity of an observation message.

5.2.1 Untimed Observation

In a distributed system without global time, a timestamp can only be interpreted within the scope of the node that created the timestamp. The timestamp of a sender that made an observation is thus meaningless at the receiver of the observation message if no global time is available. Instead, the time of arrival of an untimed observation message at the receiver node is often taken to be the time of observation t_{obs} . This timestamp is imprecise because of the delay and the jitter between the

instant of observation and the arrival instant of the message at its destination. In a system with a significant jitter of the execution time of the communication protocol (in comparison to the median execution time) and without access to a global time-base it is not possible to determine the instant of observation of an RT entity precisely. This imprecision of time measurement can reduce the quality of the observation (see Fig. 1.6).

5.2.2 Indirect Observation

In some situations, it is not possible to observe the value of an RT entity directly. Consider, for example, the measurement of the internal temperature within a slab of steel. This internal temperature (the value of the RT entity) must be measured indirectly.

The three temperature sensors T_1 , T_2 , and T_3 measure the change of temperature of the surface (Fig. 5.3) over a period of time. The value of the temperature T within the slab and the instant of its relevance must be inferred from these surface measurements by using a mathematical model of heat transfer.

5.2.3 State Observation

An observation is a *state observation* if the value of the observation contains the state of the RT entity. The time of the state observation refers to the point in real-time when the RT entity was sampled (observed). Every reading of a state observation is self-contained because it carries an *absolute value*. Many control algorithms require a sequence of equidistant state observations, a service provided by periodic time-triggered readings.

The semantics of state observations matches well with the semantics of the state messages introduced in Sect. 4.3.4. A new reading of a state observation replaces the previous readings because clients are normally interested in the most recent value of a state variable.

5.2.4 Event Observation

An *event* is an occurrence (a state change) that happens at an instant. Because an observation is also an event, it is not possible to observe an event in the controlled

Fig. 5.3 Indirect measurement of an RT entity

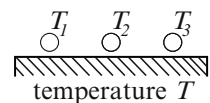
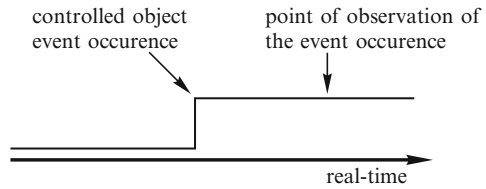


Fig. 5.4 Observation of an event



object directly. It is only possible to observe the *consequences of the controlled object's event* (Fig. 5.4), i.e., the subsequent state. An observation is an *event observation* if it contains the *change in value* between the *old* and the *new* states. The instant of the event observation denotes the best estimate of the instant of the occurrence of this event. Normally, this is the time of the L-event of the new state.

There are a number of problems with event observations:

1. Where do we get the precise time of the event occurrence? If the event observation is event-triggered, then, the time of event occurrence is assumed to be the rising edge of the interrupt signal. Any delayed response to this interrupt signal will cause an error in the timestamp of the event observation. If the event observation is time-triggered, then, the time of *event occurrence* can be at any point within the sampling interval.
2. Since the value of an event observation contains the *difference* between the old state and the new state (and not the absolute state), the loss or duplication of a single event observation causes the loss of state synchronization between the state of the observer and the state of the receiver. From the point of view of reliability, event observations are more fragile than state observations.
3. An event observation is only sent if the RT entity changes its value. The latency for the detection of a failure (e.g., a *crash*) of the observer node cannot be bounded because the receiver assumes that the RT entity has not changed its value if no new event message arrives.

On the other hand, event observations are more *data-efficient* than state observations in the case where the RT entity does not change frequently.

5.3 Real-Time Images and Real-Time Objects

5.3.1 Real-Time Images

A real-time (RT) image is a *current* picture of an RT entity. An RT image is *valid* at a given instant if it is an accurate representation of the corresponding RT entity, both in the value and the time domains. The notion of *temporal accuracy* of an RT image will be discussed in detail in the next section. While an observation records a fact that remains valid forever (a statement about an RT entity that has been observed at a particular instant), the validity of an RT image is *time-dependent*

and thus invalidated by the progression of real-time. RT images can be constructed from up-to-date state observations or from up-to-date event observations. They can also be estimated by a technique called *state estimation* that will be discussed in Sect. 5.4.3. RT images are stored either inside the computer system or in the environment (e.g., in an actuator).

5.3.2 Real-Time Objects

A real-time (RT) object is a *container* within a node of the distributed computer system that holds an RT image or an RT entity [Kop90]. A real-time clock with a specified granularity is associated with every RT object. Whenever this object clock ticks, a temporal control signal is relayed to the object to activate an object procedure [Kim94].

Distributed RT Objects. In a distributed system, an RT object can be replicated in such a manner that every local site has its own version of the RT object to provide the specified service to the local site. The quality of service of a distributed RT object must conform to some specified consistency constraints.

Example: A good example of a distributed RT object is *global time*; every node has a local clock object that provides a synchronized time service with a specified precision Π (quality of service attribute of the internal clock synchronization). Whenever a process reads its local clock, it is guaranteed that a process running on another node that reads its local clock at the same instant will get a time value that differs by at most one tick.

Example: Another example of a distributed RT object is a *membership service* in a distributed system. A *membership service* generates consistent information about the state (operational or failed) of all nodes of the system at agreed instants (*membership points*). The length and the jitter of the interval between a membership point and the instant when the consistent membership information is known at the other nodes are *quality of service parameters* of the membership service. A responsive membership service has a small maximum delay between the instant of a relevant state change of a node (failure or join), and the instant at which all other nodes have been informed in a consistent manner of this state change.

5.4 Temporal Accuracy

Temporal accuracy denotes the temporal relationship between an *RT entity* and its associated *RT image*. Because an RT image is stored in an RT object, the temporal accuracy can also be viewed as a relation between an RT entity and an RT object.

5.4.1 Definition

The temporal accuracy of an RT image is defined by referring to the *recent history* of observations of the related RT entity. A recent history RH_i at time t_i is an ordered

set of instants $\{t_i, t_i - 1, t_i - 2, \dots, t_i - k\}$, where the length of the recent history, $d_{acc} = z(t_i) - z(t_i - k)$, is called the *temporal accuracy interval* or the *temporal accuracy* d_{acc} ($z(e)$ is the timestamp of event e generated by the reference clock z ; see Sect. 3.1.2). Assume that the RT entity has been observed at every instant of the recent history. An RT image is temporally accurate at the present time t_i if

$$\exists t_j \in RH_i : \text{Value (RT image at } t_i) = \text{Value (RT entity at } t_j)$$

The current value of a temporally accurate RT image is a member of the set of observations in the recent history of the corresponding RT entity. Because the transmission of an observation message from the observing node to the receiving node takes some amount of time, the RT image lags behind the RT entity (see Fig. 5.5).

Example: Let us assume, that the temporal accuracy interval of a temperature measurement is 1 min. An RT-image is temporally accurate if the value contained in the RT image has been observed at most a minute ago, i.e., it is still in the recent history of the corresponding RT-entity.

Temporal Accuracy Interval. The size of the admissible temporal accuracy interval d_{acc} is determined by the dynamics of the RT entity in the controlled object. The delay between the observation of the RT entity and the use of the RT image causes an *error(t)* of the RT image that can be approximated by the product of the gradient of the value v of the RT entity multiplied by the length of the interval between the instant of observation and the instant of its use (see also Fig. 1.6):

$$\text{error}(t) = \frac{dv(t)}{dt} (z(t_{use}) - z(t_{obs}))$$

If a temporally valid RT image is used, the worst-case error,

$$\text{error} = \left(\max_{\forall t} \frac{dv(t)}{dt} d_{acc} \right),$$

is given by the product of the maximum gradient and the temporal accuracy d_{acc} . In a balanced design, this worst-case error caused by the temporal delay is in the same order of magnitude as the worst-case measurement error in the value domain and is typically a fraction of a percentage point of the full range of the measured variable.

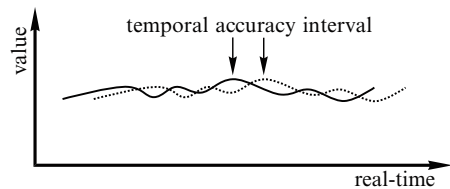


Fig. 5.5 Time lag between RT entity and RT image

If the RT entity changes its value quickly, a short accuracy interval must be maintained. Let us call t_{use} the instant when the result of a computation using an RT image is applied to the environment. For the result to be accurate, it must be based on a temporally accurate RT image, i.e.:

$$z(t_{obs}) \leq z(t_{use}) \leq (z(t_{obs}) + d_{acc})$$

where d_{acc} is the accuracy interval of the RT image. If this important condition is transformed, it follows that:

$$(z(t_{use}) - z(t_{obs})) \leq d_{acc}.$$

Phase-Aligned Transaction. Consider the case of an RT transaction consisting of the following *phase synchronized tasks*: the computational task at the sender (observing node) with a worst-case execution time $WCET_{send}$, the message transmission with a worst-case communication delay $WCCOM$, and the computational task at receiver (actuator node) with a worst-case execution time $WCET_{rec}$ (Fig. 5.6, see also Fig. 3.9). Such a transaction is called a *phase-aligned transaction*.

In such a transaction, the worst-case difference between the point of observation and the point of use,

$$(t_{use} - t_{obs}) = WCET_{send} + WCCOM + WCET_{rec},$$

is given by the sum of the worst-case execution time of the sending task, the worst-case communication delay, and the worst-case execution time of the receiving task that uses the data in the output of a setpoint to the actuator in the controlled object. If the temporal accuracy d_{acc} that is required by the dynamics of the application is smaller than this sum, the application of a new technique, *state estimation*, is inevitable in solving the temporal accuracy problem. The technique of state estimation is discussed in Sect. 5.4.3.

Example: Let us analyze the required temporal accuracy intervals of the RT images that are used in a controller of an automobile engine (Table 5.1) with a maximum rotational speed of 6,000 revolutions per minute (rpm). There is a difference of more than six orders of magnitude in the temporal accuracy intervals of these RT images. It is evident that the d_{acc} of the first data element, namely the position of the piston within the cylinder, requires the use of state estimation.

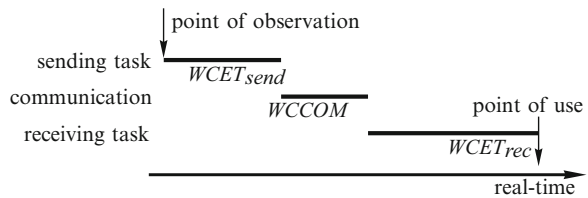


Fig. 5.6 Synchronized actions

5.4.2 Classification of Real-Time Images

Parametric RT Image. Assume that an RT image is updated periodically by a state observation message from the related RT entity with an update period d_{update} (Fig. 5.7) and assume that the transaction is phase aligned at the sender. If the temporal accuracy interval d_{acc} satisfies the condition

$$d_{acc} > (d_{update} + WCET_{send} + WCCOM + WCET_{rec}),$$

then we call the RT image *parametric* or *phase insensitive*.

A parametric RT image can be accessed at the receiver at any time without having to consider the phase relationship between the incoming observation message and the point of use of the data.

Example: The RT transaction that handles the position of the accelerator pedal (observation and preprocessing at sender, communication to the receiver, processing at the receiver and output to the actuator) takes an amount of time:

$$WCET_{send} + WCCOM + WCET_{rec} = 4 \text{ ms.}$$

Because the accuracy interval of this observation is 10 ms (Table 5.1), messages sent with periods less than 6 ms will make this RT image *parametric*.

If components are replicated, then care must be taken that all replicas access the same version of a parametric RT image, otherwise the *replica determinism* (see Sect. 5.6) will be lost.

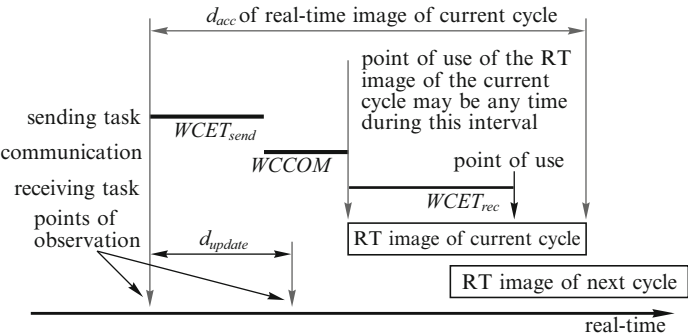


Fig. 5.7 Parametric real-time image

Table 5.1 Temporal accuracy intervals in engine control

RT image within computer	Max. change	Accuracy	d_{acc}
Position of piston within cylinder	6,000 rpm	0.1°	3 μ s
Position of accelerator pedal	100%/s	1%	10 ms
Engine load	50%/s	1%	20 ms
Temperature of the oil and the coolant	10%/min	1%	6 s